理链表的后一半元素时，当访问到链表的一个元素后，就从栈中弹出一个元素，对两个元素进行比较，若相等，则将链表中的下一个元素与栈中再弹出的元素进行比较，直至链表到尾。这时若栈是空栈，则得出链表中心对称的结论；否则，当链表中的一个元素与栈中弹出元素不等时，结论为链表非中心对称，结束算法的执行。

本题代码如下：

```
int dc(LinkList L,int n){
    int i;
    char s[n/2];                          //s 字符栈
    LNode *p=L->next;                     //工作指针 p，指向待处理的当前元素
    for(i=0;i<n/2;i++){                   //链表前一半元素入栈
        s[i]=p->data;
        p=p->next;
    }
    i--;                                  //恢复最后的 i 值
    if(n%2==1)                            //若 n 是奇数，后移过中心结点
        p=p->next;
    while(p!=NULL&&s[i]==p->data){        //检测是否中心对称
        i--;                              //i 充当栈顶指针
        p=p->next;
    }
    if(i==-1)                             //栈为空栈
        return  1;                        //链表中心对称
    else
        return  0;                        //链表中心不对称
}
```

算法先将“链表的前一半”元素（字符）入栈。当 n 为偶数时，前一半和后一半的个数相同；当 n 为奇数时，链表中心结点字符不必比较，移动链表指针到下一字符开始比较。比较过程中遇到不相等时，立即退出 while 循环，不再进行比较。

本题也可以先将单链表中的元素全部入栈，然后扫描单链表 L 并比较，直到比较到单链表 L 尾为止，但算法需要两次扫描单链表 L，效率不及上述算法高。

05.【解答】

两个栈共享向量空间，将两个栈的栈底设在向量两端，初始时，S1 栈顶指针为-1，S2 栈顶指针为 maxsize。两个栈顶指针相邻时为栈满。两个栈顶相向、迎面增长，栈顶指针指向栈顶元素。

本题代码如下：

```
#define  maxsize   100              //两个栈共享顺序存储空间所能达到的最多元素数，
                                    //初始化为 100
#define  elemtp int                 //假设元素类型为整型
typedef  struct{
    elemtp stack[maxsize];          //栈空间
    int   top[2];                   //top 为两个栈顶指针
}stk;
stk   s;                            //s 是如上定义的结构类型变量，为全局变量
```

本题的关键在于，两个栈入栈和出栈时的栈顶指针的计算。S1 栈是通常意义下的栈；而 S2 栈入栈操作时，其栈顶指针左移（减 1），出栈时，栈顶指针右移（加 1）。

此外，对于所有栈的操作，都要注意“入栈判满、出栈判空”的检查。

（1）入栈操作

代码如下：

```
int push(int i, elemtp x){
//入栈操作。i 为栈号，i=0 表示左边的 S1 栈，i=1 表示右边的 S2 栈，x 是入栈元素
```

```
    //入栈成功返回 1，否则返回 0
        if(i<0||i>1){
            printf("栈号输入不对");
            exit(0);
        }
        if(s.top[1]-s.top[0]==1){
            printf("栈已满\n");
            return  0;
        }
        switch(i){
            case 0: s.stack[++s.top[0]]=x; return  1; break;
            case 1: s.stack[--s.top[1]]=x; return  1;
        }
    }
```

（2）出栈操作

代码如下：

```
    elemtp pop(int i){
    //出栈算法。i 代表栈号，i=0 时为 S1 栈，i=1 时为 S2 栈
    //出栈成功返回出栈元素，否则返回-1
        if(i<0||i>1){
            printf("栈号输入错误\n");
            exit(0);
        }
        switch(i){
            case 0:
                if(s.top[0]==-1){
                    printf("栈空\n");
                    return  -1;
                }
                else
                    return  s.stack[s.top[0]--];
                break;
            case 1:
                if(s.top[1]==maxsize){
                    printf("栈空\n");
                    return  -1;
                }
                else
                    return  s.stack[s.top[1]++];
                break;
        }//switch
    }
```

3.2　队列

3.2.1　队列的基本概念

1．队列的定义

队列（Queue）简称队，也是一种操作受限的线性表，只允许在表的一端进行插入，而在表的另一端进行删除。向队列中插入元素称为入队或进队；删除元素称为出队或离队。这和我们日常生活中的排队是一致的，最早排队的也是最早离队的，其操作的特性是先进先出（First In First Out，FIFO），如图 3.5 所示。

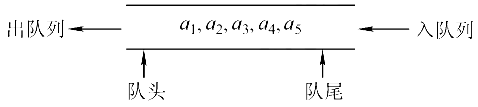


图 3.5 队列示意图

队头（Front）：允许删除的一端，也称队首。

队尾（Rear）：允许插入的一端。

空队列：不含任何元素的空表。

2．队列常见的基本操作

`InitQueue(&Q)`：初始化队列，构造一个空队列 `Q`。

`QueueEmpty(Q)`：判队列空，若队列 `Q` 为空返回 `true`，否则返回 `false`。

`EnQueue(&Q,x)`：入队，若队列 `Q` 未满，将 `x` 加入，使之成为新的队尾。

`DeQueue(&Q,&x)`：出队，若队列 `Q` 非空，删除队首元素，并用 `x` 返回。

`GetHead(Q,&x)`：读队首元素，若队列 `Q` 非空，则将队首元素赋值给 `x`。

需要注意的是，栈和队列是操作受限的线性表，因此不是任何对线性表的操作都可以作为栈和队列的操作。比如，不可以随便读取栈或队列中间的某个数据。

3.2.2 队列的顺序存储结构

1．队列的顺序存储

队列的顺序实现是指分配一块连续的存储单元存放队列中的元素，并附设两个指针：队首指针 `front` 指向队首元素，队尾指针 `rear` 指向队尾元素的下一个位置（不同教材对 `front` 和 `rear` 的定义可能不同，例如，可以让 `rear` 指向队尾元素、`front` 指向队首元素。对于不同的定义，出入队的操作是不同的，本节后面有一些相关的习题，读者可以结合习题思考）。

队列的顺序存储类型可描述为

```
#define MaxSize 50                  //定义队列中元素的最大个数
   typedef struct{
   ElemType data[MaxSize];          //用数组存放队列元素
   int front,rear;                  //队首指针和队尾指针
}SqQueue;
```

初始时：`Q.front=Q.rear=0`。

入队操作：队不满时，先送值到队尾元素，再将队尾指针加 1。

出队操作：队不空时，先取队首元素值，再将队首指针加 1。

图 3.6(a)所示为队列的初始状态，有 `Q.front==Q.rear==0` 成立，该条件可以作为队列判空的条件。

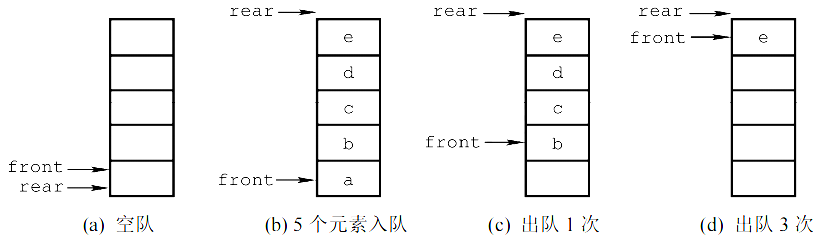


图 3.6 队列的操作

但能否用 `Q.rear==MaxSize` 作为队列满的条件呢？显然不能，图 3.6(d)中，队列中仅有一

个元素，但仍满足该条件。这时入队出现“上溢出”，但这种溢出并不是真正的溢出，在 data 数组中依然存在可以存放元素的空位置，所以是一种“假溢出”。

2．循环队列

上面指出了顺序队列“假溢出”的问题，这里引出循环队列的概念。将顺序队列臆造为一个环状的空间，即把存储队列元素的表从逻辑上视为一个环，称为循环队列。当队首指针 Q.front=MaxSize-1 后，再前进一个位置就自动到 0，这可以利用除法取模运算（%）来实现。

命题追踪 ▶ 特定条件下循环队列队头/队尾指针的初值（2011）

初始时：Q.front=Q.rear=0。

队首指针进 1：Q.front=(Q.front+1)%MaxSize。

队尾指针进 1：Q.rear=(Q.rear+1)%MaxSize。

队列长度：(Q.rear+MaxSize-Q.front)%MaxSize。

出入队时：指针都按顺时针方向进 1（如图 3.7 所示）。

命题追踪 ▶ 特定条件下循环队列队空/队满的判断条件（2014）

那么，循环队列队空和队满的判断条件是什么呢？显然，队空的条件是 Q.front==Q.rear。若入队元素的速度快于出队元素的速度，则队尾指针很快就会赶上队首指针，如图 3.7(d1)所示，此时可以看出队满时也有 Q.front==Q.rear。循环队列出入队示意图如图 3.7 所示。

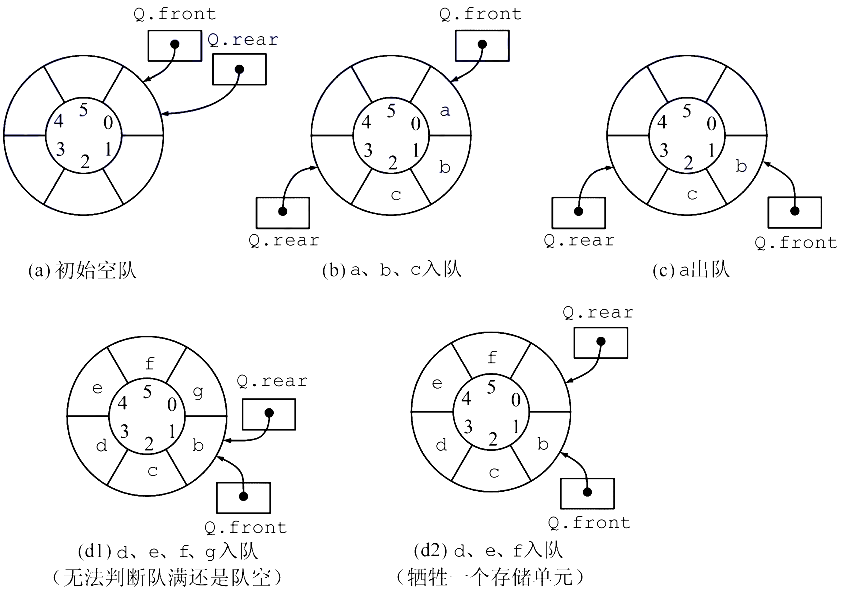


图 3.7　循环队列出入队示意图

为了区分是队空还是队满的情况，有三种处理方式：

1）牺牲一个单元来区分队空和队满，入队时少用一个队列单元，这是一种较为普遍的做法，约定以“队首指针在队尾指针的下一位置作为队满的标志”，如图 3.7(d2)所示。

队满条件：(Q.rear+1)%MaxSize==Q.front。

队空条件：Q.front==Q.rear。

队列中元素的个数：(Q.rear-Q.front+MaxSize)%MaxSize。

2）类型中增设 size 数据成员，表示元素个数。若删除成功，则 size 减 1，若插入成功，

则 size 加 1，队空时 Q.size==0；队满时 Q.size==MaxSize，两种情况都有 Q.front==Q.rear。

3）类型中增设 tag 数据成员，以区分是队满还是队空。删除成功置 tag=0，若导致 Q.front==Q.rear，则为队空；插入成功置 tag=1，若导致 Q.front==Q.rear，则为队满。

3．循环队列的操作

（1）初始化

```
void InitQueue(SqQueue &Q){
    Q.rear=Q.front=0;                    //初始化队首、队尾指针
}
```

（2）判队空

```
bool isEmpty(SqQueue Q){
    if(Q.rear==Q.front)                  //队空条件
        return true;
    else
        return  false;
}
```

（3）入队

```
bool EnQueue(SqQueue &Q,ElemType x){
    if((Q.rear+1)%MaxSize==Q.front)      //队满则报错
        return false;
    Q.data[Q.rear]=x;
    Q.rear=(Q.rear+1)%MaxSize;           //队尾指针加 1 取模
    return  true;
}
```

（4）出队

```
bool DeQueue(SqQueue &Q,ElemType &x){
    if(Q.rear==Q.front)                  //队空则报错
        return false;
    x=Q.data[Q.front];
    Q.front=(Q.front+1)%MaxSize;         //队首指针加 1 取模
    return  true;
}
```

3.2.3 队列的链式存储结构

1．队列的链式存储

命题追踪 ▶ 根据需求分析队列适合的存储结构（2019）

队列的链式表示称为链式队列，它实际上是一个同时有队首指针和队尾指针的单链表，如图 3.8 所示。队首指针指向队头结点，队尾指针指向队尾结点，即单链表的最后一个结点。

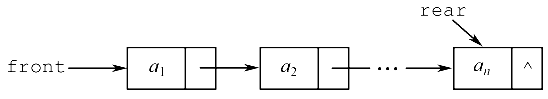


图 3.8 不带队头结点的链式队列

队列的链式存储类型可描述为

```
typedef struct LinkNode{                 //链式队列结点
    ElemType data;
    struct LinkNode *next;
}LinkNode;
```

```
typedef struct{                          //链式队列
    LinkNode *front,*rear;               //队列的队头和队尾指针
}LinkQueue;
```

不带头结点时，当 Q.front==NULL 且 Q.rear==NULL 时，链式队列为空。

命题追踪 ▶ 链式队列队空的判断（2019）

入队时，建立一个新结点，将新结点插入到链表的尾部，并让 Q.rear 指向这个新插入的结点（若原队列为空队，则令 Q.front 也指向该结点）。出队时，首先判断队是否为空，若不空，则取出队首元素，将其从链表中删除，并让 Q.front 指向下一个结点（若该结点为最后一个结点，则置 Q.front 和 Q.rear 都为 NULL）。

不难看出，不带头结点的链式队列在操作上往往比较麻烦，因此通常将链式队列设计成一个带头结点的单链表，这样插入和删除操作就统一了，如图 3.9 所示。

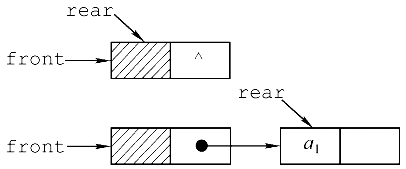


图 3.9　带队头结点的链式队列

用单链表表示的链式队列特别适合于数据元素变动比较大的情形，而且不存在队列满且产生溢出的问题。另外，假如程序中要使用多个队列，与多个栈的情形一样，最好使用链式队列，这样就不会出现存储分配不合理和“溢出”的问题。

2．链式队列的基本操作

命题追踪 ▶ 链式队列出队/入队操作的基本过程（2019）

（1）初始化

```
void InitQueue(LinkQueue &Q){   //初始化带头结点的链式队列
    Q.front=Q.rear=(LinkNode*)malloc(sizeof(LinkNode));//建立头结点
    Q.front->next=NULL;                 //初始为空
}
```

（2）判队空

```
bool IsEmpty(LinkQueue Q){
    if(Q.front==Q.rear)                 //判空条件
        return true;
    else
        return false;
}
```

（3）入队

```
void EnQueue(LinkQueue &Q,ElemType x){
    LinkNode *s=(LinkNode *)malloc(sizeof(LinkNode)); //创建新结点
    s->data=x;
    s->next=NULL;
    Q.rear->next=s;                     //插入链尾
    Q.rear=s;                           //修改尾指针
}
```

（4）出队

```
bool DeQueue(LinkQueue &Q,ElemType &x){
    if(Q.front==Q.rear)
        return  false;                  //空队
```

```
    LinkNode *p=Q.front->next;
    x=p->data;
    Q.front->next=p->next;
    if(Q.rear==p)
        Q.rear=Q.front;              //若原队列中只有一个结点，删除后变空
    free(p);
    return true;
}
```

3.2.4　双端队列

双端队列是指允许两端都可以进行插入和删除操作的线性表，如图 3.10 所示。双端队列两端的地位是平等的，为了方便理解，将左端也视为前端，右端也视为后端。

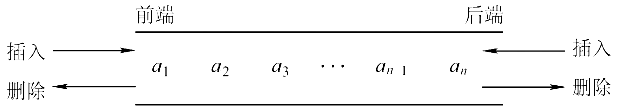


图 3.10　双端队列

在双端队列入队时，前端进的元素排列在队列中后端进的元素的前面，后端进的元素排列在队列中前端进的元素的后面。在双端队列出队时，无论是前端还是后端出队，先出的元素排列在后出的元素的前面。思考：如何由入队序列 a, b, c, d 得到出队序列 d, c, a, b?

命题追踪　双端队列出队/入队操作模拟（2010、2021）

输出受限的双端队列：允许在一端进行插入和删除，但在另一端只允许插入的双端队列称为输出受限的双端队列，如图 3.11 所示。

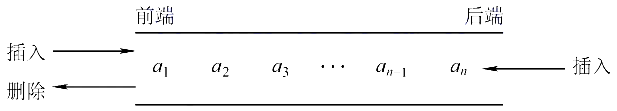


图 3.11　输出受限的双端队列

输入受限的双端队列：允许在一端进行插入和删除，但在另一端只允许删除的双端队列称为输入受限的双端队列，如图 3.12 所示。若限定双端队列从某个端点插入的元素只能从该端点删除，则该双端队列就蜕变为两个栈底相邻接的栈。

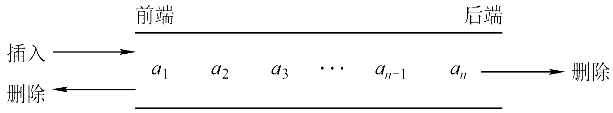


图 3.12　输入受限的双端队列

例　设有一个双端队列，输入序列为 1, 2, 3, 4，试分别求出以下条件的输出序列。

（1）能由输入受限的双端队列得到，但不能由输出受限的双端队列得到的输出序列。

（2）能由输出受限的双端队列得到，但不能由输入受限的双端队列得到的输出序列。

（3）既不能由输入受限的双端队列得到，又不能由输出受限的双端队列得到的输出序列。

解：先看输入受限的双端队列，如图 3.13 所示。假设 end1 端输入 1, 2, 3, 4，则 end2 端的输出相当于队列的输出，即 1, 2, 3, 4；而 end1 端的输出相当于栈的输出，$n = 4$ 时仅通过 end1 端有 14 种输出序列（由 Catalan 公式得出），仅通过 end1 端不能得到的输出序列有 $4! - 14 = 10$ 种：

1, 4, 2, 3　　2, 4, 1, 3　　3, 4, 1, 2　　3, 1, 4, 2　　3, 1, 2, 4
4, 3, 1, 2　　4, 1, 3, 2　　4, 2, 3, 1　　4, 2, 1, 3　　4, 1, 2, 3

通过 end1 和 end2 端混合输出，可以输出这 10 种中的 8 种，参看下表。其中，S_L, X_L 分别代表 end1 端的入队和出队，X_R 代表 end2 端的出队。

输出序列	入队出队顺序	输出序列	入队出队顺序
1, 4, 2, 3	$S_LX_RS_LS_LS_LX_LX_RX_R$	3, 1, 2, 4	$S_LS_LS_LX_LS_LX_RX_RX_R$
2, 4, 1, 3	$S_LS_LX_LS_LS_LX_LX_RX_R$	4, 1, 2, 3	$S_LS_LS_LS_LX_LX_RX_RX_R$
3, 4, 1, 2	$S_LS_LS_LX_LS_LX_LX_RX_R$	4, 1, 3, 2	$S_LS_LS_LS_LX_LX_RX_LX_R$
3, 1, 4, 2	$S_LS_LS_LX_LX_RS_LX_LX_R$	4, 3, 1, 2	$S_LS_LS_LS_LX_LX_LX_RX_R$

剩下两种是不能通过输入受限的双端队列输出的，即 4, 2, 3, 1 和 4, 2, 1, 3。

再看输出受限的双端队列，如图 3.14 所示。假设 end1 端和 end2 端都能输入，仅 end2 端可以输出。若都从 end2 端输入，就是一个栈了。当输入序列为 1, 2, 3, 4 时，输出序列有 14 种。对于其他 10 种不能得到的输出序列，交替从 end1 和 end2 端输入，还可以输出其中 8 种。设 S_L 代表 end1 端的输入，S_R、X_R 分别代表 end2 端的输入和输出，则可能的输出序列见下表。

图 3.13　例题中输入受限的双端队列

图 3.14　例题中输出受限的双端队列

输出序列	入队出队顺序	输出序列	入队出队顺序
1, 4, 2, 3	$S_LX_RS_LS_LS_RX_RX_RX_R$	3, 1, 2, 4	$S_LS_LS_RX_RX_RS_LX_RX_R$
2, 4, 1, 3	$S_LS_RX_RS_LS_RX_RX_RX_R$	4, 1, 2, 3	$S_LS_LS_LS_RX_RX_RX_RX_R$
3, 4, 1, 2	$S_LS_LS_RX_RS_RX_RX_RX_R$	4, 2, 1, 3	$S_LS_RS_LS_RX_RX_RX_RX_R$
3, 1, 4, 2	$S_LS_LS_RX_RX_RS_RX_RX_R$	4, 3, 1, 2	$S_LS_LS_RS_RX_RX_RX_RX_R$

通过输出受限的双端队列不能得到的两种输出序列是 4, 1, 3, 2 和 4, 2, 3, 1。

综上所述：

1）能由输入受限的双端队列得到，但不能由输出受限的双端队列得到的是 4, 1, 3, 2。

2）能由输出受限的双端队列得到，但不能由输入受限的双端队列得到的是 4, 2, 1, 3。

3）既不能由输入受限的双端队列得到，又不能由输出受限的双端队列得到的是 4, 2, 3, 1。

提　示

实际双端队列的考题不会这么复杂，通常仅判断序列是否满足题设条件，代入验证即可。

3.2.5　本节试题精选

一、单项选择题

01. 栈和队列的主要区别在于（　）。

A. 它们的逻辑结构不一样　　B. 它们的存储结构不一样

C. 所包含的元素不一样　　D. 插入、删除操作的限定不一样

02. 队列的“先进先出”特性是指（　）。

I. 最后插入队列中的元素总是最后被删除

II. 当同时进行插入、删除操作时，总是插入操作优先

III. 每当有删除操作时，总要先做一次插入操作

IV. 每次从队列中删除的总是最早插入的元素

A. I　　B. I和IV　　C. II和III　　D. IV

03. 允许对队列进行的操作有（　）。

A. 对队列中的元素排序　　B. 取出最近入队的元素

C. 在队列元素之间插入元素　　D. 删除队首元素

04. 一个队列的入队顺序是1, 2, 3, 4，则出队的输出顺序是（　）。

A. 4, 3, 2, 1　　B. 1, 2, 3, 4　　C. 1, 4, 3, 2　　D. 3, 2, 4, 1

05. 循环队列存储在数组A[0…n]中，入队时的操作为（　）。

A. rear=rear+1　　B. rear=(rear+1) mod (n-1)

C. rear=(rear+1) mod n　　D. rear=(rear+1) mod (n+1)

06. 已知循环队列的存储空间为数组A[21]，front指向队首元素的前一个位置，rear指向队尾元素，假设当前front和rear的值分别为8和3，则该队列的长度为（　）。

A. 5　　B. 6　　C. 16　　D. 17

07. 若用数组A[0…5]实现循环队列，且当前rear和front的值分别为1和5，当从队列中删除一个元素，再加入两个元素后，rear和front的值分别为（　）。

A. 3和4　　B. 3和0　　C. 5和0　　D. 5和1

08. 假设用数组Q[MaxSize]实现循环队列，队首指针front指向队首元素的前一位置，队尾指针rear指向队尾元素，则判断该队列为空的条件是（　）。

A. Q.rear==(Q.front+1)%MaxSize

B. (Q.rear+1)%MaxSize==Q.front+1

C. (Q.rear+1)%MaxSize==Q.front

D. Q.rear==Q.front

09. 假设循环队列Q[MaxSize]的队首指针为front，队尾指针为rear，队列的最大容量为MaxSize，此外，该队列再没有其他数据成员，则判断该队列已满足条件是（　）。

A. Q.front==Q.rear　　B. Q.front+Q.rear>=MaxSize

C. Q.front==(Q.rear+1)%MaxSize　　D. Q.rear==(Q.front+1)%MaxSize

10. 假设用A[0…n]实现循环队列，front、rear分别指向队首元素的前一个位置和队尾元素。若用(rear+1)%(n+1)==front作为队满标志，则（　）。

A. 可用front==rear作为队空标志　　B. 队列中最多可有n+1个元素

C. 可用front>rear作为队空标志

D. 可用(front+1)%(n+1)==rear作为队空标志

11. 与顺序队列相比，链式队列的（　）。

A. 优点是队列的长度不受限制

B. 优点是入队和出队时间效率更高

C. 缺点是不能进行顺序访问

D. 缺点是不能根据队首指针和队尾指针计算队列的长度

12. 下列描述的几种链表中，最适合用作队列的是（　）。

A. 带队首指针和队尾指针的循环单链表

B. 带队首指针和队尾指针的非循环单链表

C. 只带队首指针的非循环单链表

D. 只带队首指针的循环单链表

13. 下列描述的几种链表中，最不适合用作链式队列的是（　）。

A. 只带队首指针的非循环双链表　　B. 只带队首指针的循环双链表

C. 只带队尾指针的循环双链表　　D. 只带队尾指针的循环单链表

14. 在用单链表实现队列时，队头设在链表的（　）位置。

A. 链头　　B. 链尾　　C. 链中　　D. 以上都可以

15. 用链式存储方式的队列进行删除操作时需要（　）。

A. 仅修改头指针　　B. 仅修改尾指针

C. 头尾指针都要修改　　D. 头尾指针可能都要修改

16. 在一个链式队列中，假设队首指针为 front，队尾指针为 rear，x 所指向的元素需要入队，则需要执行的操作为（　）。

A. front=x, front=front->next

B. x->next=front->next, front=x

C. rear->next=x, rear=x

D. rear->next=x, x->next=NULL, rear=x

17. 假设循环单链表表示的队列长度为 n，队头固定在链表尾，若只设头指针，则入队操作的时间复杂度为（　）。

A. $O(n)$　　B. $O(1)$　　C. $O(n^2)$　　D. $O(n\log_2 n)$

18. 假设输入序列为 1, 2, 3, 4, 5，利用两个队列进行出入队操作，不可能输出的序列是（　）。

A. 1, 2, 3, 4, 5　　B. 5, 2, 3, 4, 1　　C. 1, 3, 2, 4, 5　　D. 4, 1, 5, 2, 3

19. 若以 1, 2, 3, 4 作为双端队列的输入序列，则既不能由输入受限的双端队列得到，又不能由输出受限的双端队列得到的输出序列是（　）。

A. 1, 2, 3, 4　　B. 4, 1, 3, 2　　C. 4, 2, 3, 1　　D. 4, 2, 1, 3

20. 【2010 统考真题】某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是（　）。

A. b, a, c, d, e　　B. d, b, a, c, e　　C. d, b, c, a, e　　D. e, c, b, a, d

21. 【2011 统考真题】已知循环队列存储在一维数组 A[0…n-1]中，且队列非空时 front 和 rear 分别指向队首元素和队尾元素。若初始时队列为空，且要求第一个进入队列的元素存储在 A[0]处，则初始时 front 和 rear 的值分别是（　）。

A. 0，0　　B. 0，$n-1$　　C. $n-1$，0　　D. $n-1$，$n-1$

22. 【2014 统考真题】循环队列放在一维数组 A[0…M-1]中，end1 指向队首元素，end2 指向队尾元素的后一个位置。假设队列两端均可进行入队和出队操作，队列中最多能容纳 M-1 个元素。初始时为空。下列判断队空和队满的条件中，正确的是（　）。

A. 队空：end1==end2;　　队满：end1==(end2+1) mod M

B. 队空：end1==end2;　　队满：end2==(end1+1) mod (M-1)

C. 队空：end2==(end1+1)mod M;　　队满：end1==(end2+1) mod M

D. 队空：end1==(end2+1)mod M;　　队满：end2==(end1+1) mod (M-1)

23. 【2018 统考真题】现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：①出队并输出出队元素；②出队并将出队元素入栈；③出栈并输出出栈元素，则不能得到的输出序列是（　）。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1

24.【2021统考真题】初始为空的队列Q的一端仅能进行入队操作，另外一端既能进行入队操作又能进行出队操作。若Q的入队序列是1, 2, 3, 4, 5，则不能得到的出队序列是(　)。

A. 5, 4, 3, 1, 2　　B. 5, 3, 1, 2, 4　　C. 4, 2, 1, 3, 5　　D. 4, 1, 3, 2, 5

二、综合应用题

01. 若希望循环队列中的元素都能得到利用，则需设置一个标志域tag，并以tag的值为0或1来区分队首指针front和队尾指针rear相同时的队列状态是“空”还是“满”。试编写与此结构相应的入队和出队算法。

02. Q是一个队列，S是一个空栈，实现将队列中的元素逆置的算法。

03. 利用两个栈S1和S2来模拟一个队列，已知栈的4个运算定义如下：

```
Push(S,x);                  //元素x入栈S
Pop(S,x);                   //S出栈并将出栈的值赋给x
StackEmpty(S);              //判断栈是否为空
StackOverflow(S);           //判断栈是否为满
```

如何利用栈的运算来实现该队列的3个运算（形参由读者根据要求自己设计）？

```
Enqueue;                    //将元素x入队
Dequeue;                    //出队，并将出队元素存储在x中
QueueEmpty;                 //判断队列是否为空
```

04.【2019统考真题】请设计一个队列，要求满足：①初始时队列为空；②入队时，允许增加队列占用空间；③出队后，出队元素所占用的空间可重复使用，即整个队列所占用的空间只增不减；④入队操作和出队操作的时间复杂度始终保持为$O(1)$。请回答：

1）该队列是应选择链式存储结构，还是应选择顺序存储结构？

2）画出队列的初始状态，并给出判断队空和队满的条件。

3）画出第一个元素入队后的队列状态。

4）给出入队操作和出队操作的基本过程。

3.2.6　答案与解析

一、单项选择题

01. D

栈和队列的逻辑结构都是线性结构，都可以采用顺序存储或链式存储，选项C显然也错误。只有选项D才是栈和队列的本质区别，限定表中插入和删除操作位置的不同。

02. B

队列“先进先出”的特性表现在：先入队列的元素先出队列，后入队列的元素后出队列，入队对应的是插入操作，出队对应的是删除操作。选项I和IV均正确。

03. D

删除队首元素即出队，是队列的基本操作之一，所以选择选项D。

04. B

队列的入队顺序和出队顺序是一致的，这是和栈不同的。

05. D

数组下标范围为0～n，因此数组容量为n+1。循环队列中元素入队的操作是rear=(rear+1) mod maxsize，题中maxsize=n+1。因此入队操作应为rear=(rear+1) mod (n+1)。

06. C

队列的长度为(rear-front+maxsize)%maxsize=(rear-front+21)%21=16。这种情

况和 front 指向当前元素，rear 指向队尾元素的下一个元素的计算是相同的。

注　意

数组 A[n]的下标范围为 0～n-1。若写成 A[0…n]，则说明下标范围为 0～n。

07．B

循环队列中，每删除一个元素，队首指针 front=(front+1)%6，每插入一个元素，队尾指针 rear=(rear+1)%6。上述操作后，front=0，rear=3。

08．D

当队列中只有一个元素时，front 指向该元素的前一个位置，rear 指向该元素，因此当队列为空时，队首指针等于队尾指针，这样第一个元素入队后，才能符合题目要求。

09．C

既然不能附加任何其他数据成员，只能采用牺牲一个存储单元的方法来区分是队空还是队满，约定以“队列头指针在队尾指针的下一位置作为队满的标志”，因此选择选项 C。选项 A 是判断队列是否空的条件，选项 B 和 D 都是干扰项。

注　意

对于这类具体问题，举一些特例判断往往比直接思考问题能更快得到答案。

10．A

若用(rear+1)%(n+1)==front 作为队满标志，则说明题目采用了牺牲一个存储单元的方法来区分队空和队满，因此可用 front==rear 作为队空标志。

11．D

虽然链式队列采用动态分配方式，但其长度也受内存空间的限制，不能无限制增长。顺序队列和链式队列的入队和出队时间复杂度均为 $O(1)$。顺序队列和链式队列都可以进行顺序访问。对于顺序队列，可通过队首指针和队尾指针计算队列中的元素个数，而链式队列则不能。

12．B

因为队列需在双端进行操作，所以选项 C 和 D 的链表显然不太适合链队。对于选项 A，链表在完成入队和出队后还要修改为循环的，对于队列来讲这是多余的（画蛇添足）。对于选项 B，因为有首指针，所以适合删除首结点；因为有队尾指针，所以适合在其后插入结点。

13．A

因为非循环双链表只带队首指针，所以在执行入队操作时需要修改队尾结点的指针域，而查找队尾结点需要 $O(n)$的时间。选项 B、C 和 D 均可在 $O(1)$的时间内找到队首和队尾。

14．A

因为在队头做出队操作，为便于删除队首元素，所以总是选择链头作为队头。

15．D

队列用链式存储时，删除元素从表头删除，通常仅需修改头指针，但若队列中仅有一个元素，则队尾指针也需要被修改，当仅有一个元素时，删除后队列为空，需修改队尾指针为 rear=front。

16．D

插入操作时，先将结点 x 插入到链表尾部，再让 rear 指向这个结点 x。选项 C 的做法不够严密，因为是队尾，所以队尾 x->next 必须置为空。

17. A

依题意，入队操作是在队尾进行，即链表表头。题中已明确说明链表只设头指针，即没有头结点和尾指针，入队后，循环单链表必须保持循环的性质，在只带头指针的循环单链表中寻找表尾结点的时间复杂度为$O(n)$，所以入队的时间复杂度为$O(n)$。

18. B

此类题可对各选项进行模拟，假设队列为Q_1和Q_2。对于选项A，元素依次入队Q_1，然后依次出队。对于选项B，5最先出队，只可能是1, 2, 3, 4入队Q_1，5入队Q_2，然后5出队，只能得到5, 1, 2, 3, 4。对于选项C，1, 2, 5入队Q_1，3, 4入队Q_2，然后按要求出队。选项D的分析同选项C。

19. C

使用排除法。先看可由输入受限的双端队列产生的序列：设右端输入受限，1, 2, 3, 4依次左入，则依次左出可得4, 3, 2, 1，排除选项A；左出、右出、左出、左出可得到4, 1, 3, 2，排除选项B；再看可由输出受限的双端队列产生的序列：设右端输出受限，1, 2, 3, 4依次左入、左入、右入、左入，依次左出可得到4, 2, 1, 3，排除选项D。

20. C

本题的队列实际上是一个输出受限的双端队列，如图3.11所示。A操作：*a*左入（或右入）、*b*左入、*c*右入、*d*右入、*e*右入。B操作：*a*左入（或右入）、*b*左入、*c*右入、*d*左入、*e*右入。D操作：*a*左入（或右入）、*b*左入、*c*左入、*d*右入、*e*左入。C操作：*a*左入（或右入）、*b*右入、因*d*未出，此时只能入队，*c*怎么进都不可能在*b*和*a*之间。

【另解】初始时队列为空，第1个元素*a*左入（或右入）后，第2个元素*b*无论是左入还是右入都必与*a*相邻，而选项C中*a*与*b*不相邻，不合题意。

21. B

根据题意，第一个元素进入队列后存储在`A[0]`处，此时`front`和`rear`值都为`0`。入队时因为要执行`(rear+1)%n`操作，所以若入队后指针指向`0`，则`rear`初值为`n-1`，而因为第一个元素在`A[0]`中，插入操作只改变`rear`指针，所以`front`为`0`不变。

注　意

①循环队列是指顺序存储的队列，而不是指逻辑上的循环，如循环单链表表示的队列不能称为循环队列。②`front`和`rear`的初值并不是固定的。

22. A

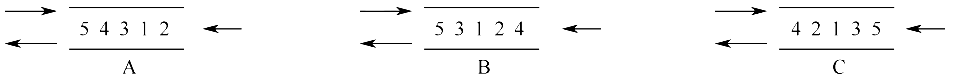
`end1`指向队首元素，可知出队操作是先从`A[end1]`读数，然后`end1`再加`1`。`end2`指向队尾元素的后一个位置，可知入队操作是先存数到`A[end2]`，然后`end2`再加`1`。若用`A[0]`存储第一个元素，则队列初始时，入队操作先把数据放到`A[0]`中，然后`end2`自增，即可知`end2`初值为`0`；而`end1`指向的是队首元素，队首元素在数组`A`中的下标为`0`，所以得知`end1`的初值也为`0`，可知队空条件为`end1==end2`；然后考虑队列满时，因为队列最多能容纳`M-1`个元素，假设队列存储在下标为`0`到`M-2`的`M-1`个区域，队头为`A[0]`，队尾为`A[M-2]`，此时队列满，考虑在这种情况下`end1`和`end2`的状态，`end1`指向队首元素，可知`end1=0`，`end2`指向队尾元素的后一个位置，可知`end2=M-2+1=M-1`，所以队满条件为`end1==(end2+1) mod M`。

23. C

选项A的操作顺序为①①②②①①③③。选项B的操作顺序为②①①①①①③。选项D的操作顺序为②②②②②①③③③③③。对于选项C：首先输出3，说明1和2必须先依次入栈，而此后2肯定比1先输出，因此无法得到1, 2的输出顺序。

24. D

假设队列左端允许入队和出队，右端只能入队。对于选项 A，依次从右端入队 1, 2，再从左端入队 3, 4, 5。对于选项 B，从右端入队 1, 2，然后从左端入队 3，再从右端入队 4，最后从左端入队 5。对于选项 C，从左端入队 1, 2，然后从右端入队 3，再从左端入队 4，最后从右端入队 5。无法验证选项 D 的序列。



二、综合应用题

01.【解答】

在循环队列的类型结构中，增设一个整型变量 tag，入队时置 tag 为 1，出队时置 tag 为 0（因为只有入队操作可能导致队满，也只有出队操作可能导致队空）。队列 Q 初始时，置 tag=0、front=rear=0。这样队列的 4 要素如下：

队空条件：Q.front==Q.rear 且 Q.tag==0。

队满条件：Q.front==Q.rear 且 Q.tag==1。

入队操作：Q.data[Q.rear]=x; Q.rear=(Q.rear+1)%MaxSize; Q.tag=1。

出队操作：x=Q.data[Q.front]; Q.front=(Q.front+1)%MaxSize; Q.tag=0。

1）设“tag”法的循环队列入队算法：

```
int EnQueue1(SqQueue &Q,ElemType x){
    if(Q.front==Q.rear&&Q.tag==1)
return 0;                       //两个条件都满足时则队满
    Q.data[Q.rear]=x;
    Q.rear=(Q.rear+1)%MaxSize;
    Q.tag=1;                    //可能队满
    return 1;
}
```

2）设“tag”法的循环队列出队算法：

```
int DeQueue1(SqQueue &Q,ElemType &x){
    if(Q.front==Q.rear&&Q.tag==0)
        return 0;               //两个条件都满足时则队空
    x=Q.data[Q.front];
    Q.front=(Q.front+1)%MaxSize;
    Q.tag=0;                    //可能队空
    return 1;
}
```

02.【解答】

本题主要考查大家对队列和栈的特性与操作的理解。因为对队列的一系列操作不可能将其中的元素逆置，而栈可以将入栈的元素逆序提取出来，所以我们可以让队列中的元素逐个地出队，入栈；全部入栈后再逐个出栈，入队。

算法的实现如下：

```
void Inverser(Stack &S,Queue &Q){
//本算法实现将队列中的元素逆置
    while(!QueueEmpty(Q)){
        x=DeQueue(Q);           //队列中全部元素依次出队
        Push(S,x);              //元素依次入栈
    }
    while(!StackEmpty(S)){
        Pop(S,x);               //栈中全部元素依次出栈
```

```
            EnQueue(Q,x);                //再入队
        }
    }
```

03.【解答】

利用两个栈S1和S2来模拟一个队列，当需要向队列中插入一个元素时，用S1来存放已输入的元素，即S1执行入栈操作。当需要出队时，则对S2执行出栈操作。因为从栈中取出元素的顺序是原顺序的逆序，所以必须先将S1中的所有元素全部出栈并入栈到S2中，再在S2中执行出栈操作，即可实现出队操作，而在执行此操作前必须判断S2是否为空，否则会导致顺序混乱。当栈S1和S2都为空时队列为空。

总结如下：

1）对S2的出栈操作用作出队，若S2为空，则先将S1中的所有元素送入S2。

2）对S1的入栈操作用作入队，若S1满，必须先保证S2为空，才能将S1中的元素全部插入S2中。

入队算法：

```
int EnQueue(Stack &S1,Stack &S2,ElemType e){
    if(!StackOverflow(S1)){
        Push(S1,e);
        return 1;
    }
    if(StackOverflow(S1)&&!StackEmpty(S2)){
        printf("队列为满");
        return 0;
    }
    if(StackOverflow(S1)&&StackEmpty(S2)){
        while(!StackEmpty(S1)){
            Pop(S1,x);
            Push(S2,x);
        }
    }
    Push(S1,e);
    return 1;
}
```

出队算法：

```
void DeQueue(Stack &S1,Stack &S2,ElemType &x){
    if(!StackEmpty(S2)){
        Pop(S2,x);
    }
    else if(StackEmpty(S1)){
        printf("队列为空");
    }
    else{
        while(!StackEmpty(S1)){
            Pop(S1,x);
            Push(S2,x);
        }
    Pop(S2,x);
    }
}
```

判断队列为空的算法：

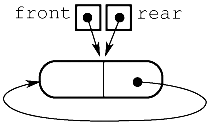
```
int QueueEmpty(Stack S1,Stack S2){
    if(StackEmpty(S1)&&StackEmpty(S2))
        return  1;
    else
        return  0;
}
```

04.【解答】

1）顺序存储无法满足要求②的队列占用空间随着入队操作而增加。根据要求来分析：要求①容易满足；链式存储方便开辟新空间，要求②容易满足；对于要求③，出队后的结点并不真正释放，用队首指针指向新的队头结点，新元素入队时，有空余结点则无须开辟新空间，赋值到队尾后的第一个空结点即可，然后用队尾指针指向新的队尾结点，这就需要设计成一个首尾相接的循环单链表，类似于循环队列的思想。设置队头、队尾指针后，链式队列的入队操作和出队操作的时间复杂度均为 $O(1)$，要求④可以满足。

因此，采用链式存储结构（两段式单向循环链表），队首指针为 front，队尾指针为 rear。

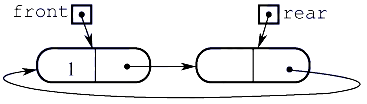
2）该循环链式队列的实现可以参考循环队列，不同之处在于循环链式队列可以方便地增加空间，出队的结点可以循环利用，入队时空间不够也可以动态增加。同样，循环链式队列也要区分队满和队空的情况，这里参考循环队列牺牲一个单元来判断。初始时，创建只有一个空结点的循环单链表，头指针 front 和尾指针 rear 均指向空结点，如下图所示。



队空的判定条件：front==rear。

队满的判定条件：front==rear->next。

3）插入第一个元素后的状态如下图所示。



4）操作的基本过程如下：

入队操作：
若（front==rear->next）　　//队满 　　则在rear后面插入一个新的空闲结点； 入队元素保存到rear所指结点中；rear=rear->next；返回。
出队操作：
若（front==rear）　　//队空 　　则出队失败，返回； 取front所指结点中的元素e；front=front->next；返回e。

3.3　栈和队列的应用

要熟练掌握栈和队列，必须学习栈和队列的应用，把握其中的规律，然后举一反三。接下来将简单介绍栈和队列的一些常见应用。

3.3.1　栈在括号匹配中的应用

假设表达式中允许包含两种括号：圆括号和方括号，其嵌套的顺序任意即（[]()）或[(([][]))]等均为正确的格式，[(])或([()]或(()]均为不正确的格式。

考虑下列括号序列：

[	(	[	]	[	]	)	]
1	2	3	4	5	6	7	8

分析如下：

1）计算机接收第1个括号“[”后，期待与之匹配的第8个括号“]”出现。

2）获得了第2个括号“(”，此时第1个括号“[”暂时放在一边，而急迫期待与之匹配的第7个括号“)”出现。

3）获得了第3个括号“[”，此时第2个括号“(”暂时放在一边，而急迫期待与之匹配的第4个括号“]”出现。第3个括号的期待得到满足，消解之后，第2个括号的期待匹配又成为当前最急迫的任务。

4）以此类推，可见该处理过程与栈的思想吻合。

算法的思想如下：

1）初始设置一个空栈，顺序读入括号。

2）若是左括号，则作为一个新的更急迫的期待压入栈中，自然使原有的栈中所有未消解的期待的急迫性降了一级。

3）若是右括号，则或使置于栈顶的最急迫期待得以消解，或是不合法的情况（括号序列不匹配，退出程序）。算法结束时，栈为空，否则括号序列不匹配。

3.3.2 栈在表达式求值中的应用

表达式求值是程序设计语言编译中一个最基本的问题，它是栈应用的一个典型范例。

1．算术表达式

中缀表达式（如3+4）是人们常用的算术表达式，操作符以中缀形式处于操作数的中间。与前缀表达式（如+34）或后缀表达式（如34+）相比，中缀表达式不容易被计算机解析，但仍被许多程序语言使用，因为它更符合人们的思维习惯。

与前缀表达式或后缀表达式不同的是，中缀表达式中的括号是必需的。计算过程中必须用括号将操作符和对应的操作数括起来，用于指示运算的次序。后缀表达式的运算符在操作数后面，后缀表达式中考虑了运算符的优先级，没有括号，只有操作数和运算符。

中缀表达式A+B*(C-D)-E/F对应的后缀表达式为ABCD-*+EF/-，将后缀表达式与原表达式对应的表达式树（图3.15）的后序遍历序列进行比较，可发现它们有异曲同工之妙。

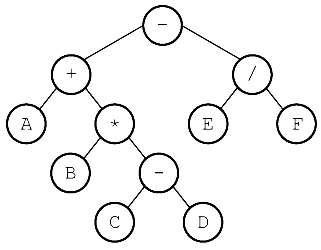


图3.15 A+B*(C-D)-E/F对应的表达式树

2．中缀表达式转后缀表达式

命题追踪 ▶ 中缀表达式转后缀表达式的方法（2024）

下面先给出一种由中缀表达式转后缀表达式[①]的手算方法。

① 中缀转前缀的方法类似，且统考真题仅考查过中缀转后缀的过程，因此本节仅介绍中缀转后缀的方法。

1）按照运算符的运算顺序对所有运算单位加括号。

2）将运算符移至对应括号的后面，相当于按“左操作数 右操作数 运算符”重新组合。

3）去除所有括号。

例如，中缀表达式 A+B*(C-D)-E/F 转后缀表达的过程如下（下标表示运算符的运算顺序）：

1）加括号：((A+③(B*②(C-①D)))-⑤(E/④F))。

2）运算符后移：((A(B(CD)-①)*②)+③(EF)/④)-⑤。

3）去除括号后，得到后缀表达式：ABCD-①*②+③EF/④-⑤。

在计算机中，中缀表达式转后缀表达式时需要借助一个栈，用于保存暂时还不能确定运算顺序的运算符。从左到右依次扫描中缀表达式中的每一项，具体转化过程如下：

1）遇到操作数。直接加入后缀表达式。

2）遇到界限符。若为“(”，则直接入栈；若为“)”，则不入栈，且依次弹出栈中的运算符并加入后缀表达式，直到遇到“(”为止，并直接删除“(”。

3）遇到运算符。① 若其优先级高于栈顶运算符或遇到栈顶为“(”，则直接入栈；② 若其优先级低于或等于栈顶运算符，则依次弹出栈中的运算符并加入后缀表达式，直到遇到一个优先级低于它的运算符或遇到“(”或栈空为止，之后将当前运算符入栈。

按上述方法扫描所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

例如，中缀表达式 A+B*(C-D)-E/F 转后缀表达式的过程如表 3.1 所示。

表 3.1　中缀表达式 A+B*(C-D)-E/F 转后缀表达式的过程

步	待处理序列	栈内	后缀表达式	扫描项	说　明
1	A+B*(C-D)-E/F			A	A 加入后缀表达式
2	+B*(C-D)-E/F		A	+	+入栈
3	B*(C-D)-E/F	+	A	B	B 加入后缀表达式
4	*(C-D)-E/F	+	AB	*	*优先级高于栈顶，*入栈
5	(C-D)-E/F	+*	AB	(	(直接入栈
6	C-D)-E/F	+*(	AB	C	C 加入后缀表达式
7	-D)-E/F	+*(	ABC	-	栈顶为(，-直接入栈
8	D)-E/F	+*(-	ABC	D	D 加入后缀表达式
9	)-E/F	+*(-	ABCD	)	遇到)，弹出-，删除(
10	-E/F	+*	ABCD-	-	-优先级低于栈顶，依次弹出*、+，-入栈
11	E/F	-	ABCD-*+	E	E 加入后缀表达式
12	/F	-	ABCD-*+E	/	/优先级高于栈顶，/入栈
13	F	-/	ABCD-*+E	F	F 加入后缀表达式
14		-/	ABCD-*+EF		字符扫描完毕，弹出剩余运算符
			ABCD-*+EF/-		结束

命题追踪 ▶ 栈的深度分析（2009、2012）

所谓栈的深度，是指栈中的元素个数，通常是给出入栈和出栈序列，求最大深度（栈的容量应大于或等于最大深度）。有时会间接给出入栈和出栈序列，例如以中缀表达式和后缀表达式的形式给出入栈和出栈序列。掌握栈的先进后出的特点进行手工模拟是解决这类问题的有效方法。

3．后缀表达式求值

命题追踪 ▶ 用栈实现表达式求值的分析（2018）

通过后缀表示计算表达式值的过程：从左往右依次扫描表达式的每一项，若该项是操作数，

则将其压入栈中；若该项是操作符<op>，则从栈中退出两个操作数 Y 和 X，形成运算指令 X<op>Y，并将计算结果压入栈中。当所有项都扫描并处理完后，栈顶存放的就是最后的计算结果。

例如，后缀表达式 ABCD-*+EF/-求值的过程需要 12 步，见表 3.2。

表 3.2　后缀表达式 ABCD-*+EF/-求值的过程

步	扫　描　项	项　类　型	动　　作	栈中内容
1			置空栈	空
2	A	操作数	入栈	A
3	B	操作数	入栈	A B
4	C	操作数	入栈	A B C
5	D	操作数	入栈	A B C D
6	-	操作符	D、C 出栈，计算 C-D，结果 R_1 入栈	A B R_1
7	*	操作符	R_1、B 出栈，计算 B×R_1，结果 R_2 入栈	A R_2
8	+	操作符	R_2、A 出栈，计算 A+R_2，结果 R_3 入栈	R_3
9	E	操作数	入栈	R_3 E
10	F	操作数	入栈	R_3 E F
11	/	操作符	F、E 出栈，计算 E/F，结果 R_4 入栈	R_3 R_4
12	-	操作符	R_4、R_3 出栈，计算 R_3-R_4，结果 R_5 入栈	R_5

3.3.3　栈在递归中的应用

递归是一种重要的程序设计方法。简单地说，若在一个函数、过程或数据结构的定义中又应用了它自身，则这个函数、过程或数据结构称为是递归定义的，简称递归。

递归通常把一个大型的复杂问题层层转化为一个与原问题相似的规模较小的问题来求解，递归策略只需少量的代码就可以描述出解题过程所需要的多次重复计算，大大减少了程序的代码量。但在通常情况下，它的效率并不是太高。

以斐波那契数列为例，其定义为

$$F(n)=\begin{cases}F(n-1)+F(n-2), & n>1\\ 1, & n=1\\ 0, & n=0\end{cases}$$

这就是递归的一个典型例子，用程序实现时如下：

```
int F(int n){                                    //斐波那契数列的实现
    if(n==0)
        return 0;                                //边界条件
    else if(n==1)
        return 1;                                //边界条件
    else
        return F(n-1)+F(n-2);                    //递归表达式
}
```

必须注意递归模型不能是循环定义的，其必须满足下面的两个条件：

- 递归表达式（递归体）。
- 边界条件（递归出口）。

递归的精髓在于能否将原始问题转换为属性相同但规模较小的问题。

命题追踪 ▶ 栈在函数调用中的作用和工作原理（2015、2017）

在递归调用的过程中，系统为每一层的返回点、局部变量、传入实参等开辟了递归工作栈来进行数据存储，递归次数过多容易造成栈溢出等。而其效率不高的原因是递归调用过程中包含很

多重复的计算。下面以 $n=5$ 为例，列出递归调用执行过程，如图 3.16 所示。

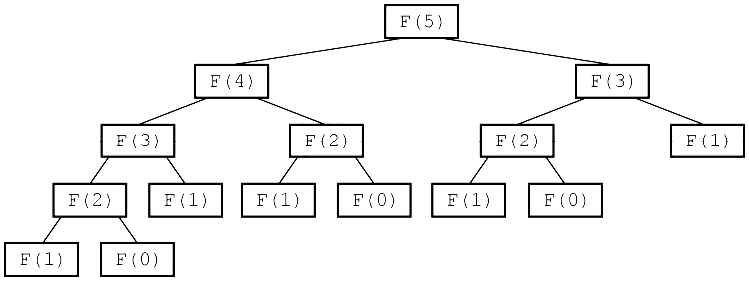


图 3.16　F(5)的递归执行过程

显然，在递归调用的过程中，F(3)被计算 2 次，F(2)被计算 3 次。F(1)被调用 5 次，F(0)被调用 3 次。所以，递归的效率低下，但优点是代码简单，容易理解。在第 5 章的树中利用了递归的思想，代码变得十分简单。通常情况下，初学者很难理解递归的调用过程，若读者想具体了解递归是如何实现的，可以参阅编译原理教材中的相关内容。

可以将递归算法转换为非递归算法，通常需要借助栈来实现这种转换。

3.3.4　队列在层次遍历中的应用

在信息处理中有一大类问题需要逐层或逐行处理。这类问题的解决方法往往是在处理当前层或当前行时就对下一层或下一行做预处理，把处理顺序安排好，等到当前层或当前行处理完毕，就可以处理下一层或下一行。使用队列是为了保存下一步的处理顺序。下面用二叉树（见图 3.17）层次遍历的例子，说明队列的应用。表 3.3 显示了层次遍历二叉树的过程。

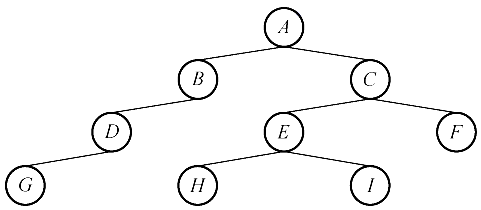


图 3.17　二叉树

表 3.3　层次遍历二叉树的过程

序	说　明	队内	队　外
1	*A* 入	*A*	
2	*A* 出，*BC* 入	*BC*	*A*
3	*B* 出，*D* 入	*CD*	*AB*
4	*C* 出，*EF* 入	*DEF*	*ABC*
5	*D* 出，*G* 入	*EFG*	*ABCD*
6	*E* 出，*HI* 入	*FGHI*	*ABCDE*
7	*F* 出	*GHI*	*ABCDEF*
8	*GHI* 出		*ABCDEFGHI*

该过程的简单描述如下：

① 根结点入队。

② 若队空（所有结点都已处理完毕），则结束遍历；否则重复③操作。

③ 队列中第一个结点出队，并访问之。若其有左孩子，则将左孩子入队；若其有右孩子，则将右孩子入队，返回②。

3.3.5 队列在计算机系统中的应用

队列在计算机系统中的应用非常广泛，以下仅从两个方面来阐述：第一个方面是解决主机与外部设备之间速度不匹配的问题，第二个方面是解决由多用户引起的资源竞争问题。

命题追踪 ▶ 缓冲区的逻辑结构（2009）

对于第一个方面，仅以主机和打印机之间速度不匹配的问题为例做简要说明。主机输出数据给打印机打印，输出数据的速度比打印数据的速度要快得多，因为速度不匹配，若直接把输出的数据送给打印机打印，则显然是不行的。解决的方法是设置一个打印数据缓冲区，主机把要打印输出的数据依次写入这个缓冲区，写满后就暂停输出，转去做其他的事情。打印机就从缓冲区中按照先进先出的原则依次取出数据并打印，打印完后再向主机发出请求。主机接到请求后再向缓冲区写入打印数据。这样做既保证了打印数据的正确，又使主机提高了效率。由此可见，打印数据缓冲区中所存储的数据就是一个队列。

命题追踪 ▶ 多队列出队/入队操作的应用（2016）

对于第二个方面，CPU（中央处理器，它包括运算器和控制器）资源的竞争就是一个典型的例子。在一个带有多终端的计算机系统上，有多个用户需要CPU各自运行自己的程序，它们分别通过各自的终端向操作系统提出占用CPU的请求。操作系统通常按照每个请求在时间上的先后顺序，把它们排成一个队列，每次把CPU分配给队首请求的用户使用。当相应的程序运行结束或用完规定的时间间隔后，令其出队，再把CPU分配给新的队首请求的用户使用。这样既能满足每个用户的请求，又使CPU能够正常运行。

3.3.6 本节试题精选

一、单项选择题

01. 栈的应用不包括（ ）。

A. 递归　　B. 表达式求值　　C. 括号匹配　　D. 缓冲区

02. 表达式a*(b+c)-d的后缀表达式是（ ）。

A. abcd*+-　　B. abc+*d-　　C. abc*+d-　　D. -+*abcd

03. 下面（ ）用到了队列。

A. 括号匹配　　B. 表达式求值　　C. 递归　　D. FIFO页面替换算法

04. 利用栈求表达式的值时，设立运算数栈OPEN。假设OPEN只有两个存储单元，则在下列表达式中，不会发生溢出的是（ ）。

A. A-B*(C-D)　　B. (A-B)*C-D　　C. (A-B*C)-D　　D. (A-B)*(C-D)

05. 执行完下列语句段后，i的值为（ ）。

```
int f(int x){
    return ((x>0)? x*f(x-1):2);
}
int i;
i=f(f(1));
```

A. 2　　B. 4　　C. 8　　D. 无限递归

06. 设有如下递归函数，则计算 F(8)需要调用该递归函数的次数为（　）。

```
int F(int n){
    if(n<=3) return 1;
    else return F(n-2)+F(n-4)+1;
}
```

A. 7　　B. 8　　C. 9　　D. 10

07. 设有如下递归函数，在 func(func(5))的执行过程中，第 4 个被执行的 func 函数是（　）。

```
int func(int x){
    if(x<=3) return 2;
    else return func(x-2)+func(x-4);
}
```

A. func(2)　　B. func(3)　　C. func(4)　　D. func(5)

08. 对于一个问题的递归算法求解和其相对应的非递归算法求解，（　）。

A. 递归算法通常效率高一些　　B. 非递归算法通常效率高一些

C. 两者相同　　D. 无法比较

09. 执行函数时，其局部变量一般采用（　）进行存储。

A. 树形结构　　B. 静态链表　　C. 栈结构　　D. 队列结构

10. 执行（　）操作时，需要使用队列作为辅助存储空间。

A. 查找散列（哈希）表　　B. 广度优先搜索图

C. 前序（根）遍历二叉树　　D. 深度优先搜索图

11. 下列说法中，正确的是（　）。

A. 消除递归不一定需要使用栈

B. 对同一输入序列进行两组不同的合法入栈和出栈组合操作，所得的输出序列也一定相同

C. 通常使用队列来处理函数或过程调用

D. 队列和栈都是运算受限的线性表，只允许在表的两端进行运算

12. 【2009 统考真题】为解决计算机主机与打印机之间速度不匹配的问题，通常设置一个打印数据缓冲区，主机将要输出的数据依次写入该缓冲区，而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是（　）。

A. 栈　　B. 队列　　C. 树　　D. 图

13. 【2012 统考真题】已知操作符包括“+”“-”“*”“/”“(”和“)”。将中缀表达式 a+b-a*((c+d)/e-f)+g 转换为等价的后缀表达式 ab+acd+e/f-*-g+时，用栈来存放暂时还不能确定运算次序的操作符。栈初始时为空时，转换过程中同时保存在栈中的操作符的最大个数是（　）。

A. 5　　B. 7　　C. 8　　D. 11

14. 【2014 统考真题】假设栈初始为空，将中缀表达式 a/b+(c*d-e*f)/g 转换为等价的后缀表达式的过程中，当扫描到 f 时，栈中的元素依次是（　）。

A. +(*-　　B. +(-*　　C. /+(*-*　　D. /+-*

15. 【2015 统考真题】已知程序如下：

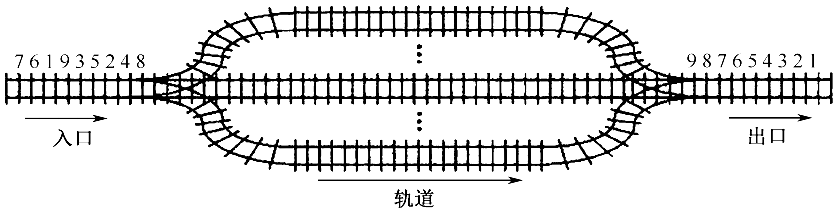
```
int S(int n)
{     return (n<=0)?0:S(n-1)+n;}
void main()
{     cout<<S(1);}
```

程序运行时使用栈来保存调用过程的信息，自栈底到栈顶保存的信息依次对应的是（　）。

A. main()→S(1)→S(0)　　B. S(0)→S(1)→main()

C. main()→S(0)→S(1)　　D. S(1)→S(0)→main()

16. 【2016统考真题】设有如下图所示的火车车轨，入口和出口之间有 n 条轨道，列车的行进方向均为从左至右，列车可驶入任意一条轨道。现有编号为1～9的9列列车，驶入的次序依次是8, 4, 2, 5, 3, 9, 1, 6, 7。若期望驶出的次序依次为1～9，则 n 至少是（　）。



A. 2　　B. 3　　C. 4　　D. 5

17. 【2017统考真题】下列关于栈的叙述中，错误的是（　）。

I. 采用非递归方式重写递归程序时必须使用栈

II. 函数调用时，系统要用栈保存必要的信息

III. 只要确定了入栈次序，即可确定出栈次序

IV. 栈是一种受限的线性表，允许在其两端进行操作

A. 仅I　　B. 仅I、II、III　　C. 仅I、III、IV　　D. 仅II、III、IV

18. 【2018统考真题】若栈S1中保存整数，栈S2中保存运算符，函数F()依次执行下述各步操作：

1）从S1中依次弹出两个操作数a和b。

2）从S2中弹出一个运算符op。

3）执行相应的运算b op a。

4）将运算结果压入S1中。

假定S1中的操作数依次是5, 8, 3, 2（2在栈顶），S2中的运算符依次是*、-、+（+在栈顶）。调用3次F()后，S1栈顶保存的值是（　）。

A. −15　　B. 15　　C. −20　　D. 20

19. 【2024统考真题】与表达式 $x+y*(z-u)/v$ 等价的后缀表达式是（　）。

A. $xyzu-*v/+$　　B. $xyzu-v/*+$　　C. $+x/*y-zuv$　　D. $+x*y/-zuv$

二、综合应用题

01. 假设一个算术表达式中包含圆括号、方括号和花括号3种类型的括号，编写一个算法来判别表达式中的括号是否配对，以字符“\0”作为算术表达式的结束符。

3.3.7 答案与解析

一、单项选择题

01. D

缓冲区是用队列实现的，选项A、B、C都是栈的典型应用。

02. B

后缀表达式中，每个运算符均直接位于其两个操作数的后面，按照这样的方式逐步根据计算的优先级将每个计算式进行变换，即可得到后缀表达式。

【另解】将两个直接操作数用括号括起来，再将操作符提到括号后，最后去掉括号。例如，

对于(①(②a*(③b+c))-d)，提出操作符并去掉括号后，可得后缀表达式为 abc+*d-。

学完第 5 章后，可将表达式画成二叉树的形式，再用后序遍历即可求得后缀表达式。

03．D

FIFO 页面替换算法用到了队列。其余的都只用到了栈。

04．B

利用栈求表达式的值时，可以分别设立运算符栈和运算数栈，其原理不变。选项 B 中 A 入栈，B 入栈，计算得 R1，C 入栈，计算得 R2，D 入栈，计算得 R3，由此得栈深为 2。选项 A、C、D 依次计算得栈深为 4、3、3。因此选择选项 B。

技　巧

根据运算符优先级，统计已依次入栈但还未参与计算的运算符的个数。以选项 C 为例，当“(”“A”“-”入栈时，“(”和“-”还未参与运算，此时运算符栈大小为 2，“B”和“*”入栈时运算符大小为 3，“C”入栈时“B*C”运算，运算符栈大小为 2，以此类推。

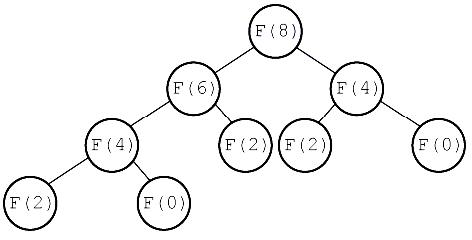
05．B

栈与递归有着紧密的联系。递归模型包括递归出口和递归体两个方面。递归出口是递归算法的出口，即终止递归的条件。递归体是一个递推的关系式。根据题意有

```
f(0)=2;
f(1)=1*f(0)=2;
f(f(1))=f(2)=2*f(1)=4。
```

06．C

计算 F(8)的递归调用树如下图所示：



由图可知，递归函数 F()调用的次数为 9。

07．C

首先，执行内层参数 func(5)=func(3)+func(1)=4，共执行 3 次 func 函数。然后，执行 func(func(5))=func(4)=func(2)+func(0)=4，因此第 4 个被执行的 func 函数是 func(4)。也可以采用画出递归调用树的方式，即某个函数的执行次序等于其在递归调用树的先序遍历中的次序。

08．B

通常情况下，递归算法在计算机实际执行的过程中包含很多的重复计算，所以效率会低。

09．C

调用函数时，系统会为调用者构造一个由参数表和返回地址组成的活动记录，并将记录压入系统提供的栈中，若被调用函数有局部变量，也要压入栈中。

10．B

本题涉及第 5 章和第 6 章的内容，图的广度优先搜索类似于树的层序遍历，都要借助于队列。

11．A

使用栈可以模拟递归的过程，以此来消除递归，但对于单向递归和尾递归而言，可以用迭代的方式来消除递归，选项 A 正确。不同的入栈和出栈组合操作，会产生许多不同的输出序列，B

错误。通常使用栈来处理函数或过程调用，选项C错误。队列和栈都是操作受限的线性表，但只有队列允许在表的两端进行运算，而栈只允许在栈顶方向进行操作，选项D错误。

12．B

在提取数据时必须保持原来数据的顺序，所以缓冲区的特性是先进先出。

13．A

在中缀表达式转后缀表达式的过程中，扫描到操作数时直接输出，扫描到操作符时根据其优先级进行相应的出入栈操作。有几点需要注意：①若遇到界限符“(”，则直接入栈；②若遇到界限符“)”，则不入栈，且依次弹出栈顶运算符，直到遇到“(”为止，并删除“(”；③若当前运算符的优先级高于栈顶运算符或遇到栈顶为“(”，则直接入栈；④若当前运算符的优先级低于或等于栈顶运算符，则依次弹出栈中的运算符并加入后缀表达式，直到遇到一个优先级低于它的运算符或遇到“(”或栈空为止，之后将当前运算符入栈。

求栈中操作符的最大个数时，为简单起见，可省略对操作数的处理。

步	待处理运算符	栈	扫描运算符	动　作
1	+b-a*((c+d)/e-f)+g		+	+入栈
2	-a*((c+d)/e-f)+g	+	-	-优先级等于栈顶+，弹出+，-入栈
3	*((c+d)/e-f)+g	-	*	*优先级高于栈顶-，*入栈
4	((c+d)/e-f)+g	-*	(	(直接入栈
5	(c+d)/e-f)+g	-*(	(	(直接入栈
6	+d)/e-f)+g	-*((	+	栈顶为(，+直接入栈
7	)/e-f)+g	-*((+	)	遇到)，弹出+，删除(
8	/e-f)+g	-*(	/	栈顶为(，/直接入栈
10	-f)+g	-*(/	-	-优先级低于栈顶/，弹出/，-入栈
11	)+g	-*(-	)	遇到)，弹出-，删除(
12	+g	-*	+	+优先级低于栈顶*，等于-，依次弹出*和-；+入栈
13		+		

由上述过程可知，栈中操作符的最大个数为5。

14．B

中缀表达式a/b+(c*d-e*f)/g转换为后缀表达式的过程如下：

步	待处理序列	栈	后缀表达式	扫描项	动　作
1	a/b+(c*d-e*f)/g			a	a加入后缀表达式
2	/b+(c*d-e*f)/g		a	/	/入栈
3	b+(c*d-e*f)/g	/	a	b	b加入后缀表达式
4	+(c*d-e*f)/g	/	ab	+	+优先级低于栈顶的/，弹出/，+入栈
5	(c*d-e*f)/g	+	ab/	(	(入栈
6	c*d-e*f)/g	+(	ab/	c	c加入后缀表达式
7	*d-e*f)/g	+(	ab/c	*	栈顶为(，*入栈
8	d-e*f)/g	+(*	ab/c	d	d加入后缀表达式
9	-e*f)/g	+(*	ab/cd	-	-优先级低于栈顶的*，弹出*，-入栈
10	e*f)/g	+(-	ab/cd*	e	e加入后缀表达式
11	*f)/g	+(-	ab/cd*e	*	*优先级高于栈顶的-，*入栈
12	f)/g	+(-*	ab/cd*e	f	f加入后缀表达式
13	)/g	+(-*	ab/cd*ef	)	遇到)，依次弹出*、-加入表达式，删除(
14	/g	+	ab/cd*ef*-	/	/优先级高于栈顶的+，/入栈
15	g	+/	ab/cd*ef*-	g	g加入后缀表达式
16		+/	ab/cd*ef*-g		扫描完毕，运算符依次弹出加入表达式
17			ab/cd*ef*-g/+		完成

由此可知，当扫描到 f 时，栈中的元素依次是+(-*。

【另解】采用手算方法，得出中缀式 a/b+(c*d-e*f)/g 对应的后缀式为 ab/cd*ef*-g/+。中缀表达式转后缀表达式时，操作数都直接输出，因此操作数的顺序是固定的。扫描到 f 时，在后缀表达式 f 后面的运算符要么还未入栈，要么还在栈中，需要结合中缀式来判断，f 后面依次出栈的运算符为*-/+，/在中缀表达式 f 之后，此时还未入栈，因此栈中的运算符（从栈底到栈顶）为+-*；此外，已入栈的界限符（此时还未消解，因此(也还在栈中，栈中的元素依次是+(-*。

15. A

递归调用函数时，在系统栈中保存的函数信息需满足先进后出的特点，依次调用了 main(),S(1),S(0)，所以栈底到栈顶的信息依次是 main(),S(1),S(0)。

注　意

在递归中，系统为每一层的返回点、局部变量、传入实参等开辟了递归工作栈来存储。

16. C

根据题意：入队顺序为 8, 4, 2, 5, 3, 9, 1, 6, 7，出队顺序为 1～9。入口和出口之间有多个队列（n 条轨道），且每个队列（轨道）可容纳多个元素（多列列车），为便于区分，队列用字母编号。分析如下：显然先入队的元素必须小于后入队的元素（否则，若 8 和 4 入同一个队列，8 在 4 前面，则出队时也只能 8 在 4 前面），这样 8 入队列 A，4 入队列 B，2 入队列 C，5 入队列 B（按照前述原则“大的元素在小的元素后面”也可将 5 入队列 C，但这时剩下的元素 3 就必须放入一个新的队列中，无法确保“至少”），3 入队列 C，9 入队列 A，这时共占了 3 个队列，后面还有元素 1，直接再用一个新的队列 D，1 从队列 D 出队后，剩下的元素 6 和 7 或入队列 B，或入队列 C。综上，共占用了 4 个队列。当然还有其他的入队、出队情况，请读者自己推演，但要确保满足：①队列中后面的元素大于前面的元素；②确保占用最少（满足题意中“至少”）的队列。

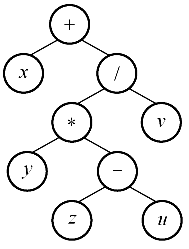
17. C

I 的反例：计算斐波那契数列迭代实现只需要一个循环即可实现。III 的反例：入栈序列为 1, 2，进行 Push, Push, Pop, Pop 操作，出栈次序为 2、1；进行 Push, Pop, Push, Pop 操作，出栈次序为 1, 2。IV，栈是一种受限的线性表，只允许在一端进行操作。II 正确。

18. B

第一次调用：①从 S1 中弹出 2 和 3；②从 S2 中弹出+；③执行 3+2=5；④将 5 压入 S1 中，第一次调用结束后 S1 中剩余 5、8、5（5 在栈顶），S2 中剩余*、-（-在栈顶）。第二次调用：①从 S1 中弹出 5 和 8；②从 S2 中弹出-；③执行 8-5=3；④将 3 压入 S1 中，第二次调用结束后 S1 中剩余 5、3（3 在栈顶），S2 中剩余*。第三次调用：①从 S1 中弹出 3 和 5；②从 S2 中弹出*；③执行 5*3=15；④将 15 压入 S1 中，第三次调用结束后 S1 中仅剩余 15，S2 为空。

19. A



根据中缀表达式画出对应的二叉树如右图所示，对该二叉树进行后序遍历即可得到后缀表达式为 $xyzu-*v/+$。本题也可采用本书前面描述的手算方法。

二、综合应用题

01.【解答】

括号匹配是栈的一个典型应用，给出这道题是希望读者好好掌握栈的应用。算法的基本思想是扫描每个字符，遇到花、方、圆的左括号时入栈，遇到花、方、圆的右括号时检查栈顶元素是否为相应的左括号，若是，出栈，否则配对错误。最后栈若不为空也为错误。

```
bool BracketsCheck(char *str){
    InitStack(S);              //初始化栈
    int i=0;
    while(str[i]!='\0'){
        switch(str[i]){
            //左括号入栈
            case '(': push(S,'('); break;
            case '[': push(S,'['); break;
            case '{': push(S,'{'); break;
            //遇到右括号，检测栈顶
            case ')': Pop(S,e);
                if(e!='(') return false;
            break;
            case ']': pop(S,e);
                if(e!='[') return false;
            break;
            case '}': pop(S,e);
                if(e!='{') return false;
            break;
            default:
            break;
        }//switch
        i++;
    }//while
    if(!IsEmpty(S)){
        printf("括号不匹配\n");
        return false;
    }
    else {
        printf("括号匹配\n");
        return true;
    }
}
```

3.4 数组和特殊矩阵

矩阵在计算机图形学、工程计算中占有举足轻重的地位。在数据结构中考虑的是如何用最小的内存空间来存储同样的一组数据。所以，我们不研究矩阵及其运算等，而把精力放在如何将矩阵更有效地存储在内存中，并能方便地提取矩阵中的元素。

3.4.1 数组的定义

数组是由 n（$n\geq1$）个相同类型的数据元素构成的有限序列，每个数据元素称为一个数组元素，每个元素在 n 个线性关系中的序号称为该元素的下标，下标的取值范围称为数组的维界。

数组与线性表的关系：数组是线性表的推广。一维数组可视为一个线性表；二维数组可视为其元素是定长数组的线性表，以此类推。数组一旦被定义，其维数和维界就不再改变。因此，除结构的初始化和销毁外，数组只会有存取元素和修改元素的操作。

3.4.2 数组的存储结构

大多数计算机语言都提供了数组数据类型，逻辑意义上的数组可采用计算机语言中的数组数据类型进行存储，一个数组的所有元素在内存中占用一段连续的存储空间。

以一维数组 A[0…n-1]为例，其存储结构关系式为

$$\mathrm{LOC}(a_i)=\mathrm{LOC}(a_0)+i\times L\quad(0\leqslant i<n)$$

其中，L 是每个数组元素所占的存储单元。

命题追踪 ▶ 二维数组按行优先存储的下标对应关系（2021）

对于多维数组，有两种映射方法：按行优先和按列优先。以二维数组为例，按行优先存储的基本思想是：先行后列，先存储行号较小的元素，行号相等先存储列号较小的元素。设二维数组的行下标与列下标的范围分别为$[0,h_1]$与$[0,h_2]$，则存储结构关系式为

$$\mathrm{LOC}(a_{i,j})=\mathrm{LOC}(a_{0,0})+[i\times(h_2+1)+j]\times L$$

例如，对于数组 $\boldsymbol{A}_{[2][3]}$，它按行优先方式在内存中的存储形式如图 3.18 所示。

$$\boldsymbol{A}_{[2][3]}=\begin{bmatrix} a_{[0][0]} & a_{[0][1]} & a_{[0][2]} \\ a_{[1][0]} & a_{[1][1]} & a_{[1][2]} \end{bmatrix}$$

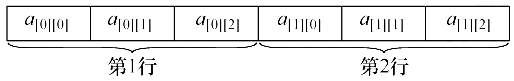


图 3.18　二维数组按行优先顺序存放

当以列优先方式存储时，得出存储结构关系式为

$$\mathrm{LOC}(a_{i,j})=\mathrm{LOC}(a_{0,0})+[j\times(h_1+1)+i]\times L$$

例如，对于数组 $\boldsymbol{A}_{[2][3]}$，它按列优先方式在内存中的存储形式如图 3.19 所示。

$$\boldsymbol{A}_{[2][3]}=\begin{bmatrix} a_{[0][0]} & a_{[0][1]} & a_{[0][2]} \\ a_{[1][0]} & a_{[1][1]} & a_{[1][2]} \end{bmatrix}$$

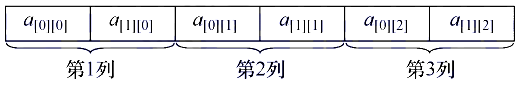


图 3.19　二维数组按列优先顺序存放

3.4.3　特殊矩阵的压缩存储

压缩存储：指为多个值相同的元素只分配一个存储空间，对零元素不分配空间。

特殊矩阵：指具有许多相同矩阵元素或零元素，并且这些相同矩阵元素或零元素的分布有一定规律性的矩阵。常见的特殊矩阵有对称矩阵、上（下）三角矩阵、对角矩阵等。

特殊矩阵的压缩存储方法：找出特殊矩阵中值相同的矩阵元素的分布规律，把那些呈现规律性分布的、值相同的多个矩阵元素压缩存储到一个存储空间中。

1．对称矩阵

命题追踪 ▶ 对称矩阵压缩存储的下标对应关系（2018、2020）

若对一个 n 阶矩阵 $\boldsymbol{A}$ 中的任意一个元素 $a_{i,j}$ 都有 $a_{i,j}=a_{j,i}$（$1\leqslant i,j\leqslant n$），则称其为对称矩阵。其中的元素可以划分为 3 个部分，即上三角区、主对角线和下三角区，如图 3.20 所示。

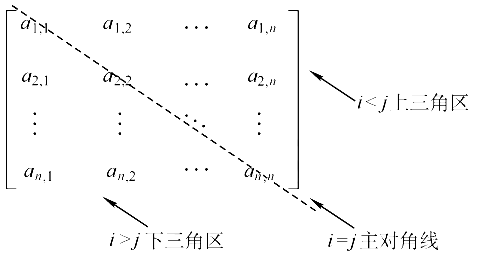


图 3.20　n 阶矩阵的划分

对于 n 阶对称矩阵，上三角区的所有元素和下三角区的对应元素相同，若仍采用二维数组存放，则会浪费几乎一半的空间，为此将 n 阶对称矩阵 $\boldsymbol{A}$ 存放在一维数组 B[n(n+1)/2]中，即元素 $a_{i,j}$ 存放在 b_k 中。比如只存放下三角部分（含主对角线）的元素。

在数组 B 中，位于元素 $a_{i,j}$（$i \geqslant j$）前面的元素个数为

第 1 行：1 个元素（$a_{1,1}$）。

第 2 行：2 个元素（$a_{2,1}$, $a_{2,2}$）。

……

第 $i-1$ 行：$i-1$ 个元素（$a_{i-1,1}, a_{i-1,2}, \cdots, a_{i-1,i-1}$）。

第 i 行：$j-1$ 个元素（$a_{i,1}, a_{i,2}, \cdots, a_{i,j-1}$）。

因此，元素 $a_{i,j}$ 在数组 B 中的下标 $k=1+2+\cdots+(i-1)+j-1=i(i-1)/2+j-1$（数组下标从 0 开始）。因此，元素下标之间的对应关系如下：

$$k=\begin{cases}\dfrac{i(i-1)}{2}+j-1, & i \geqslant j \text{（下三角区和主对角线元素）} \\ \dfrac{j(j-1)}{2}+i-1, & i<j \text{（上三角区元素} a_{i,j}=a_{j,i}\text{）}\end{cases}$$

当数组下标从 1 开始时，可以采用同样的推导方法，请读者自行思考。

注 意

二维数组 A[n][n]和 A[0…n-1][0…n-1]的写法是等价的。若数组写成 A[1…n][1…n]，则表示指定了下标是从 1 开始的。二维数组元素写为 a[i][j]，注意数组元素下标 i 和 j 通常是从 0 开始的。矩阵元素通常写为 $a_{i,j}$，行号 i 和列号 j 通常是从 1 开始的。

2．三角矩阵

下三角矩阵［见图 3.22(a)］中，上三角区的所有元素均为同一常量。其存储思想与对称矩阵类似，不同之处在于存储完下三角区和主对角线上的元素之后，紧接着存储对角线上方的常量一次，所以可以将 n 阶下三角矩阵 $\boldsymbol{A}$ 压缩存储在 B[n(n+1)/2+1]中。

元素下标之间的对应关系为

$$k=\begin{cases}\dfrac{i(i-1)}{2}+j-1, & i \geqslant j \text{（下三角区和主对角线元素）} \\ \dfrac{n(n+1)}{2}, & i<j \text{（上三角区元素）}\end{cases}$$

下三角矩阵在内存中的压缩存储形式如图 3.21 所示。

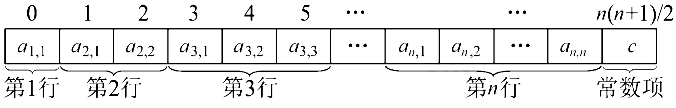


图 3.21 下三角矩阵的压缩存储

命题追踪 ▶ 上三角矩阵采用行优先存储的应用（2011）

上三角矩阵［见图 3.22(b)］中，下三角区的所有元素均为同一常量。只需存储主对角线、上三角区上的元素和下三角区的常量一次，可将其压缩存储在 B[n(n+1)/2+1]中。

$$\begin{bmatrix} a_{1,1} & & & \\ a_{2,1} & a_{2,2} & & \\ \vdots & \vdots & \ddots & \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n} \end{bmatrix} \qquad \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ & a_{2,2} & \cdots & a_{2,n} \\ & & \ddots & \vdots \\ & & & a_{n,n} \end{bmatrix}$$

(a) 下三角矩阵　　(b) 上三角矩阵

图 3.22　三角矩阵

在数组 B 中，位于元素 $a_{i,j}$（$i \leqslant j$）前面的元素个数为

第 1 行：n 个元素

第 2 行：$n-1$ 个元素

……

第 $i-1$ 行：$n-i+2$ 个元素

第 i 行：$j-i$ 个元素

因此，元素 $a_{i,j}$ 在数组 B 中的下标

$$k = n+(n-1)+\cdots+(n-i+2)+(j-i+1)-1 = (i-1)(2n-i+2)/2+(j-i)$$

因此，元素下标之间的对应关系如下：

$$k = \begin{cases} \dfrac{(i-1)(2n-i+2)}{2}+(j-i), & i \leqslant j \text{（上三角区和主对角线元素）} \\ \dfrac{n(n+1)}{2}, & i > j \text{（下三角区元素）} \end{cases}$$

上三角矩阵在内存中的压缩存储形式如图 3.23 所示。

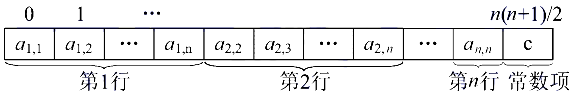


图 3.23　上三角矩阵的压缩存储

以上推导均假设数组的下标从 0 开始，若题设有具体要求，则应该灵活应对。

3．三对角矩阵　公众号：小兔网盘 免费分享无水印PDF

对角矩阵也称带状矩阵。对 n 阶矩阵 $\boldsymbol{A}$ 中的任意一个元素 $a_{i,j}$，当 $|i-j|>1$ 时，若有 $a_{i,j}=0$（$1 \leqslant i,j \leqslant n$），则称为三对角矩阵，如图 3.24 所示。在三对角矩阵中，所有非零元素都集中在以主对角线为中心的 3 条对角线的区域，其他区域的元素都为零。

$$\begin{bmatrix} a_{1,1} & a_{1,2} & & & & \\ a_{2,1} & a_{2,2} & a_{2,3} & & 0 & \\ & a_{3,2} & a_{3,3} & a_{3,4} & & \\ & & \ddots & \ddots & \ddots & \\ & 0 & & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\ & & & & a_{n,n-1} & a_{n,n} \end{bmatrix}$$

图 3.24　三对角矩阵 $\boldsymbol{A}$

三对角矩阵 $\boldsymbol{A}$ 也可以采用压缩存储，将 3 条对角线上的元素按行优先方式存放在一维数组 B 中，且 $a_{1,1}$ 存放于 B[0]中，其存储形式如图 3.25 所示。

$a_{1,1}$	$a_{1,2}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	…	$a_{n-1,n}$	$a_{n,n-1}$	$a_{n,n}$

图 3.25　三对角矩阵的压缩存储

命题追踪 ▶ 三对角矩阵压缩存储的下标对应关系（2016）

由此可以计算矩阵 A 中 3 条对角线上的元素 $a_{i,j}$（$1\leqslant i, j\leqslant n, |i-j|\leqslant 1$）在一维数组 B 中存放的下标为 $k=2i+j-3$。

反之，若已知三对角矩阵中的某个元素 $a_{i,j}$ 存放在一维数组 B 的第 k 个位置，则有 $i=\lfloor (k+1)/3+1\rfloor$，$j=k-2i+3$。例如，当 $k=0$ 时，$i=\lfloor (0+1)/3+1\rfloor=1$，$j=0-2\times 1+3=1$，存放的是 $a_{1,1}$；当 $k=2$ 时，$i=\lfloor (2+1)/3+1\rfloor=2$，$j=2-2\times 2+3=1$，存放的是 $a_{2,1}$；当 $k=4$ 时，$i=\lfloor (4+1)/3+1\rfloor=2$，$j=4-2\times 2+3=3$，存放的是 $a_{2,3}$。

3.4.4　稀疏矩阵

矩阵中非零元素的个数 t，相对矩阵元素的个数 s 来说非常少，即 $s\gg t$ 的矩阵称为稀疏矩阵。例如，一个矩阵的阶为 100×100，该矩阵中只有少于 100 个非零元素。

命题追踪 ▶ 存储稀疏矩阵需要保存的信息（2023）

若采用常规的方法存储稀疏矩阵，则相当浪费存储空间，因此仅存储非零元素。但通常非零元素的分布没有规律，所以仅存储非零元素的值是不够的，还要存储它所在的行和列。因此，将非零元素及其相应的行和列构成一个三元组（行标 i，列标 j，值 $a_{i,j}$），如图 3.26 所示。然后按照某种规律存储这些三元组线性表。稀疏矩阵压缩存储后便失去了随机存取特性。

$$M=\begin{bmatrix}4&0&0&0\\0&0&6&0\\0&9&0&0\\0&23&0&0\end{bmatrix}$$

对应的三元组

i	j	$a_{i,j}$
0	0	4
1	2	6
2	1	9
3	1	23

图 3.26　稀疏矩阵及其对应的三元组

命题追踪 ▶ 适合稀疏矩阵压缩存储的存储结构（2017）

稀疏矩阵的三元组表既可以采用数组存储，又可以采用十字链表存储（见 6.2 节）。当存储稀疏矩阵时，不仅要保存三元组表，而且要保存稀疏矩阵的行数、列数和非零元素的个数。

3.4.5　本节试题精选

单项选择题

01. 对特殊矩阵采用压缩存储的主要目的是（　）。

A. 表达变得简单　　B. 对矩阵元素的存取变得简单

C. 去掉矩阵中的多余元素　　D. 减少不必要的存储空间

02. 对 n 阶对称矩阵压缩存储时，需要表长为（　）的顺序表。

A. $n/2$　　B. $n\times n/2$　　C. $n(n+1)/2$　　D. $n(n-1)/2$

03. 有一个 $n\times n$ 的对称矩阵 A，将其下三角部分按行存放在一维数组 B 中，而 A[0][0]存放于 B[0]中，则元素 A[i][i]存放于 B 中的（　）处。

A. $(i+3)i/2$　　B. $(i+1)i/2$　　C. $(2n-i+1)i/2$　　D. $(2n-i-1)i/2$

04. 在二维数组 A 中，假设每个数组元素的长度为 3 个存储单元，行下标 i 为 0～8，列下标 j 为 0～9，从首地址 SA 开始连续存放。在这种情况下，元素 A[8][5]的起始地址为（　）。

A. SA+141　　B. SA+144　　C. SA+222　　D. SA+255

05. 二维数组 A 按行优先存储，其中每个元素占 1 个存储单元。若 A[1][1]的存储地址为

420，A[3][3]的存储地址为 446，则 A[5][5]的存储地址为（　）。

A. 472　　B. 471　　C. 458　　D. 457

06. 将三对角矩阵即数组 A[1…100][1…100]按行优先存入一维数组 B[1…298]中，数组 A 中元素 A[66][65]在数组 B 中的位置 k 为（　）。

A. 198　　B. 195　　C. 197　　D. 196

07. 若将 n 阶上三角矩阵 $\boldsymbol{A}$ 按列优先级压缩存放在一维数组 B[1…n(n+1)/2+1]中，则存放到 B[k]中的非零元素 $a_{i,j}$（$1 \leqslant i,j \leqslant n$）的下标 i、j 与 k 的对应关系是（　）。

A. $i(i+1)/2+j$　　B. $i(i-1)/2+j-1$　　C. $j(j-1)/2+i$　　D. $j(j-1)/2+i-1$

08. 若将 n 阶下三角矩阵 $\boldsymbol{A}$ 按列优先顺序压缩存放在一维数组 B[1…n(n+1)/2+1]中，则存放到 B[k]中的非零元素 $a_{i,j}$（$1 \leqslant i,j \leqslant n$）的下标 i,j 与 k 的对应关系是（　）。

A. $(j-1)(2n-j+1)/2+i-j$　　B. $(j-1)(2n-j+2)/2+i-j+1$

C. $(j-1)(2n-j+2)/2+i-j$　　D. $(j-1)(2n-j+1)/2+i-j-1$

09. 稀疏矩阵采用压缩存储后的缺点主要是（　）。

A. 无法判断矩阵的行列数　　B. 丧失随机存取的特性

C. 无法由行、列值查找某个矩阵元素　　D. 使矩阵元素之间的逻辑关系更复杂

10. 下列关于矩阵的说法中，正确的是（　）。

I. 在 n（$n>3$）阶三对角矩阵中，每行都有 3 个非零元

II. 稀疏矩阵的特点是矩阵中的元素较少

A. 仅 I　　B. 仅 II　　C. I 和 II　　D. 无正确项

11. 【2016 统考真题】有一个 100 阶的三对角矩阵 $\boldsymbol{M}$，其元素 $m_{i,j}$（$1 \leqslant i, j \leqslant 100$）按行优先依次压缩存入下标从 0 开始的一维数组 N 中。元素 $m_{30,30}$ 在 N 中的下标是（　）。

A．86　　B．87　　C．88　　D．89

12. 【2017 统考真题】适用于压缩存储稀疏矩阵的两种存储结构是（　）。

A．三元组表和十字链表　　B．三元组表和邻接矩阵

C．十字链表和二叉链表　　D．邻接矩阵和十字链表

13. 【2018 统考真题】设有一个 12×12 阶对称矩阵 $\boldsymbol{M}$，将其上三角部分的元素 $m_{i,j}$（$1 \leqslant i \leqslant j \leqslant 12$）按行优先存入 C 语言的一维数组 N 中，元素 $m_{6,6}$ 在 N 中的下标是（　）。

A. 50　　B. 51　　C. 55　　D. 66

14. 【2020 统考真题】将一个 10×10 阶对称矩阵 $\boldsymbol{M}$ 的上三角部分的元素 $m_{i,j}$（$1 \leqslant i \leqslant j \leqslant 10$）按列优先存入 C 语言的一维数组 N 中，元素 $m_{7,2}$ 在 N 中的下标是（　）。

A. 15　　B. 16　　C. 22　　D. 23

15. 【2021 统考真题】二维数组 A 按行优先方式存储，每个元素占用 1 个存储单元。若元素 A[0][0]的存储地址是 100，A[3][3]的存储地址是 220，则元素 A[5][5]的存储地址是（　）。

A. 295　　B. 300　　C. 301　　D. 306

16. 【2023 统考真题】若采用三元组表存储结构存储稀疏矩阵 $\boldsymbol{M}$，则除三元组表外，下列数据中还需要保存的是（　）。

I. $\boldsymbol{M}$ 的行数　　II. $\boldsymbol{M}$ 中包含非零元素的行数

III. $\boldsymbol{M}$ 的列数　　IV. $\boldsymbol{M}$ 中包含非零元素的列数

A. 仅 I、III　　B. 仅 I、IV　　C. 仅 II、IV　　D. I、II、III、IV

3.4.6 答案与解析

单项选择题

01. D

特殊矩阵中含有很多相同元素或零元素，所以可采用压缩存储，以节省存储空间。

02. C

只需存储其上三角或下三角部分（含对角线），元素个数为 $n+(n-1)+\cdots+1=n(n+1)/2$。

03. A

此题要注意 3 个细节：矩阵的最小下标为 0；数组下标也是从 0 开始的；矩阵按行优先存在数组中。注意到此三点，答案不难得到为选项 A。此外，本类题建议采用特殊值代入法求解，例如，A[1][1]对应的下标应为 2，代入后只有选项 A 满足条件。

技 巧

对于特殊三角矩阵压缩存储的题，心中应有“平移”搬动的思想，并结合草图，这样会比较形象，在计算时再注意矩阵和数组的起始下标，就不容易出错。

04. D

二维数组计算地址（按行优先顺序）的公式为

$$\mathrm{LOC}(i,j)=\mathrm{LOC}(0,0)+(i\times m+j)\times L$$

其中，LOC(0, 0) = SA，是数组存放的首地址；$L=3$ 是每个数组元素的长度；$m=9-0+1=10$ 是数组的列数。因此有 LOC(8, 5) = SA + (8×10 + 5)×3 = SA + 255，所以选择选项 D。

05. **A**

本题未直接给出数组 A 的行数和列数，因此需要根据题目中的信息来推理。因为该二维数组按行优先存储，且 A[3][3]的存储地址为 446，所以 A[3][1]的存储地址为 444，又 A[1][1]的存储地址为 420，显然 A[1][1]和 A[3][1]正好相差 2 行，所以该矩阵的列数为 12。而 A[5][3]和 A[3][3]正好相差 2 行，A[5][5]和 A[5][3]又相差 2 个元素，所以 A[5][5]的存储地址是 446 + 24 + 2 = 472。

06. B

对于三对角矩阵，将A[1…n][1…n]压缩至B[1…3n-2]时，$a_{i,j}$与b_k的对应关系为$k=2i+j-2$。则 A 中的元素 A[66][65]在数组 B 中的位置 k 为 2×66 + 65 − 2 = 195。

07. C

按列优先存储，所以元素 $a_{i,j}$ 前面有 $j-1$ 列，共有 $1+2+3+\cdots+j-1=j(j-1)/2$ 个元素，元素 $a_{i,j}$ 在第 j 列上是第 i 个元素，数组 B 的下标是从 1 开始，因此 $k=j(j-1)/2+i$。

08. B

按列优先存储，所以元素 $a_{i,j}$ 之前有 $j-1$ 列，共有 $n+(n-1)+\cdots+(n-j+2)=(j-1)(2n-j+2)/2$ 个元素，元素 $a_{i,j}$ 是第 j 列上第 $i-j+1$ 个元素，数组 B 的下标从 1 开始，$k=(j-1)(2n-j+2)/2+i-j+1$。

09. B

稀疏矩阵通常采用三元组来压缩存储，存储矩阵元素的行列下标和相应的值，因此不能根据矩阵元素的行列下标快速定位矩阵元素，失去了随机存取的特性。

10. **D**

在三对角矩阵中，第 1 行和最后 1 行只有 2 个非零元，其余各行均有 3 个非零元。稀疏矩阵的特点是矩阵中非零元的个数较少。

11．B

三对角矩阵如下所示。

$$\begin{bmatrix} a_{1,1} & a_{1,2} & & & & \\ a_{2,1} & a_{2,2} & a_{2,3} & & 0 & \\ & a_{3,2} & a_{3,3} & a_{3,4} & & \\ & & \ddots & \ddots & \ddots & \\ & 0 & & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\ & & & & a_{n,n-1} & a_{n,n} \end{bmatrix}$$

采用压缩存储，将 3 条对角线上的元素按行优先方式存放在一维数组 B 中，且 $a_{1,1}$ 存放于 B[0] 中，其存储形式如下所示：

$a_{1,1}$	$a_{1,2}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	…	$a_{n-1,n}$	$a_{n,n-1}$	$a_{n,n}$

可以计算矩阵 $\boldsymbol{A}$ 中 3 条对角线上的元素 $a_{i,j}$（$1 \leqslant i, j \leqslant n, |i-j| \leqslant 1$）在一维数组 B 中存放的下标为 $k = 2i + j - 3$，公式很难记忆，我们通常采用解法 2。

解法 1：针对该题，仅需将数字逐一代入公式：$k = 2\times30 + 30 - 3 = 87$，结果为 87。

解法 2：观察上图的三对角矩阵不难发现，第一行有两个元素，剩下的在元素 $m_{30,30}$ 所在行之前的 28 行（注意下标 $1 \leqslant i, j \leqslant 100$）中，每行都有 3 个元素，而 $m_{30,30}$ 之前仅有一个元素 $m_{30,29}$，不难发现元素 $m_{30,30}$ 在数组 N 中的下标是 $2 + 28\times3 + 2 - 1 = 87$。

注　意

矩阵和数组的下标从 0 或 1 开始（如矩阵可能从 $a_{0,0}$ 或 $a_{1,1}$ 开始，数组可能从 B[0] 或 B[1] 开始），这时就需要适时调整计算方法（方法无非是多计算 1 或少计算 1 的问题）。

12．A

三元组表的结点存储了行（row）、列（col）、值（value）三种信息，是主要用来存储稀疏矩阵的一种数据结构。十字链表将行单链表和列单链表结合起来存储稀疏矩阵。邻接矩阵空间复杂度达 $O(n^2)$，不适合于存储稀疏矩阵。二叉链表又名左孩子右兄弟表示法，可用于表示树或森林。

13．A

在 C 语言中，数组 N 的下标从 0 开始。第一个元素 $m_{1,1}$ 对应存入 n_0，矩阵 $\boldsymbol{M}$ 的第一行有 12 个元素，第二行有 11 个，第三行有 10 个，第四行有 9 个，第五行有 8 个，所以 $m_{6,6}$ 是第 $12 + 11 + 10 + 9 + 8 + 1 = 51$ 个元素，下标应为 50。

14．C

上三角矩阵按列优先存储，先存储只有 1 个元素的第一列，再存储有 2 个元素的第二列，以此类推。$m_{7,2}$ 位于左下角，对应右上角的元素为 $m_{2,7}$，在 $m_{2,7}$ 之前存有

第 1 列：1

第 2 列：2

……

第 6 列：6

第 7 列：1

前面共存储有 $1 + 2 + 3 + 4 + 5 + 6 + 1 = 22$ 个元素（数组下标范围为 0～21），注意数组下标从 0 开始，所以 $m_{2,7}$ 在数组 N 中的下标为 22，即 $m_{7,2}$ 在数组 N 中的下标为 22。

15. B

二维数组A按行优先存储，每个元素占用1个存储单元，由A[0][0]和A[3][3]的存储地址可知A[3][3]是二维数组A中的第121个元素，假设二维数组A的每行有n个元素，则$n\times3+4=121$，求得$n=39$，所以元素A[5][5]的存储地址为$100+39\times5+6-1=300$。

16. A

用三元组表存储结构存储稀疏矩阵 $\boldsymbol{M}$ 时，每个非零元素都由三元组（行标、列标、关键字值）组成。但是，仅通过三元组表中的元素无法判断稀疏矩阵 $\boldsymbol{M}$ 的大小，因此还要保存 $\boldsymbol{M}$ 的行数和列数。此外，还可以保存 $\boldsymbol{M}$ 的非零元素个数。例如，如下两个稀疏矩阵的三元组表是相同的，若不保存行数和列数，则无法判断两个稀疏矩阵的大小。

0	2	0	0
0	0	5	0
0	0	0	0
0	0	0	0

0	2	0
0	0	5
0	0	0

归纳总结

本章所讲的几种数据结构类型是线性表的应用和推广，在考试中主要以选择题形式进行考查，但栈和队列也仍然有可能出现在算法设计题中。很多读者看到课本上有好多个函数时很恐惧，若考到了栈或队列的大题，难道要把每个操作的函数都写出来吗？

其实，在考试中，栈或队列都是作为一个工具来解决其他问题的，我们可以把栈或队列的声明和操作写得很简单，而不必分函数写出。以顺序栈的操作为例：

（1）声明一个栈并初始化：

```
Elemtype stack[maxSize]; int top=-1; //两句话连声明带初始化都有了
```

（2）元素入栈：

```
stack[++top]=x;        //仅一句话即实现入栈操作
```

（3）元素x出栈：

```
X=stack[top--];        //单目运算符在变量之前表示“先运算后使用”，之后则相反
```

对于链式栈，同样只需定义一个结构体，然后从讲解中摘取必要的语句组合在自己的函数代码中即可。另外，在考研真题中，链式栈出现的概率要比顺序栈低得多，因此大家应该有所侧重，多训练与顺序栈相关的题目。

思维拓展

设计一个栈，使它可以在$O(1)$的时间复杂度内实现Push、Pop和min操作。所谓min操作，是指得到栈中最小的元素。

提　示

使用双栈，两个栈是同步关系。主栈是普通栈，用来实现栈的基本操作Push和Pop；辅助栈用来记录同步的最小值min，例如元素x入栈，则辅助栈stack_min[top++]= (x<min)?x:min; 即在每次Push中，都将当前最小元素放到stack_min的栈顶。在主栈中Pop最小元素y时，stack_min栈中相同位置的最小元素y也会随着top--而出栈。因此stack_min的栈顶元素必然是y之前入栈的最小元素。本题是典型的以空间换时间的算法。

第4章 串

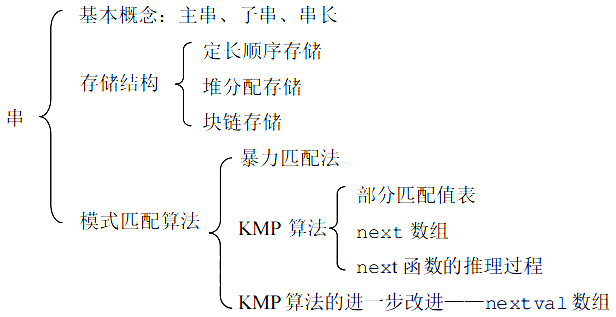
【考纲内容】

字符串模式匹配

扫一扫

视频讲解

【知识框架】



【复习提示】

本章是统考大纲第 6 章内容，采纳读者建议单独作为一章，大纲只要求掌握字符串模式匹配，重点掌握 KMP 匹配算法的原理及 next 数组的推理过程，手工求 next 数组可以先计算出部分匹配值表然后变形，或根据公式来求解。了解 nextval 数组的求解方法。

*4.1 串的定义和实现[①]

字符串简称串，计算机上非数值处理的对象基本都是字符串数据。我们常见的信息检索系统（如搜索引擎）、文本编辑程序（如 Word）、问答系统、自然语言翻译系统等，都是以字符串数据作为处理对象的。本章详细介绍字符串的存储结构及相应的操作。

4.1.1 串的定义

串（string）是由零个或多个字符组成的有限序列。一般记为

$$S='a_1a_2\cdots a_n' \quad (n\geqslant 0)$$

其中，S 是串名，单引号括起来的字符序列是串的值；a_i 可以是字母、数字或其他字符；串中字符的个数 n 称为串的长度。$n=0$ 时的串称为空串（用 $\varnothing$ 表示）。

串中任意多个连续的字符组成的子序列称为该串的子串，包含子串的串称为主串。某个字

① 本节不在统考大纲范围，仅供学习参考。

符在串中的序号称为该字符在串中的位置。子串在主串中的位置以子串的第 1 个字符在主串中的位置来表示。当两个串的长度相等且每个对应位置的字符都相等时，称这两个串是相等的。

例如，有串 A='China Beijing'，B='Beijing'，C='China'，则它们的长度分别为13、7 和 5。B 和 C 是 A 的子串，B 在 A 中的位置是 7，C 在 A 中的位置是 1。

注意，由一个或多个空格（空格是特殊字符）组成的串称为空格串（空格串不是空串），其长度为串中空格字符的个数。

串的逻辑结构和线性表极为相似，区别仅在于串的数据对象限定为字符集。在基本操作上，串和线性表有很大差别。线性表的基本操作主要以单个元素作为操作对象，如查找、插入或删除某个元素等；而串的基本操作通常以子串作为操作对象，如查找、插入或删除一个子串等。

4.1.2 串的基本操作

- StrAssign(&T,chars)：赋值操作。把串 T 赋值为 chars。
- StrCopy(&T,S)：复制操作。由串 S 复制得到串 T。
- StrEmpty(S)：判空操作。若 S 为空串，则返回 TRUE，否则返回 FALSE。
- StrCompare(S,T)：比较操作。若 S>T，则返回值>0；若 S=T，则返回值=0；若 S<T，则返回值<0。
- StrLength(S)：求串长。返回串 S 的元素个数。
- SubString(&Sub,S,pos,len)：求子串。用 Sub 返回串 S 的第 pos 个字符起长度为 len 的子串。
- Concat(&T,S1,S2)：串联接。用 T 返回由 S1 和 S2 联接而成的新串。
- Index(S,T)：定位操作。若主串 S 中存在与串 T 值相同的子串，则返回它在主串 S 中第一次出现的位置；否则函数值为 0。
- ClearString(&S)：清空操作。将 S 清为空串。
- DestroyString(&S)：销毁串。将串 S 销毁。

不同的高级语言对串的基本操作集可以有不同的定义方法。在上述定义的操作中，串赋值 StrAssign、串比较 StrCompare、求串长 StrLength、串联接 Concat 及求子串 SubString 五种操作构成串类型的最小操作子集，即这些操作不可能利用其他串操作来实现；反之，其他串操作（除串清除 ClearString 和串销毁 DestroyString 外）均可在该最小操作子集上实现。

4.1.3 串的存储结构

1. 定长顺序存储表示

类似于线性表的顺序存储结构，用一组地址连续的存储单元来存储串值的字符序列。在串的定长顺序存储结构中，为每个串变量分配一个固定长度的存储区，即定长数组。

```
#define MAXLEN 255        //预定义最大串长为 255
typedef struct{
    char ch[MAXLEN];      //每个分量存储一个字符
    int length;           //串的实际长度
}SString;
```

串的实际长度只能小于或等于 MAXLEN，超过预定义长度的串值会被舍去，称为截断。串长有两种表示方法：一是如上述定义描述的那样，用一个额外的变量 len 来存放串的长度；二是在串值后面加一个不计入串长的结束标记字符“\0”，此时的串长为隐含值。

在一些串的操作（如插入、联接等）中，若串值序列的长度超过上界 MAXLEN，约定用“截

断”法处理，要克服这种弊端，只能不限定串长的最大长度，即采用动态分配的方式。

2．堆分配存储表示

堆分配存储表示仍然以一组地址连续的存储单元存放串值的字符序列，但它们的存储空间是在程序执行过程中动态分配得到的。

```
typedef struct{
    char *ch;              //按串长分配存储区，ch 指向串的基地址
    int length;            //串的长度
}HString;
```

在 C 语言中，存在一个称为堆的自由存储区，并用 malloc()和 free()函数来完成动态存储管理。利用 malloc()为每个新产生的串分配一块实际串长所需的存储空间，若分配成功，则返回一个指向起始地址的指针，作为串的基地址，这个串由 ch 指针来指示；若分配失败，则返回 NULL。已分配的空间可用 free()释放掉。

上述两种存储表示通常为高级程序设计语言所采用。块链存储表示仅做简单介绍。

3．块链存储表示

类似于线性表的链式存储结构，也可采用链表方式存储串值。由于串的特殊性（每个元素只有一个字符），在具体实现时，每个结点既可以存放一个字符，又可以存放多个字符。每个结点称为块，整个链表称为块链结构。图 4.1(a)是结点大小为 4（每个结点存放 4 个字符）的链表，最后一个结点占不满时通常用“#”补上；图 4.1(b)是结点大小为 1 的链表。

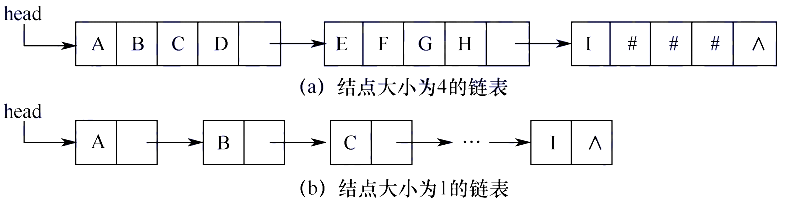


图 4.1　串值的链式存储方式

4.2　串的模式匹配

4.2.1　简单的模式匹配算法

模式匹配是指在主串中找到与模式串（想要搜索的某个字符串）相同的子串，并返回其所在的位置。这里采用定长顺序存储结构，给出一种不依赖于其他串操作的暴力匹配算法。

```
int Index(SString S,SString T){
    int i=1,j=1;
    while(i<=S.length&&j<=T.length){
        if(S.ch[i]==T.ch[j]){
            ++i;++j;              //继续比较后继字符
        }
        else{
            i=i-j+2;j=1;          //指针后退重新开始匹配
        }
    }
    if(j>T.length) return i-T.length;
    else return 0;
}
```

在上述算法中，分别用计数指针 i 和 j 指示主串 S 和模式串 T 中当前待比较的字符位置。

算法思想是：从主串 S 的第一个字符起，与模式串 T 的第一个字符比较，若相等，则继续逐个比较后续字符；否则从主串的下一个字符起，再重新和模式串 T 的字符比较；以此类推，直至模式串 T 中的每个字符依次和主串 S 中的一个连续的字符序列相等，则称匹配成功，函数值为与模式串 T 中第一个字符相等的字符在主串 S 中的序号，否则称匹配不成功，函数值为零。

图 4.2 展示了模式串 T='abcac'和主串 S 的匹配过程。

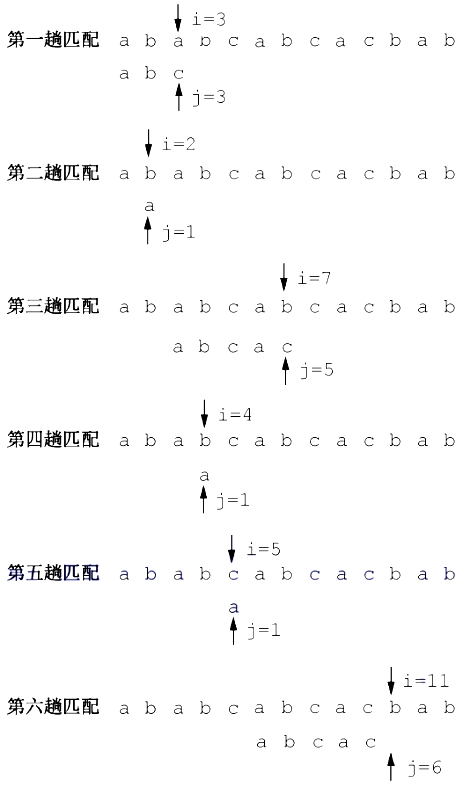


图 4.2　简单模式匹配算法举例

在简单模式匹配算法中，设主串和模式串的长度分别为 n 和 m（$n \gg m$），则最多需要进行 $n-m+1$ 趟匹配，每趟最多需要进行 m 次比较，最坏时间复杂度为 $O(nm)$。例如，当模式串为'0000001'而主串为'00000000000000000000000000000000000000000000 000001'时，由于模式串中的前 6 个字符均为'0'，主串中的前 45 个字符均为'0'，每趟匹配都是比较到模式串中的最后一个字符时才发现不等，整个匹配过程中指针 i 需要回溯 39 次，总比较次数为 40×7＝280 次。

4.2.2　串的模式匹配算法——KMP 算法

在图 4.2 的第三趟匹配过程中，i=7、j=5 的字符比较，结果不等，于是又从 i=4、j=1 重新开始比较。然而，仔细观察会发现，i=4 和 j=1、i=5 和 j=1 以及 i=6 和 j=1 这三次比较都是不必进行的。从第三趟部分匹配的结果可知，主串的第 4 个、第 5 介、第 6 个字符是'b'、'c'、'a'（模式串的第 2、第 3、第 4 个字符），因为模式串的第 1 个字符是'a'，所以再和这三个字符进行比较纯属多余，而只需将模式串向右滑动三个字符的位置，再进行 i=7、j=2 的比较即可。

在简单模式匹配算法中，每趟匹配失败都是模式串向右滑动一位后从头开始比较的。而某趟已匹配相等的字符序列是模式串的某个前缀，因此可从分析模式串本身的结构着手，若已匹配相等的前缀序列中有某个后缀正好是模式串的前缀，则可将模式串向右滑动到与这些相等字符对齐

的位置（也是后面手算 next 数组的依据），主串指针 i 无须回溯，并从该位置开始继续比较。而模式串向右滑动位数的计算仅与模式串本身的结构有关，与主串无关。

1．KMP 算法的原理

要了解模式串的结构，首先要弄清楚几个概念：前缀、后缀和部分匹配值。前缀是指除最后一个字符外，字符串的所有头部子串；后缀是指除第一个字符外，字符串的所有尾部子串；部分匹配值则是指字符串的前缀和后缀的最长相等前后缀长度。下面以'ababa'为例进行说明：

- 'a'的前缀和后缀都为空集，最长相等前后缀长度为 0。
- 'ab'的前缀为{a}，后缀为{b}，{a}∩{b}=∅，最长相等前后缀长度为 0。
- 'aba'的前缀为{a,ab}，后缀为{a,ba}，{a,ab}∩{a,ba}={a}，最长相等前后缀长度为 1。
- 'abab'的前缀{a,ab,aba}∩后缀{b,ab,bab}={ab}，最长相等前后缀长度为 2。
- 'ababa'的前缀{a,ab,aba,abab}∩后缀{a,ba,aba,baba}={a,aba}，公共元素有两个，最长相等前后缀长度为 3。

因此，模式串'ababa'的部分匹配值为 00123。

这个部分匹配值有什么作用呢？

回到最初的问题，主串为'ababcabcacbab'，模式串为'abcac'。

利用上述方法容易求出模式串'abcac'的部分匹配值为 00010，将部分匹配值写成数组形式，就得到了部分匹配值（Partial Match，PM）的表。

编号	1	2	3	4	5
S	a	b	c	a	c
PM	0	0	0	1	0

下面用 PM 表来进行字符串匹配：

```
主串    a  b  a  b  c  a  b  c  a  c  b  a  b
模式串  a  b  c
```

第一趟匹配过程：

发现 c 与 a 不匹配，前面的 2 个字符'ab'是匹配的，查表可知，最后一个匹配字符 b 对应的部分匹配值为 0，因此按照下面的公式算出模式串需要的右滑位数：

右滑位数 = 已匹配的字符数 - 对应的部分匹配值

因为 2 - 0 = 2，所以将模式串向右滑动 2 位如下，进行第二趟匹配：

```
主串    a  b  a  b  c  a  b  c  a  c  b  a  b
模式串        a  b  c  a  c
```

第二趟匹配过程：

发现 c 与 b 不匹配，前面的 4 个字符'abca'是匹配的，查表可知，最后一个匹配字符 a 对应的部分匹配值为 1，4 – 1 = 3，将模式串向右滑动 3 位如下，进行第三趟匹配：

```
主串    a  b  a  b  c  a  b  c  a  c  b  a  b
模式串              a  b  c  a  c
```

第三趟匹配过程：

模式串全部比较完成，匹配成功。整个匹配过程中，主串始终没有回退，所以 KMP 算法可在 $O(n+m)$的时间数量级上完成串的模式匹配操作，大大提高了匹配效率。

某趟发生失配时，若已匹配相等的序列中没有相等的前后缀，则对应的部分匹配值为 0，此时滑动的位数最大，直接将模式串首字符向右滑动到主串当前位置进行下一趟比较；若已匹配相等的序列中存在最大相等前后缀（可理解为首尾重合），则将模式串向右滑动到和主串中该相等后缀对齐（这些重合的字符下一趟显然无需再比较），然后从主串当前位置进行下一趟比较。两种情况的模式串右滑位数都等于“已匹配的字符数 - 对应的部分匹配值”。

还有一种特例，在上述举例中并未出现，当某趟第一个字符比较就失配时，应如何处理呢？此时，应让模式串向右滑动一位，再从主串当前位置的下一位开始比较。

2．next 数组的手算方法

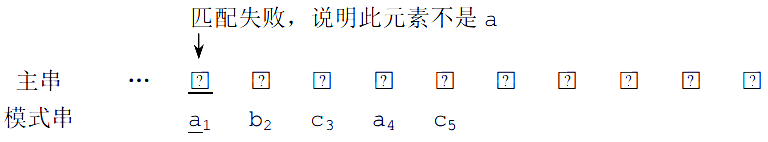
在实际的匹配过程中，模式串在内存中是不会滑动的，发生变化的是指针，前面的举例只是手动模拟 KMP 算法的过程，也是为了让读者更为形象地进行理解。

命题追踪 ▶ KMP 算法中指针变化、比较次数的分析（2015、2019）

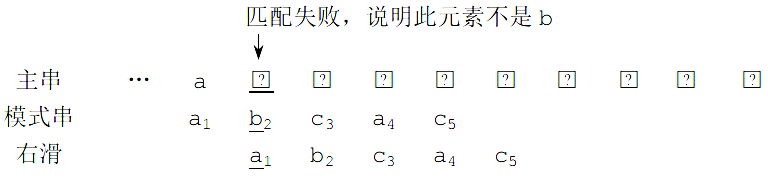
每趟匹配失败时，只有模式串指针 i 在变化，主串指针 j 不会回溯，为此可以定义一个 next 数组，next[j]的含义是当模式串的第 j 个字符失配时，跳到 next[j]位置继续比较。

下面给出一种求 next 数组的手算方法，仍以模式串'abcac'为例。

第 1 个字符失配时，令 next[1]=0，然后指针 i 和 j 同时加 1，即下次将模式串的第 1 个位置与主串当前位置的下一位置进行比较（注意，图中的下标为模式串编号）。

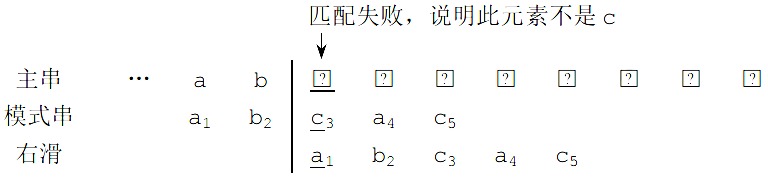


第 2 个字符失配时，令 next[2]=1，模式串的下次比较位置为 1，相当于向右滑动 1 位。注，模式串的 next[1]=0、next[2]=1 都是固定不变的。

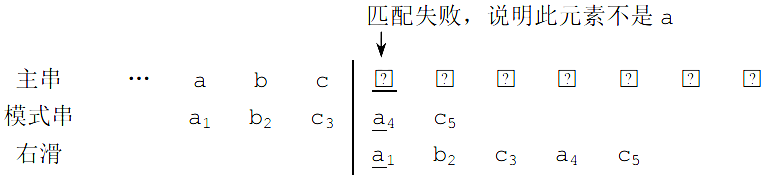


在后面的手算过程中，在不匹配的位置前画一条分界线，模式串一步一步往后退，直到分界线之前能对上（首尾重合），或模式串完全跨过分界线为止。

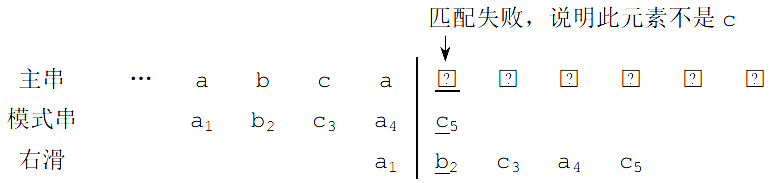
第 3 个字符失配时，模式串的下次比较位置为 1，即 next[3]=1，相当于向右滑动 2 位。



第 4 个字符失配时，模式串的下次比较位置为 1，即 next[4]=1，相当于向右滑动 3 位。



第 5 个字符失配时，模式串的下次比较位置为 2，即 next[5]=2，相当于向右滑动 3 位。



next 数组和 PM 表的关系是怎样的?

通过上述举例，可以推理出 next 数组和 PM 表之间的关系：

next[j] = j-右滑位数 = j-(已匹配的字符数[①]-对应的部分匹配值)

= j-[(j-1)-PM[j-1]]

= PM[j-1]+1

根据上述结论，将模式串'abcac'的 PM 表右移一位，并整体加 1，就得到了模式串'abcac'对应的 next 数组，通过与前面手算的结果比较，可以验证上述结论。

编号	1	2	3	4	5
S	a	b	c	a	c
next	0	1	1	1	2

我们注意到：

1）第一个元素右滑以后空缺的用 0 来填充，因为若是第一个元素匹配失败，则需要将主串指针和模式串指针同步右移一位，从而不需要计算模式串指针移动的位数。

2）最后一个元素在右滑的过程中溢出，因为原来的模式串中，最后一个元素的部分匹配值是其下一个元素使用的，但显然已没有下一个元素，所以可以舍去。

注　意

上述 KMP 算法的举例中，都假设串的编号是从 1 开始的；若串的编号是从 0 开始的，则 next 数组需要整体减 1。

*3．next 数组的推理公式

如何推理 next 数组的一般公式？设主串为'$s_1s_2\cdots s_n$'，模式串为'$p_1p_2\cdots p_m$'，当主串的第 i 个字符与模式串的第 j 个字符失配时，应让主串当前位置与模式串的哪个字符进行比较？

假设此时应与模式串的第 k（k<j）个字符进行比较，则模式串的前 k-1 个字符的子串必须满足下列条件，且不可能存在 k′>k 满足下列条件：

$$'p_1p_2\cdots p_{k-1}'='p_{j-k+1}p_{j-k+2}\cdots p_{j-1}'$$

若存在满足如上条件的子串，则发生失配时，仅需将模式串的第 k 个字符和主串的第 i 个字符对齐，此时模式串的前 k-1 个字符的子串必定与主串的第 i 个字符之前长度为 k-1 的子串相等，因此，只需从模式串的第 k 个字符与主串的第 i 个字符进行比较即可，如图 4.3 所示。

主串	s_1	…	…	…	…	…	s_{i-k+1}	…	s_{i-1}	s_i	…	…	…	…	s_n
子串			p_1	…	p_{k-1}	…	p_{j-k+1}	…	p_{j-1}	p_j	…	p_m			
右滑							p_1	…	p_{k-1}	p_k	…	…	p_m		

图 4.3　模式串右滑到合适位置（阴影对齐部分表示上下字符相等）

① 注意，当模式串匹配到位置 j 失配时，已匹配成功的字符数为 j-1。

当模式串已匹配相等序列中不存在满足上述条件的子串时（可视为 k=1），显然应让主串的第 i 个字符和模式串的第 1 个字符进行比较。

当模式串的第 1 个字符（j=1）与主串的第 i 个字符发生失配时，规定 next[1]=0。

通过上述分析可以得出 next 函数的公式：

$$\text{next}[j]=\begin{cases}0, & j=1\\ \max\{k \mid 1<k<j \text{且} 'p_1\cdots p_{k-1}'='p_{j-k+1}\cdots p_{j-1}'\}, & \text{当此集合不为空时}\\ 1, & \text{其他情况}\end{cases}$$

要用代码来实现，难度貌似还不小，下面来尝试推理求解的科学步骤。

首先由公式可知

$$\text{next}[1]=0$$

设 next[j]=k，此时 k 应满足的条件在上文中已描述。

此时 next[j+1]=? 可能有两种情况：

（1）若 $p_k=p_j$，则表明在模式串中

$$'p_1\cdots p_{k-1}p_k'='p_{j-k+1}\cdots p_{j-1}p_j'$$

且不可能存在 k′>k 满足上述条件，此时 next[j+1]=k+1，即

$$\text{next}[j+1]=\text{next}[j]+1$$

（2）若 $p_k\neq p_j$，则表明在模式串中

$$'p_1\cdots p_{k-1}p_k'\neq'p_{j-k+1}\cdots p_{j-1}p_j'$$

此时可将求 next 函数值的问题视为一个模式匹配问题。用前缀 $p_1\cdots p_k$ 去与后缀 $p_{j-k+1}\cdots p_j$ 匹配，当 $p_k\neq p_j$ 时，应将 $p_1\cdots p_k$ 向右滑动至用第 next[k]个字符与 p_j 进行比较，若 $p_{\text{next}[k]}$ 与 p_j 仍不匹配，则需要寻找长度更短的相等前后缀，下一步继续用 $p_{\text{next}[\text{next}[k]]}$ 与 p_j 进行较，以此类推，直到找到某个更小的 k′=next[next…[k]]（1<k′<k<j），满足条件

$$'p_1\cdots p_{k'}'='p_{j-k'+1}\cdots p_j'$$

则 next[j+1]=k′+1。

也可能不存在任何 k′满足上述条件，即不存在长度更短的相等前后缀，令 next[j+1]=1。

理解起来有点儿费劲？下面举一个简单的例子。

图 4.4 的模式串中已求得 6 个字符的 next 值，现求 next[7]，因为 next[6]=3，又 $p_6\neq p_3$，所以需要比较 p_6 和 p_1（因 next[3]=1），$p_6\neq p_1$，而 next[1]=0，因此 next[7]=1；求 next[8]，因为 $p_7=p_1$，所以 next[8]=next[7]+1=2；求 next[9]，因为 $p_8=p_2$，所以 next[9]=3。

j	1	2	3	4	5	6	7	8	9
模式串	a	b	a	a	b	c	a	b	a
next[j]	0	1	1	2	2	3	?	?	?

图 4.4　求模式串的 next 值

***4．KMP 算法的实现**

通过上述分析写出求 next 值的程序如下：

```
void get_next(SString T,int next[]){
    int i=1,j=0;
    next[1]=0;
    while(i<T.length){
        if(j==0||T.ch[i]==T.ch[j]){
            ++i; ++j;
            next[i]=j;  //若 pi=pj，则 next[j+1]=next[j]+1
        }
```

```
            else
                j=next[j];  //否则令 j=next[j]，循环继续
        }
    }
```

计算机执行起来效率很高，但需要手工计算时，仍然采用前面的方法。

与 next 数组的求解相比，KMP 的匹配算法相对要简单很多，它在形式上与简单的模式匹配算法很相似。不同之处仅在于当匹配过程产生失配时，指针 i 不变，指针 j 退回到 next[j]的位置并重新进行比较，且当指针 j 为 0 时，指针 i 和 j 同时加 1。也就是说，若主串的第 i 个位置和模式串的第 1 个字符不等，则应从主串的第 i+1 个位置开始匹配。具体代码如下：

```
int Index_KMP(SString S,SString T,int next[]){
    int i=1, j=1;
    while(i<=S.length&&j<=T.length){
        if(j==0||S.ch[i]==T.ch[j]){
            ++i; ++j;                  //继续比较后继字符
        }
        else
            j=next[j];                 //模式串向右滑动
    }
    if(j>T.length)
        return i-T.length;             //匹配成功
    else
        return 0;
}
```

尽管普通模式匹配的时间复杂度是 $O(mn)$，KMP 算法的时间复杂度是 $O(m+n)$，但在一般情况下，普通模式匹配的实际执行时间复杂度近似为 $O(m+n)$，因此至今仍被采用。KMP 算法仅在主串与子串有很多“部分匹配”时才显得比普通算法快，其主要优点是主串不回溯。

4.2.3　KMP 算法的进一步优化

命题追踪　nextval 数组的计算（2024）

前面定义的 next 数组在某些情况下尚有缺陷，还可以进一步优化。如图 4.5 所示，模式串'aaaab'在和主串'aaabaaaab'进行匹配时。

主串	a	a	a	b	a	a	a	a	b
模式串	a	a	a	a	b				
j	1	2	3	4	5				
next[j]	0	1	2	3	4				
nextval[j]	0	0	0	0	4				

图 4.5　KMP 算法进一步优化示例

当 i=4、j=4 时，s_4 跟 p_4（b≠a）失配，若用之前的 next 数组，则还需要进行 s_4 与 p_3、s_4 与 p_2、s_4 与 p_1 这 3 次比较。事实上，因为 $p_{next[4]=3}=p_4=a$、$p_{next[3]=2}=p_3=a$、$p_{next[2]=1}=p_2=a$，显然后面 3 次用一个和 p_4 相同的字符跟 s_4 比较毫无意义，必然失配。那么问题出在哪里呢？

问题在于不应该出现 $p_j=p_{next[j]}$。理由是：当 $p_j\neq s_i$ 时，下次匹配必然是 $p_{next[j]}$ 跟 s_i 比较，若 $p_j=p_{next[j]}$，则相当于拿一个和 p_j 相等的字符跟 s_i 比较，这必然导致继续失配，这样的比较毫无意义。若出现 $p_j=p_{next[j]}$，则如何处理呢？

若出现 $p_j=p_{next[j]}$，则需要再次递归，将 next[j]修正为 next[next[j]]，直至两者不相等为止，更新后的数组命名为 nextval。计算 next 数组修正值的算法如下，此时匹配算法不变。

```
void get_nextval(SString T,int nextval[]){
```

```
    int i=1, j=0;
    nextval[1]=0;
    while(i<T.length){
        if(j==0||T.ch[i]==T.ch[j]){
            ++i; ++j;
            if(T.ch[i]!=T.ch[j])  nextval[i]=j;
            else  nextval[i]=nextval[j];
        }
        else
            j=nextval[j];
    }
}
```

KMP算法对于初学者来说可能不太容易掌握，建议读者结合王道课程来理解。

4.2.4 本节试题精选

一、单项选择题

01. 设有两个串S_1和S_2，求S_2在S_1中首次出现的位置的运算称为（ ）。

A. 求子串　　B. 判断是否相等　　C. 模式匹配　　D. 连接

02. KMP算法的特点是在模式匹配时，指示主串的指针（ ）。

A. 不会变大　　B. 不会变小　　C. 都有可能　　D. 无法判断

03. 设主串的长度为n，子串的长度为m，则简单的模式匹配算法的时间复杂度为（ ），KMP算法的时间复杂度为（ ）。

A. $O(m)$　　B. $O(n)$　　C. $O(mn)$　　D. $O(m+n)$

04. 在KMP算法中，用next数组存放模式串的部分匹配信息，当模式串位j与主串位i比较时，两个字符不相等，则j的位移方式是（ ）。

A. j=0　　B. j=j+1　　C. j不变　　D. j=next[j]

05. 在KMP算法中，用next数组存放模式串的部分匹配信息，当模式串位j与主串位i比较时，两个字符不相等，则i的位移方式是（ ）。

A. i=next[i]　　B. i不变　　C. i=0　　D. i=i+1

06. 串'abababaababaa'的next数组为（ ）。

A. 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 9　　B. 0, 1, 2, 1, 2, 1, 1, 1, 1, 2, 1, 2

C. 0, 1, 1, 2, 3, 4, 2, 2, 3, 4, 5, 6　　D. 0, 1, 2, 3, 0, 1, 2, 3, 2, 2, 3, 4, 5

07. 串'abababaababaa'的next数组为（ ）。

A. −1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 8, 8　　B. −1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1

C. −1, 0, 0, 1, 2, 3, 1, 1, 2, 3, 4, 5　　D. −1, 0, 1, 2, −1, 0, 1, 2, 1, 1, 2, 3

08. 设主串S='aabaaaba'，模式串T='aaab'，采用KMP算法进行模式匹配，到匹配成功时为止，在匹配过程中进行的单个字符间的比较次数是（ ）。

A. 10　　B. 9　　C. 8　　D. 7

09. 设主串S='aabaaaba'，模式串T='aaab'，采用改进后的KMP算法进行模式匹配，到匹配成功时为止，在匹配过程中进行的单个字符间的比较次数是（ ）。

A. 9　　B. 8　　C. 7　　D. 6

10. KMP算法使用nextval数组进行模式匹配，模式串为S='ababa**a**a'，当主串中的某字符与S中的第6个字符失配时，S向右滑动的距离是（ ）。

A. 1　　B. 2　　C. 3　　D. 4

11. 【2015统考真题】已知字符串s为'abaabaabacacaabaabcc'，模式串t为'abaabc'。

采用 KMP 算法进行匹配，第一次出现“失配”（s[i]≠t[j]）时，i=j=5，则下次开始匹配时，i 和 j 的值分别是（　）。

A. i=1，j=0　　B. i=5，j=0　　C. i=5，j=2　　D. i=6，j=2

12. 【2019 统考真题】设主串 T='abaabaabcabaabc'，模式串 S='abaabc'，采用 KMP 算法进行模式匹配，到匹配成功时为止，在匹配过程中进行的单个字符间的比较次数是（　）。

A. 9　　B. 10　　C. 12　　D. 15

13. 【2024 统考真题】KMP 算法使用修正后的 next 数组进行模式匹配，模式串为 S='aabaab'，当主串的某个字符与 S 的某个字符失配时，S 向右滑动的最长距离是（　）。

A. 5　　B. 4　　C. 3　　D. 2

二、综合应用题

01. 在字符串模式匹配的 KMP 算法中，求模式的 next 数组值的定义如下：

$$\text{next}[j]=\begin{cases}0, & j=1\\ \max\{k \mid 1<k<j \text{且} 'p_1\cdots p_{k-1}'='p_{j-k+1}\cdots p_{j-1}'\}, & \text{集合不为空}\\ 1, & \text{其他情况}\end{cases}$$

1）当 j=1 时，为什么要取 next[1]=0？

2）为什么要取 max{k}，k 最大是多少？

3）其他情况是什么情况，为什么取 next[j]=1？

02. 设有字符串 S='aabaabaabaac'，P='aabaac'。

1）求出 P 的 next 数组。

2）若 S 作主串，P 作模式串，试给出 KMP 算法的匹配过程。

4.2.5　答案与解析

一、单项选择题

01. C

求子串操作是从串 S 中截取第 i 个字符起长度为 l 的子串，选项 A 错误。选项 B、D 明显错误。

02. B

在 KMP 算法的比较过程中，主串不会回溯，所以主串的指针不会变小。

03. C、D

尽管实际应用中，一般情况下简单的模式匹配算法的时间复杂度近似为 $O(m+n)$，但它的理论时间复杂度还是 $O(mn)$。KMP 算法的时间复杂度为 $O(m+n)$。

04. D

在 KMP 算法中，当主串的第 i 个字符和模式串的第 j 个字符不匹配时，主串的位指针 i 不变，将主串的第 i 个字符与模式串的第 next[j]个字符比较，即 j=next[j]。

05. B

在 KMP 算法中，当主串的第 i 个字符和模式串的第 j 个字符失配时，主串指针 i 不回溯。

06. C

本题采用先求串 S='ababaaababaa'的部分匹配值，再求 next 数组的方法。

- 'a'的前后缀都为空，最长相等前后缀长度为 0。
- 'ab'的前缀{a}∩后缀{b}=∅，最长相等前后缀长度为 0。
- 'aba'的前缀{a,ab}∩后缀{a,ba}={a}，最长相等前后缀长度为 1。

- 'abab'的前缀{a,ab,aba}∩后缀{b,ab,bab}={ab}，最长相等前后缀长度为2。
- ……

依次求出的部分匹配值见下表第3行，将其整体右移一位，低位用-1填充，见下表第4行。

编号	1	2	3	4	5	6	7	8	9	10	11	12
S	a	b	a	b	a	a	a	b	a	b	a	a
PM	0	0	1	2	3	1	1	2	3	4	5	6
next	-1	0	0	1	2	3	1	1	2	3	4	5

选项中next[1]等于0，所以将next数组整体加1。

07. C

解析见上题。读者也可尝试用前面介绍的手算方法。

注意，next数组是否整体加1都正确，需根据题意具体分析。

08. B

假设位序从1开始的，手工计算出T的next数组如下。

编号	1	2	3	4
S	a	a	a	b
next	0	1	2	3

采用KMP匹配算法时：第一趟，经过3次比较后，发现S[3]≠T[3]；第二趟，用S[3]和T[2]（next[3]=2）比较，不相等；第三趟，用S[3]和T[1]（next[2]=1）比较，不相等；第四趟，next[1]=0，因此开始用S[4]和T[1]比较，经过4次比较后，匹配成功。整个匹配过程如下。

	a	a	b	a	a	a	b	a
第一趟	a	a	a	b				
第二趟		a	a	a	b			
第三趟			a	a	a	b		
第四趟				a	a	a	b	

总比较次数为3+1+1+4=9。

09. C

假设位序从1开始的，计算出T的nextval数组如下。

编号	1	2	3	4
S	a	a	a	b
nextval	0	0	0	3

采用改进的KMP匹配算法时：第一趟，经过3次比较后，发现S[3]≠T[3]；第二趟，nextval[3]=0，因此开始用S[4]和T[1]比较，经过4次比较后，匹配成功。整个匹配过程如下。

	a	a	b	a	a	a	b	a
第一趟	a	a	a	b				
第二趟				a	a	a	b	

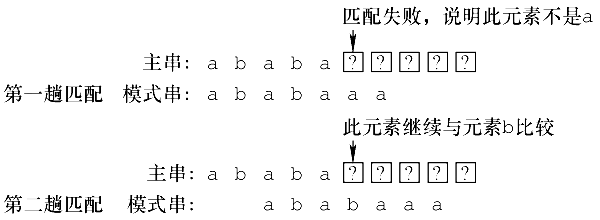
总比较次数为3+4=7。

10. B

假设位序从0开始的，计算出nextval数组。当比较到S[j]失配时，模式串向右滑动的距离为j-nextval[j]（0≤j≤6），由下图可知，当j=5时向右滑动的距离为5-3=2。

编号 j	0	1	2	3	4	5	6
S[j]	a	b	a	b	a	a	a
next[j]	-1	0	0	1	2	3	1
nextval[j]	-1	0	-1	0	-1	3	1
j-nextval[j]	1	1	3	3	5	2	5

此外，若对 KMP 算法很熟悉，则此类题也可直接动手模拟，而不需要求 nextval 数组，与第 6 个字符匹配失败，说明前面的 ababa 匹配成功，S 可向右滑动 2 位继续尝试匹配。



11．C

由题中“失配 s[i]≠t[j]时，i=j=5”，可知题中的主串和模式串的位序都是从 0 开始的（要注意灵活应变）。按照 next 数组生成算法，对于 t 有

编号	0	1	2	3	4	5
t	a	b	a	a	b	c
next	-1	0	0	1	1	2

发生失配时，主串指针 i 不变，子串指针 j 回退到 next[j]位置重新比较，当 s[i]≠t[j]时，i=j=5，由 next 表得知 next[j]=next[5]=2（位序从 0 开始）。因此，i=5，j=2。

12．B

假设位序从 0 开始的，按照 next 数组生成算法，对于 S 有

编号	0	1	2	3	4	5
S	a	b	a	a	b	c
next	-1	0	0	1	1	2

第一趟连续比较 6 次，在模式串的 5 号位和主串的 5 号位匹配失败，模式串的下一个比较位置为 next[5]，即下一次比较从模式串的 2 号位和主串的 5 号位开始，然后直到模式串的 5 号位和主串的 8 号位匹配，第二趟比较 4 次，匹配成功。单个字符的比较次数为 10 次。

13．A

假设位序从 0 开始的，计算出 nextval 数组。当比较到 S[j]失配时，模式串向右滑动的距离为 j－nextval[j]（0≤j≤5），由下图可知，当 j＝4 时向右滑动的距离最长，此时距离为 5。

编号 j	0	1	2	3	4	5
S[j]	a	a	b	a	a	b
next[j]	-1	0	1	0	1	2
nextval[j]	-1	-1	1	-1	-1	1
j-nextval[j]	1	2	1	4	5	4

二、综合应用题

01. 【解答】

1）当模式串的第1个字符与主串的当前字符比较不相等时，next[1]=0，表示模式串应右滑一位，主串当前指针后移一位，再和模式串的第1个字符进行比较。

2）当主串的第 i 个字符与模式串的第 j 个字符失配时，主串 i 不回溯，则假定模式串的第 k 个字符与主串的第 i 个字符比较，k 值应满足条件 1<k<j 且 '$p_1 \cdots p_{k-1}$'= '$p_{j-k+1} \cdots p_{j-1}$'，即 k 为模式串的下次比较位置。k 值可能有多个，为了不使向右滑动丢失可能的匹配，右滑距离应该取最小，因为 j-k 表示右滑的距离，所以取 max{k}。k 的最大值为 j-1。

3）除上面两种情况外，发生失配时，主串指针 i 不回溯，在最坏情况下，模式串从第1个字符开始与主串的第 i 个字符比较。

02. 【解答】

1）P='aabaac'，按照 next 数组生成算法，对于 P 有：

① 设 next[1]=0，next[2]=1。

编号	1	2	3	4	5	6
S	a	a	b	a	a	c
next	0	1				

② j=3 时 k=next[j-1]=next[2]=1，观察 S[j-1](S[2])与 S[k](S[1])是否相等，S[2]=a，S[1]=a，S[2]=S[1]，所以 next[j]=k+1=2。

```
    ↓j-1=2
a   a   b   a   a   c
    a   a   b   a   a   c
   ↑k=1
```

③ j=4 时 k=next[j-1]=next[3]=2，观察 S[j-1](S[3])与 S[k](S[2])是否相等，S[3]=b，S[2]=a，S[3]!=S[2]。

```
      ↓j-1=3
a   a   b   a   a   c
    a   a   b   a   a   c
      ↑k=2
```

此时 k=next[k]=1，观察 S[3]与 S[k](S[1])是否相等，S[3]=b，S[1]=a，S[3]!=S[1]。k=next[k]=0，因为 k=0，所以 next[j]=1。

```
      ↓j-1=3
a   a   b   a   a   c
        a   a   b   a   a   c
  ↑k=0
```

④ j=5 时 k=next[j-1]=next[4]=1，观察 S[j-1](S[4])与 S[k](S[1])是否相等，S[4]=a，S[1]=a，S[4]=S[1]，所以 next[j]=k+1=2。

```
        ↓j-1=4
a   a   b   a   a   c
        a   a   b   a   a   c
        ↑k=1
```

⑤ j=6 时 k=next[j-1]=next[5]=2，观察 S[j-1](S[5])与 S[k](S[2])是否相等，S[5]=a，S[2]=a，S[5]=S[2]，所以 next[j]=k+1=3。

```
                  ↓j-1=5
  a   a   b   a   a   c
          a   a   b   a   a   c
              ↑k=2
```

最后的结果为

编号	1	2	3	4	5	6
S	a	a	b	a	a	c
next	0	1	2	1	2	3

也可以通过求部分匹配值表的方法来求 next 数组。

2）利用 KMP 算法的匹配过程如下。

第一趟：从主串和模式串的第 1 个字符开始比较，失配时 i=6，j=6。

主串	a	a	b	a	a	b	a	a	b	a	a	c
	a	a	b	a	a	c						

第二趟：next[6]=3，主串当前位置和模式串的第 3 个字符继续比较，失配时 i=9，j=6。

主串	a	a	b	a	a	b	a	a	b	a	a	c
				a	a	b	a	a	c			

第三趟：next[6]=3，主串当前位置和模式串的第 3 个字符继续比较，匹配成功。

主串	a	a	b	a	a	b	a	a	b	a	a	c
							a	a	b	a	a	c

归纳总结

学习 KMP 算法时，应从分析暴力法的弊端入手，思考如何去优化它。实际上，已匹配相等的序列就是模式串的某个前缀，因此每次回溯就相当于模式串与模式串的某个前缀在比较，这种频繁的重复比较是其效率低的原因。这时，可从分析模式串本身的结构入手，以便得知当匹配到某个字符不等时，应该向右滑动到什么位置，即已匹配相等的某个前缀若与模式串首尾重合，则对齐它们，对齐部分显然无需再比较，然后直接从主串的当前位置继续开始比较。

思维拓展

编程实现：模式串在主串中有多少个完全匹配的子串？注意，统考不考 KMP 算法题。

第 5 章

树与二叉树

【考纲内容】

扫一扫

视频讲解

（一）树的基本概念

（二）二叉树

二叉树的定义及其主要特征；二叉树的顺序存储结构和链式存储结构；二叉树的遍历；线索二叉树的基本概念和构造

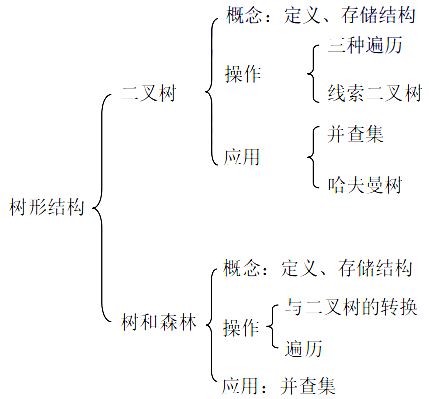
（三）树、森林

树的存储结构；森林与二叉树的转换；树和森林的遍历

（四）树与二叉树的应用

哈夫曼（Huffman）树和哈夫曼编码；并查集及其应用；堆及其应用

【知识框架】



【复习提示】

本章内容多以选择题或综合题的形式考查，但统考也会出涉及树遍历相关的算法题。树和二叉树的性质、遍历操作、转换、存储结构和操作特性等，满二叉树、完全二叉树、线索二叉树、哈夫曼树的定义和性质，都是选择题必然会涉及的内容。

5.1 树的基本概念

5.1.1 树的定义

树是 n（$n \geqslant 0$）个结点的有限集。当 $n=0$ 时，称为空树。在任意一棵非空树中应满足：

1）有且仅有一个特定的称为根的结点。

2）当 $n>1$ 时，其余结点可分为 m（$m>0$）个互不相交的有限集 $T_1, T_2, \cdots, T_m$，其中每个集合本身又是一棵树，并且称为根的子树。

显然，树的定义是递归的，即在树的定义中又用到了其自身，树是一种递归的数据结构。树作为一种逻辑结构，同时也是一种分层结构，具有以下两个特点：

1）树的根结点没有前驱，除根结点外的所有结点有且只有一个前驱。

2）树中所有结点都可以有零个或多个后继。

树适用于表示具有层次结构的数据。树中的某个结点（除根结点外）最多只和上一层的一个结点（其父结点）有直接关系，根结点没有直接上层结点，因此在 n 个结点的树中有 $n-1$ 条边。而树中每个结点与其下一层的零个或多个结点（其孩子结点）都有直接关系。

5.1.2　基本术语

下面结合图 5.1 中的树来说明一些基本术语和概念。

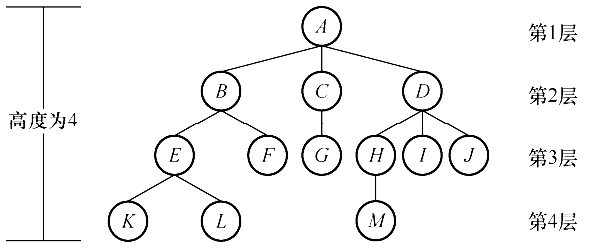


图 5.1　树的树形表示

1）祖先、子孙、双亲、孩子、兄弟和堂兄弟。

考虑结点 K，从根 A 到结点 K 的唯一路径上的所有其他结点，称为结点 K 的祖先。如结点 B 是结点 K 的祖先，而 K 是 B 的子孙，结点 B 的子孙包括 E, F, K, L。路径上最接近结点 K 的结点 E 称为 K 的双亲，而 K 为 E 的孩子。根 A 是树中唯一没有双亲的结点。有相同双亲的结点称为兄弟，如结点 K 和结点 L 有相同的双亲 E，即 K 和 L 为兄弟。双亲在同一层的结点互为堂兄弟，结点 G 与 E, F, H, I, J 互为堂兄弟。

2）结点的层次、深度和高度。

结点的层次从树根开始定义，根结点为第 1 层，它的孩子为第 2 层，以此类推。结点的深度就是结点所在的层次。树的高度（或深度）是树中结点的最大层数。结点的高度是以该结点为根的子树的高度。图 5.1 中树的高度为 4。

3）结点的度和树的度。

树中一个结点的孩子个数称为该结点的度，树中结点的最大度数称为树的度。如结点 B 的度为 2，结点 D 的度为 3，树的度为 3。

4）分支结点和叶结点。

度大于 0 的结点称为分支结点（也称非终端结点）；度为 0（没有孩子结点）的结点称为叶结点（也称终端结点）。在分支结点中，每个结点的分支数就是该结点的度。

5）有序树和无序树。

树中结点的各子树从左到右是有次序的，不能互换，称该树为有序树，否则称为无序树。假设图 5.1 为有序树，若将子结点位置互换，则变成一棵不同的树。

6）路径和路径长度。

树中两个结点之间的路径是由这两个结点之间所经过的结点序列构成的，而路径长度是路径上所经过的边的个数。

注 意

因为树中的分支是有向的，即从双亲指向孩子，所以树中的路径是从上向下的，同一双亲的两个孩子之间不存在路径。

7）森林。

命题追踪 ▶ 森林中树的数量、边数和结点数的关系（2016）

森林是 m（$m \geqslant 0$）棵互不相交的树的集合。森林的概念与树的概念十分相近，因为只要把树的根结点删去就成了森林。反之，只要给 m 棵独立的树加上一个结点，并把这 m 棵树作为该结点的子树，则森林就变成了树。

注 意

上述概念无须刻意记忆，根据实例理解即可。考研时不大可能直接考查概念，而都是结合具体的题目考查。做题时，遇到不熟悉的概念可以翻书，练习得多自然就记住了。

5.1.3 树的性质

树具有如下最基本的性质：

命题追踪 ▶ 树中结点数和度数的关系的应用（2010、2016）

1）树的结点数 n 等于所有结点的度数之和加 1。

结点的度是指该结点的孩子数量，每个结点与其每个孩子都由唯一的边相连，因此树中所有结点的度数之和等于树中的边数之和。树中的结点（除根外）都有唯一的双亲，因此结点数 n 等于边数之和加 1，即所有结点的度数之和加 1。

2）度为 m 的树中第 i 层上至多有 m^{i-1} 个结点（$i \geqslant 1$）。

第 1 层至多有 1 个结点（根结点），第 2 层至多有 m 个结点，第 3 层至多有 m^2 个结点，以此类推。使用数学归纳法可推出第 i 层至多有 m^{i-1} 个结点。

3）高度为 h 的 m 叉树至多有 $(m^h-1)/(m-1)$ 个结点。

当各层结点数达到最大时，树中至多有 $1+m+m^2+\cdots+m^{h-1}=(m^h-1)/(m-1)$ 个结点。

命题追踪 ▶ 指定结点数的三叉树的最小高度分析（2022）

4）度为 m、具有 n 个结点的树的最小高度 h 为 $\lceil \log_m(n(m-1)+1) \rceil$。

为使树的高度最小，在前 $h-1$ 层中，每层的结点数都要达到最大，前 $h-1$ 层最多有 $(m^{h-1}-1)/(m-1)$ 个结点，前 h 层最多有 $(m^h-1)/(m-1)$ 个结点。因此 $(m^{h-1}-1)/(m-1)<n \leqslant (m^h-1)/(m-1)$，即 $h-1<\log_m(n(m-1)+1) \leqslant h$，解得 $h_{\min}=\lceil \log_m(n(m-1)+1) \rceil$。

5）度为 m、具有 n 个结点的树的最大高度 h 为 $n-m+1$。

树的度为 m，因此至少有一个结点有 m 个孩子，它们处于同一层。为使树的高度最大，其他层可仅有一个结点，因此最大高度（层数）为 $n-m+1$。由此，也可逆推出高度为 h、度为 m 的树至少有 $h+m-1$ 个结点。

5.1.4 本节试题精选

一、单项选择题

01. 树最适合用来表示（ ）的数据。

A. 有序　　B. 无序

C. 任意元素之间具有多种联系　　D. 元素之间具有分支层次关系

02. 一棵有 n 个结点的树的所有结点的度数之和为（　）。

A. $n-1$　　B. n　　C. $n+1$　　D. $2n$

03. 树的路径长度是从树根到每个结点的路径长度的（　）。

A. 总和　　B. 最小值　　C. 最大值　　D. 平均值

04. 对于一棵具有 n 个结点、度为 4 的树来说，（　）。

A. 树的高度至多是 $n-3$　　B. 树的高度至多是 $n-4$

C. 第 i 层上至多有 $4(i-1)$个结点　　D. 至少在某一层上正好有 4 个结点

05. 度为 4、高度为 h 的树，（　）。

A. 至少有 $h+3$ 个结点　　B. 至多有 $4h-1$ 个结点

C. 至多有 $4h$ 个结点　　D. 至少有 $h+4$ 个结点

06. 假定一棵度为 3 的树中，结点数为 50，则其最小高度为（　）。

A. 3　　B. 4　　C. 5　　D. 6

07. 设有一棵度为 3 的树，其中度为 3 的结点数 $n_3=2$，度为 2 的结点数 $n_2=1$，叶结点数 $n_0=6$，则该树的结点总数为（　）。

A. 12　　B. 9　　C. 10　　D. ≥9 的任意整数

08. 设一棵 m 叉树中有 N_1 个度数为 1 的结点，N_2 个度数为 2 的结点……N_m 个度数为 m 的结点，则该树中共有（　）个叶结点。

A. $\sum_{i=1}^{m}(i-1)N_i$　　B. $\sum_{i=1}^{m}N_i$　　C. $\sum_{i=2}^{m}(i-1)N_i$　　D. $\sum_{i=2}^{m}(i-1)N_i+1$

09. 【2010 统考真题】在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度为 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶结点个数是（　）。

A. 41　　B. 82　　C. 113　　D. 122

10. 【2016 统考真题】若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是（　）。

A. 8　　B. 9　　C. 10　　D. 11

二、综合应用题

01. 含有 n 个结点的三叉树的最小高度是多少？

02. 已知一棵度为 4 的树中，度为 0, 1, 2, 3 的结点数分别为 14, 4, 3, 2，求该树的结点总数 n 和度为 4 的结点个数，并给出推导过程。

03. 已知一棵度为 m 的树中，有 n_1 个度为 1 的结点，有 n_2 个度为 2 的结点……有 n_m 个度为 m 的结点，问该树有多少个叶结点？

5.1.5　答案与解析

一、单项选择题

01. D

树是一种分层结构，它特别适合组织那些具有分支层次关系的数据。

02. A

除根结点外，其他每个结点都是某个结点的孩子，因此树中所有结点的度数加 1 等于结点数，

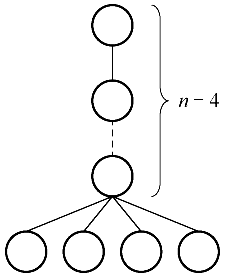
即所有结点的度数之和等于总结点数减1。这是一个重要的结论，做题时经常用到。

03．A

树的路径长度是指树根到每个结点的路径长的总和，根到每个结点的路径长度的最大值应是树的高度减1。注意与哈夫曼树的带权路径长度相区别。

04．A

要使得具有 n 个结点、度为4的树的高度最大，就要使得每层的结点数尽可能少，类似下图所示的树，除最后一层外，每层的结点数是1，最终该树的高度为 $n-3$。树的度为4只能说明存在某结点正好（也最多）有4个孩子结点，选项D错误。



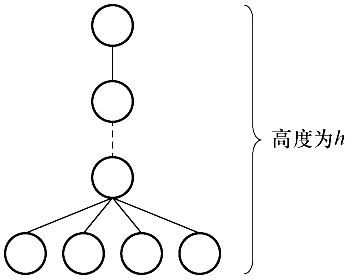
05．A

要使得度为4、高度为 h 的树的总结点数最少，需要满足以下两个条件：

① 至少有一个结点有4个分支。

② 每层的结点数目尽可能少。

情况类似下图所示的树，结点个数为 $h+3$。



要使得度为4、高度为 h 的树的总结点数最多，应使每个非叶结点的度均为4，即满树，总结点个数最多为 $1+4+4^2+\cdots+4^{h-1}$。

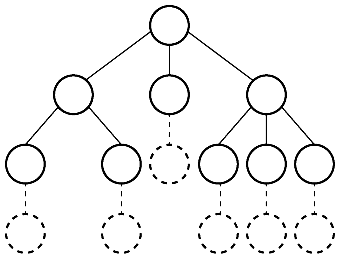
对于上面的两题，应画出草图来求解，就能一目了然。

06．C

要求满足条件的树，那么该树是一棵完全三叉树。在度为3的完全三叉树中，第1层有1个结点，第2层有 $3^1=3$ 个结点，第3层有 $3^2=9$ 个结点，第4层有 $3^3=27$ 个结点，因此结点数之和为 $1+3+9+27=40$，第5层的结点数 $=50-40=10$ 个，因此最小高度为5。

07．D

总结点数 $n=n_0+n_1+n_2+n_3=6+n_1+1+2=n_1+9$，总度数 $=n-1=n_1+2n_2+3n_3=n_1+2+6=n_1+8$，根据题目条件无法得出 n 的具体值，只能证明 n 是一个大于或等于9的任意整数。画出满足题目条件的树，可以是如下图所示的一棵树，该树中无法确定 n 的具体数量。



08．D

设叶结点数为 N_0，总结点数为 N，则 $N=N_1+2N_2+3N_3+\cdots+mN_m+1$，又因为 $N=N_0+N_1+N_2+N_3+\cdots+N_m$，所以 $N_0=N_2+2N_3+\cdots+(m-1)N_m+1=\sum_{i=2}^{m}(i-1)N_i+1$。

09．B

设树中度为 i（$i=0,1,2,3,4$）的结点数分别为 n_i，树中结点总数为 n，则 $n=$ 分支数 $+1$，而分支数又等于树中各结点的度之和，即 $n=1+n_1+2n_2+3n_3+4n_4=n_0+n_1+n_2+n_3+n_4$。依题意，$n_1+2n_2+3n_3+4n_4=10+2+30+80=122$，$n_1+n_2+n_3+n_4=10+1+10+20=41$，可得出 $n_0=82$，即树 T 的叶结点的个数是 82。

10．C

解法 1：树有一个重要性质，即在 n 个结点的树中有 $n-1$ 条边，“那么对于每棵树，其结点数比边数多 1”。本题森林中的结点数比边数多 10（$25-15=10$），显然共有 10 棵树。

解法 2：仔细分析后发现此题也是考查图的性质：生成树和生成森林。对于图的生成树有一个重要的性质，即图中顶点数若为 n，则其生成树含有 $n-1$ 条边。对比解法 1 中树的性质，不难发现两种解法都用到了性质“树中结点数比边数多 1”，后面的分析如解法 1。

二、综合应用题

01．【解答】

要求含有 n 个结点的三叉树的最小高度，那么满足条件的一定是一棵完全三叉树，设含有 n 个结点的完全三叉树的高度为 h，第 h 层至少有 1 个结点，至多有 3^{h-1} 个结点。则有

$$1+3^1+3^2+\cdots+3^{h-2}<n\leqslant 1+3^1+3^2+\cdots+3^{h-2}+3^{h-1}$$

即 $(3^{h-1}-1)/2<n\leqslant(3^h-1)/2$，得 $3^{h-1}<2n+1\leqslant 3^h$，即 $h<\log_3(2n+1)+1$，$h\geqslant\log_3(2n+1)$。

因为 h 只能为正整数，$h=\lceil\log_3(2n+1)\rceil$，所这种三叉树的最小高度是 $\lceil\log_3(2n+1)\rceil$。

02．【解答】

设树中度为 i（$i=0,1,2,3,4$）的结点数为 n_i，则结点总数 $n=n_0+n_1+n_2+n_3+n_4$，即 $n=23+n_4$，根据“树中所有结点的度数加 1 等于结点数”的结论，有 $n=0+n_1+2n_2+3n_3+4n_4+1$，即有 $n=17+4n_4$。

综合两式得 $n_4=2$，$n=25$。所以该树的结点总数为 25，度为 4 的结点个数为 2。

03．【解答】

树中的结点数等于所有结点的度数加 1，因此有 $n=\sum_{i=0}^{m}in_i+1=n_1+2n_2+3n_3+\cdots+mn_m+1$。

又有 $n=n_0+n_1+n_2+\cdots+n_m$，所以

$$\begin{aligned}n_0&=(n_1+2n_2+3n_3+\cdots+mn_m+1)-(n_1+n_2+\cdots+n_m)\\&=n_2+2n_3+\cdots+(m-1)n_m+1=1+\sum_{i=2}^{m}(i-1)n_i\end{aligned}$$

注　意

综合以上几题，常用于求解树结点与度之间关系的有：

① 总结点数 $= n_0 + n_1 + n_2 + \cdots + n_m$。

② 总分支数 $= 1n_1 + 2n_2 + \cdots + mn_m$（度为 m 的结点引出 m 条分支）。

③ 总结点数 = 总分支数 + 1。

这类题目常在选择题中出现，读者对以上关系应当熟练掌握并灵活应用。

5.2　二叉树的概念

5.2.1　二叉树的定义及其主要特性

1．二叉树的定义

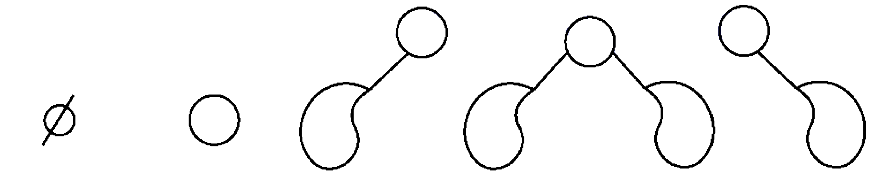
二叉树是一种特殊的树形结构，其特点是每个结点至多只有两棵子树（二叉树中不存在度大于 2 的结点），并且二叉树的子树有左右之分，其次序不能任意颠倒。

与树相似，二叉树也以递归的形式定义。二叉树是 n（$n \geqslant 0$）个结点的有限集合：

① 或者为空二叉树，即 $n = 0$。

② 或者由一个根结点和两个互不相交的被称为根的左子树和右子树组成。左子树和右子树又分别是一棵二叉树。

二叉树是有序树，若将其左、右子树颠倒，则成为另一棵不同的二叉树。即使树中结点只有一棵子树，也要区分它是左子树还是右子树。二叉树的 5 种基本形态如图 5.2 所示。



(a) 空二叉树　(b) 只有根结点　(c) 只有左子树　(d) 左右子树都有　(e) 只有右子树

图 5.2　二叉树的 5 种基本形态

二叉树与度为 2 的有序树的区别：

① 度为 2 的树至少有 3 个结点，而二叉树可以为空。

② 度为 2 的有序树的孩子的左右次序是相对于另一个孩子而言的，若某个结点只有一个孩子，则这个孩子就无须区分其左右次序，而二叉树无论其孩子数是否为 2，均需确定其左右次序，即二叉树的结点次序不是相对于另一结点而言的，而是确定的。

2．几种特殊的二叉树

1）满二叉树。一棵高度为 h，且有 $2^h - 1$ 个结点的二叉树称为满二叉树，即二叉树中的每层都含有最多的结点，如图 5.3(a)所示。满二叉树的叶结点都集中在二叉树的最下一层，并且除叶结点之外的每个结点度数均为 2。

可以对满二叉树按层序编号：约定编号从根结点（根结点编号为 1）起，自上而下，自左向右。这样，每个结点对应一个编号，对于编号为 i 的结点，若有双亲，则其双亲为$\lfloor i/2 \rfloor$，若有左孩子，则左孩子为 $2i$；若有右孩子，则右孩子为 $2i + 1$。

命题追踪 ▶ 完全二叉树中结点数和叶结点数的关系（2009、2011、2018）

2）完全二叉树。高度为 h、有 n 个结点的二叉树，当且仅当其每个结点都与高度为 h 的满二叉树中编号为 1～n 的结点一一对应时，称为完全二叉树，如图 5.3(b)所示。

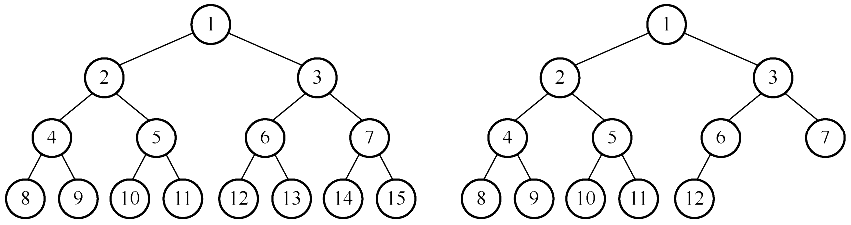


图 5.3　两种特殊形态的二叉树[①]

3）二叉排序树。左子树上所有结点的关键字均小于根结点的关键字；右子树上所有结点的关键字均大于根结点的关键字；左子树和右子树又各是一棵二叉排序树。

4）平衡二叉树。树中任意一个结点的左子树和右子树的高度之差的绝对值不超过 1。关于二叉排序树和平衡二叉树的详细介绍，见本书中的 7.3 节。

命题追踪 ▶ 正则 k 叉树树高和结点数的关系的应用（2016）

5）正则二叉树。树中每个分支结点都有 2 个孩子，即树中只有度为 0 或 2 的结点。

3．二叉树的性质

1）非空二叉树上的叶结点数等于度为 2 的结点数加 1，即 $n_0 = n_2 + 1$。

证明：设度为 0, 1 和 2 的结点个数分别为 n_0, n_1 和 n_2，结点总数 $n = n_0 + n_1 + n_2$。

再看二叉树中的分支数，除根结点外，其余结点都有一个分支进入，设 B 为分支总数，则 $n = B + 1$。这些分支是由度为 1 或 2 的结点射出的，因此又有 $B = n_1 + 2n_2$。

于是得 $n_0 + n_1 + n_2 = n_1 + 2n_2 + 1$，则 $n_0 = n_2 + 1$。

注　意

该性质经常在选择题中涉及，希望读者牢记并灵活应用。

2）非空二叉树的第 k 层最多有 2^{k-1} 个结点（$k \geqslant 1$）。

第 1 层最多有 $2^{1-1} = 1$ 个结点（根），第 2 层最多有 $2^{2-1} = 2$ 个结点，以此类推，可以证明其为一个公比为 2 的等比数列 2^{k-1}。

3）高度为 h 的二叉树至多有 $2^h - 1$ 个结点（$h \geqslant 1$）。

该性质利用性质 2 求前 h 项的和，即等比数列求和的结果。

注　意

性质 2 和性质 3 还可以拓展到 m 叉树的情况，即 m 叉树的第 k 层最多有 m^{k-1} 个结点，高度为 h 的 m 叉树至多有 $(m^h - 1)/(m - 1)$ 个结点。

4）对完全二叉树按从上到下、从左到右的顺序依次编号 $1, 2, \cdots, n$，则有以下关系：

① 最后一个分支结点的编号为 $\lfloor n/2 \rfloor$，若 $i \leqslant \lfloor n/2 \rfloor$，则结点 i 为分支结点，否则为叶结点。

① 完全二叉树可视为从满二叉树中删去若干最底层、最右边的一些连续叶结点后所得到的二叉树。

② 叶结点只可能在最后两层上出现（相当于在相同高度的满二叉树的最底层、最右边减少一些连续叶结点，当减少 2 个或以上叶结点时，次底层将出现叶结点）。

③ 若有度为 1 的结点，则最多只可能有一个，且该结点只有左孩子而无右孩子（度为 1 的分支结点只可能是最后一个分支结点，其结点编号为$\lfloor n/2 \rfloor$）。

④ 按层序编号后，一旦出现某结点（如编号 i）为叶结点或只有左孩子的情况，则编号大于 i 的结点均为叶结点（与结论①和结论③是相通的）。

⑤ 若 n 为奇数，则每个分支结点都有左、右孩子；若 n 为偶数，则编号最大的分支结点（编号为 $n/2$）只有左孩子，没有右孩子，其余分支结点都有左、右孩子。

⑥ 当 $i>1$ 时，结点 i 的双亲结点的编号为$\lfloor i/2 \rfloor$。

⑦ 若结点 i 有左、右孩子，则左孩子编号为 $2i$，右孩子编号为 $2i+1$。

⑧ 结点 i 所在层次（深度）为$\lfloor \log_2 i \rfloor + 1$。

5）具有 n 个（$n>0$）结点的完全二叉树的高度为$\lceil \log_2(n+1) \rceil$或$\lfloor \log_2 n \rfloor + 1$。

设高度为 h，根据性质 3 和完全二叉树的定义有

$$2^{h-1}-1<n\leqslant 2^h-1 \quad \text{或者} \quad 2^{h-1}\leqslant n<2^h$$

得 $2^{h-1}<n+1\leqslant 2^h$，即 $h-1<\log_2(n+1)\leqslant h$，因为 h 为正整数，所以 $h=\lceil \log_2(n+1) \rceil$，或者得 $h-1\leqslant \log_2 n<h$，所以 $h=\lfloor \log_2 n \rfloor+1$。

5.2.2　二叉树的存储结构

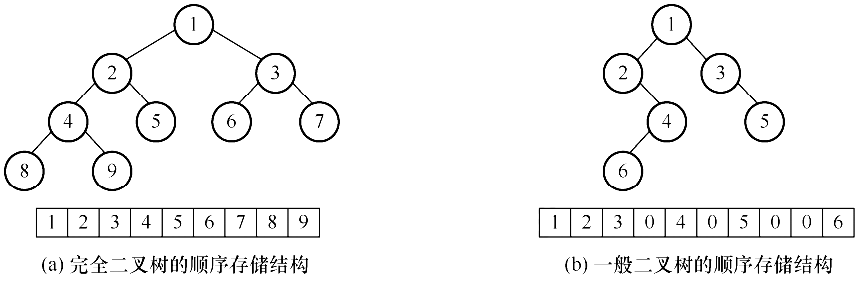
1．顺序存储结构

二叉树的顺序存储是指用一组连续的存储单元依次自上而下、自左至右存储完全二叉树上的结点元素，即将完全二叉树上编号为 i 的结点元素存储在一维数组下标为 $i-1$ 的分量中。

依据二叉树的性质，完全二叉树和满二叉树采用顺序存储比较合适，树中结点的序号可以唯一地反映结点之间的逻辑关系，这样既能最大可能地节省存储空间，又能利用数组元素的下标值确定结点在二叉树中的位置，以及结点之间的关系。

命题追踪 ▶ 特定条件下二叉树树形及占用存储空间的分析（2020）

但对于一般的二叉树，为了让数组下标能反映二叉树中结点之间的逻辑关系，只能添加一些并不存在的空结点，让其每个结点与完全二叉树上的结点相对照，再存储到一维数组的相应分量中。然而，在最坏情况下，一个高度为 h 且只有 h 个结点的单支树却需要占据近 2^h-1 个存储单元。二叉树的顺序存储结构如图 5.4 所示，其中 0 表示并不存在的空结点。



(a) 完全二叉树的顺序存储结构　　(b) 一般二叉树的顺序存储结构

图 5.4　二叉树的顺序存储结构

注　意

建议从数组下标 1 开始存储树中的结点，保证数组下标和结点编号一致。

2．链式存储结构

顺序存储的空间利用率较低，因此二叉树一般都采用链式存储结构，用链表结点来存储二叉树中的每个结点。在二叉树中，结点结构通常包括若干数据域和若干指针域，二叉链表至少包含 3 个域：数据域 data、左指针域 lchild 和右指针域 rchild，如图 5.5 所示。

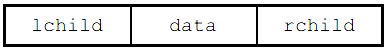


图 5.5　二叉树链式存储的结点结构

图 5.6 所示为一棵二叉树及其对应的二叉链表。而实际上在不同的应用中，还可以增加某些指针域，如增加指向父结点的指针后，变为三叉链表的存储结构。

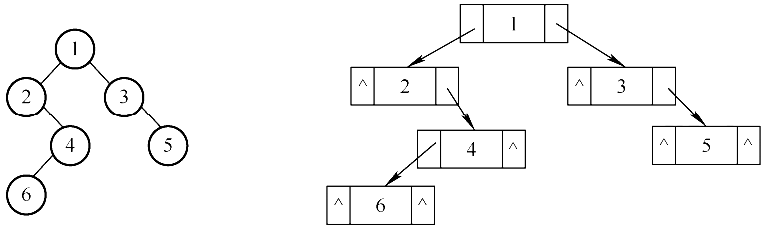


图 5.6　二叉链表的存储结构

二叉树的链式存储结构描述如下：

```
typedef struct BiTNode{
    ElemType data;                          //数据域
    struct BiTNode *lchild,*rchild;         //左、右孩子指针
}BiTNode,*BiTree;
```

使用不同的存储结构时，实现二叉树操作的算法也会不同，因此要根据实际应用场合（二叉树的形态和需要进行的运算）来选择合适的存储结构。

容易验证，在含有 n 个结点的二叉链表中，含有 $n+1$ 个空链域（重要结论，经常出现在选择题中）。在下一节中，我们将利用这些空链域来组成另一种链表结构——线索链表。

5.2.3　本节试题精选

一、单项选择题

01. 下列关于二叉树的说法中，正确的是（　）。

A. 度为 2 的有序树就是二叉树

B. 含有 n 个结点的二叉树的高度为 $\lfloor \log_2 n \rfloor + 1$

C. 在完全二叉树中，若一个结点没有左孩子，则它必是叶结点

D. 含有 n 个结点的完全二叉树的高度为 $\lceil \log_2 n \rceil$

02. “二叉树为空”意味着二叉树（　）。

A. 根结点没有子树　　B. 不存在

C. 没有结点　　D. 由一些没有赋值的空结点构成

03. 下列关于完全二叉树的说法中，正确的是（　）。

A. 在完全二叉树中，叶结点的双亲的左兄弟（若存在）一定不是叶结点

B. 任何一棵二叉树中，叶结点数为度为 2 的结点数减 1，即 $n_0 = n_2 - 1$

C. 完全二叉树不适合顺序存储结构，只有满二叉树适合顺序存储结构

D. 结点按完全二叉树层序编号的二叉树中，第 i 个结点的左孩子的编号为 $2i$

04. 具有10个叶结点的二叉树中有（ ）个度为2的结点。

A. 8　　B. 9　　C. 10　　D. 11

05. 设高度为 h 的二叉树上只有度为0和度为2的结点，则此类二叉树中所包含的结点数至少为（ ）。

A. h　　B. $2h-1$　　C. $2h+1$　　D. $h+1$

06. 具有 n 个结点且高度为 n 的二叉树的数目为（ ）。

A. $\log_2 n$　　B. $n/2$　　C. n　　D. 2^{n-1}

07. 假设一棵二叉树的结点个数为50，则它的最小高度是（ ）。

A. 4　　B. 5　　C. 6　　D. 7

08. 设二叉树有 $2n$ 个结点，且 $m<n$，则不可能存在（ ）的结点。

A. n 个度为0　　B. $2m$ 个度为0　　C. $2m$ 个度为1　　D. $2m$ 个度为2

09. 一个具有1025个结点的二叉树的高 h 为（ ）。

A. 11　　B. 10　　C. 11～1025　　D. 10～1024

10. 设二叉树只有度为0和2的结点，其结点个数为15，则该二叉树的最大深度为（ ）。

A. 4　　B. 5　　C. 8　　D. 9

11. 高度为 h 的完全二叉树最少有（ ）个结点。

A. 2^h　　B. 2^h+1　　C. 2^{h-1}　　D. 2^h-1

12. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则完全二叉树的结点个数最少是（ ）。

A. 39　　B. 52　　C. 111　　D. 119

13. 若一棵深度为6的完全二叉树的第6层有3个叶结点，则该二叉树共有（ ）个叶结点。

A. 17　　B. 18　　C. 19　　D. 20

14. 一棵完全二叉树上有1001个结点，其中叶结点的个数是（ ）。

A. 250　　B. 500　　C. 254　　D. 501

15. 若一棵二叉树有126个结点，在第7层（根结点在第1层）至多有（ ）个结点。

A. 32　　B. 64　　C. 63　　D. 不存在第7层

16. 一棵有124个叶结点的完全二叉树，最多有（ ）个结点。

A. 247　　B. 248　　C. 249　　D. 250

17. 某完全二叉树 T 中，结点个数最大的层有8个结点，则 T 中至多有（ ）个结点。

A. 8　　B. 15　　C. 23　　D. 31

18. 一棵有 n 个结点的二叉树采用二叉链存储结点，其中空指针数为（ ）。

A. n　　B. $n+1$　　C. $n-1$　　D. $2n$

19. 设有 n（$n\geq 1$）个结点的二叉树采用三叉链表表示，其中每个结点包含三个指针，分别指向其左孩子、右孩子及双亲（若不存在，则置为空），则下列说法中正确的是（ ）。

I. 树中空指针的数量为 $n+2$　　II. 所有度为2的结点均被三个指针指向

III. 每个叶结点均被一个指针所指向

A. I　　B. I、II　　C. I、III　　D. II、III

20. 在一棵完全二叉树中，其根的序号为1，（ ）可判定序号为 p 和 q 的两个结点是否在同一层。

A. $\lfloor\log_2 p\rfloor=\lfloor\log_2 q\rfloor$　　B. $\log_2 p=\log_2 q$

C. $\lfloor\log_2 p\rfloor+1=\lfloor\log_2 q\rfloor$　　D. $\lfloor\log_2 p\rfloor=\lfloor\log_2 q\rfloor+1$

21. 在一个用数组表示的完全二叉树中，根结点的下标为1，那么下标为17和19的结点的

最近公共祖先的下标是（　）。

A. 1　　B. 2　　C. 4　　D. 8

22. 假定一棵三叉树的结点数为 50，则它的最小高度为（　）。

A. 3　　B. 4　　C. 5　　D. 6

23. 具有 n 个结点的三叉树用三叉链表表示，则树中空指针域的个数为（　）。

A. $3n+1$　　B. $2n+1$　　C. $3n-1$　　D. $3n$

24. 对于一棵满二叉树，共有 n 个结点和 m 个叶结点，高度为 h，则（　）。

A. $n=h+m$　　B. $n+m=2h$　　C. $m=h-1$　　D. $n=2^h-1$

25. 【2009 统考真题】已知一棵完全二叉树的第 6 层（设根为第 1 层）有 8 个叶结点，则该完全二叉树的结点个数最多是（　）。

A. 39　　B. 52　　C. 111　　D. 119

26. 【2011 统考真题】若一棵完全二叉树有 768 个结点，则该二叉树中叶结点的个数是（　）。

A. 257　　B. 258　　C. 384　　D. 385

27. 【2018 统考真题】设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有 2 个子结点。若 T 有 k 个叶结点，则 T 的结点总数是（　）。

A. $2k-1$　　B. $2k$　　C. k^2　　D. 2^k-1

28. 【2020 统考真题】对于任意一棵高度为 5 且有 10 个结点的二叉树，若采用顺序存储结构保存，每个结点占 1 个存储单元（仅存放结点的数据信息），则存放该二叉树需要的存储单元数量至少是（　）。

A. 31　　B. 16　　C. 15　　D. 10

29. 【2022 统考真题】若三叉树 T 中有 244 个结点（叶结点的高度为 1），则 T 的高度至少是（　）。

A. 8　　B. 7　　C. 6　　D. 5

二、综合应用题

01. 在一棵完全二叉树中，含有 n_0 个叶结点，当度为 1 的结点数为 1 时，该树的高度是多少？当度为 1 的结点数为 0 时，该树的高度是多少？

02. 一棵有 n 个结点的满二叉树有多少个分支结点和多少个叶结点？该满二叉树的高度是多少？

03. 已知完全二叉树的第 9 层有 240 个结点，则整个完全二叉树有多少个结点？有多少个叶结点？

04. 一棵高度为 h 的满 m 叉树有如下性质：根结点所在层次为第 1 层，第 h 层上的结点都是叶结点，其余各层上每个结点都有 m 棵非空子树，若按层次自顶向下，同一层自左向右，顺序从 1 开始对全部结点进行编号，试问：

1）各层的结点个数是多少？

2）编号为 i 的结点的双亲结点（若存在）的编号是多少？

3）编号为 i 的结点的第 k 个孩子结点（若存在）的编号是多少？

4）编号为 i 的结点有右兄弟的条件是什么？其右兄弟结点的编号是多少？

05. 已知一棵二叉树按顺序存储结构进行存储，设计一个算法，求编号分别为 i 和 j 的两个结点的最近的公共祖先结点的值。

06. 【2016 统考真题】若一棵非空 k（$k\geqslant 2$）叉树 T 中的每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题并给出推导过程。

1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个？

2）若 T 的高度为 h（单结点的树 $h=1$），则 T 的结点数最多为多少个？最少为多少个？

5.2.4 答案与解析

一、单项选择题

01. C

在二叉树中，若某个结点只有一个孩子，则这个孩子的左右次序是确定的；而在度为 2 的有序树中，若某个结点只有一个孩子，则这个孩子就无须区分其左右次序，选项 A 错误。选项 B 仅当是完全二叉树时才有意义，对于任意一棵二叉树，高度可能为 $\lfloor \log_2 n \rfloor + 1 \sim n$。在完全二叉树中，若有度为 1 的结点，则只可能有一个，且该结点只有左孩子而无右孩子，选项 C 正确。完全二叉树的高度为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$，也可通过举例 $n=4$ 来排除，选项 D 错误。

02. C

“二叉树为空”意味着二叉树中没有结点，但并不意味着二叉树不存在。注意，线性表可以是空表，树可以是空树，但图不能是空图（图中不能没有结点）。

03. A

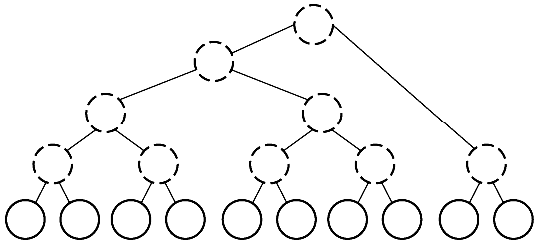
在完全二叉树中，叶结点的双亲的左兄弟的孩子一定在其前面（且一定存在），所以双亲的左兄弟（若存在）一定不是叶结点，选项 A 正确。n_0 应等于 n_2+1，选项 B 错误。完全二叉树和满二叉树均可以采用顺序存储结构，选项 C 错误。第 i 个结点的左孩子不一定存在，选项 D 错误。

选项 B 的这种通用公式适用于所有二叉树，我们应能立即联想到采用特殊值代入法验证，如画一个只含 3 个结点的满二叉树的草图来验证是否满足条件。

04. B

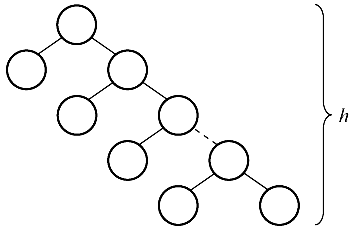
由二叉树的性质 $n_0 = n_2 + 1$，得 $n_2 = n_0 - 1 = 10 - 1 = 9$。

【另解】画出草图，如下图所示。首先画出 10 个叶结点，然后每 2 个结点向上合并，构造一个新的度为 2 的分支结点，直到构成如下图所示的二叉树，其中度为 2 的分支结点数为 9。



05. B

结点最少的情况如下图所示。除根结点层只有 1 个结点外，其他 $h-1$ 层均有两个结点，结点总数 $= 2(h-1)+1 = 2h-1$。



06. D

除根结点外，在其余 $n-1$ 个结点中，每个结点要么是其父结点的左孩子，要么是其父结点的右孩子，每个结点都有两种可能，$n-1$ 个结点共有 2^{n-1} 种不同的组合形态。

07. C

要求满足条件的树，分析可知当这 50 个结点构成一棵完全二叉树时高度最小，$h=\lfloor\log_2 n\rfloor+1=\lfloor\log_2 50\rfloor+1=6$。

【另解】第 1 层最多有 1 个结点，第 2 层最多有 2^1 个结点，第 3 层最多有 2^2 个结点，第 4 层最多有 2^3 个结点，以此类推，可以得到 h 最少为 6。

08．C

由二叉树的性质 1 可知 $n_0=n_2+1$，结点总数 $=2n=n_0+n_1+n_2=n_1+2n_2+1$，则 $n_1=2(n-n_2)-1$，所以 n_1 为奇数，说明该二叉树中不可能有 $2m$ 个度为 1 的结点。

09．C

当二叉树为单支树时具有最大高度，即每层上只有一个结点，最大高度为 1025。而当树为完全二叉树时，其高度最小，最小高度为 $\lfloor\log_2 n\rfloor+1=11$。

10．C

建议画图，第一层有 1 个结点，其余 $h-1$ 层各有 2 个结点，总结点数 $=1+2(h-1)=15$，$h=8$。

11．C

高度为 h 的完全二叉树中，第 1 层～第 $h-1$ 层构成一个高度为 $h-1$ 的满二叉树，结点个数为 $2^{h-1}-1$。第 h 层至少有一个结点，所以最少的结点个数 $=(2^{h-1}-1)+1=2^{h-1}$。

12．A

第 6 层有叶结点说明完全二叉树的高度可能为 6 或 7，显然树高为 6 时结点最少。若第 6 层上有 8 个叶结点，则前 5 层为满二叉树，所以完全二叉树的结点个数最少为 $2^5-1+8=39$ 个结点。

13．A

深度为 6 的完全二叉树，第 5 层共有 $2^4=16$ 个结点。第 6 层最左边有 3 个叶结点，其对应的双亲结点为第 5 层最左边的两个结点，所以第 5 层剩余的结点均为叶结点，共有 $16-2=14$ 个，加上第 6 层的 3 个叶结点，共有 17 个叶结点。

14．D

由完全二叉树的性质，最后一个分支结点的序号为 $\lfloor 1001/2\rfloor=500$，所以叶结点个数为 501。

【另解】$n=n_0+n_1+n_2=n_0+n_1+(n_0-1)=2n_0+n_1-1$，因为 $n=1001$，而在完全二叉树中，n_1 只能取 0 或 1。当 $n_1=1$ 时，n_0 为小数，不符合题意。所以 $n_1=0$，于是有 $n_0=501$。

15．C

要使二叉树第 7 层的结点数最多，只考虑树高为 7 层的情况，7 层满二叉树有 127 个结点，126 仅比 127 少 1 个结点，只能少在第 7 层，所以第 7 层最多有 $2^6-1=63$ 个结点。

16．B

在非空的二叉树当中，由度为 0 和 2 的结点数的关系 $n_0=n_2+1$ 可知 $n_2=123$；总结点数 $n=n_0+n_1+n_2=247+n_1$，其最大值为 248（n_1 的取值为 1 或 0，当 $n_1=1$ 时结点最多）。注意，由完全二叉树总结点数的奇偶性可以确定 n_1 的值，但不能根据 n_0 来确定 n_1 的值。

【另解】$124<2^7=128$，所以第 8 层没满，前 7 层为完全二叉树，由此可推算第 8 层可能有 120 个叶结点，第 7 层的最右 4 个为叶结点，考虑最多的情况，这 4 个叶结点中的最左边可以有 1 个左孩子（不改变叶结点数），因此结点总数 $=2^7-1+120+1=248$。

17．C

在完全二叉树中，第 4 层刚好最多有 8 个结点（前 4 层对应高度为 4 的满二叉树），若第 5 层也有 8 个结点，则对应于结点个数最多的情况，此时树高为 5，总结点数为 15 + 8 = 23。

18．B

非空指针数 = 总分支数 $=n-1$，空指针数 = 2×结点总数 − 非空指针数 $=2n-(n-1)=n+1$。

【另解】在树中，1 个指针对应 1 个分支，n 个结点的树共有 $n-1$ 个分支，即 $n-1$ 个非空指针，每个结点都有 2 个指针域，所以空指针数 $=2n-(n-1)=n+1$。

19．A

二叉链表表示的二叉树中空指针的数量为 $n+1$，三叉链表表示的二叉树多了一个根结点指向双亲的空指针，所以树中空指针的数量为 $n+2$，选项 I 正确。若根结点的度为 2，则只有左、右两个孩子指向它，选项 II 错误。若整棵树只有一个根结点，则没有指针指向它，选项 III 错误。

20．A

由完全二叉树的性质，编号为 i（$i\geqslant 1$）的结点所在的层次为 $\lfloor \log_2 i \rfloor+1$，若两个结点位于同一层，则一定有 $\lfloor \log_2 p \rfloor+1=\lfloor \log_2 q \rfloor+1$，因此有 $\lfloor \log_2 p \rfloor=\lfloor \log_2 q \rfloor$ 成立。

21．C

当根结点下标为 1 时，下标为 i 的结点的父结点下标为 $\lfloor i/2 \rfloor$，那么下标为 17 的祖先的下标有 8, 4, 2, 1，下标为 19 的祖先的下标有 9, 4, 2, 1，因此两者最近的公共祖先的下标是 4。

22．C

分析可知，满足条件的三叉树可以是完全三叉树，这棵树的第 i（$i\geqslant 1$）层最多有 3^{i-1} 个结点。设高度为 h，则 $3^0+3^1+\cdots+3^{h-1}=(3^h-1)/2$ 是结点数的上限，问题是求解 $50\leqslant(3^h-1)/2$ 的最小 h 值，即 $h\geqslant \log_3 101$，有 $h=\lceil \log_3 101 \rceil=5$。

23．B

三叉树采用三叉链表表示，每个结点均有 3 个指针域指向 3 个孩子，共有 $3n$ 个指针域，但 n 个结点构成的一棵树中只需要 $n-1$ 个指针（对于 $n-1$ 条边），因此空指针域有 $2n+1$ 个。

24．D

对于高度为 h 的满二叉树，结点总数 $n=2^0+2^1+\cdots+2^{h-1}=2^h-1$，叶结点数 $m=2^{h-1}$。

25．C

第 6 层有叶结点，完全二叉树的高度可能为 6 或 7，显然树高为 7 时结点最多。完全二叉树与满二叉树相比，只是在最下一层的右边缺少部分叶结点，而最后一层之上是个满二叉树，且只有最后两层上有叶结点。若第 6 层上有 8 个叶结点，则前 6 层为满二叉树，而第 7 层缺失 $8\times 2=16$ 个叶结点，所以完全二叉树的结点个数最多为 $2^7-1-16=111$。

26．C

最后一个分支结点的编号为 $\lfloor 768/2 \rfloor=384$，所以叶结点的个数为 $768-384=384$。

【另解】$n=n_0+n_1+n_2=n_0+n_1+(n_0-1)=2n_0+n_1-1$，其中 $n=768$，而在完全二叉树中，n_1 只能取 0 或 1，当 $n_1=0$ 时，n_0 为小数，不符合题意。因此 $n_1=1$，所以 $n_0=384$。

27．A

非叶结点的度均为 2，且所有叶结点都位于同一层的完全二叉树就是满二叉树。对于一棵高度为 h 的满二叉树（空树 $h=0$），其最后一层全部是叶结点，数目为 2^{h-1}；总结点数为 2^h-1。因此当 $2^{h-1}=k$ 时，可以得到 $2^h-1=2k-1$。

28．A

二叉树采用顺序存储时，用数组下标来表示结点之间的父子关系。对于一棵高度为 5 的二叉树，为了满足任意性，其 1～5 层的所有结点都要被存储起来，即考虑为一棵高度为 5 的满二叉树，共需要存储单元的数量为 $1+2+4+8+16=31$。

29．C

高度一定的三叉树中结点数最多的情况是满三叉树。高度为 5 的满三叉树的结点数 $=3^0+3^1+3^2+3^3+3^4=121$，高度为 6 的满三叉树的结点数 $=3^0+3^1+3^2+3^3+3^4+3^5=364$。三叉树 T 的结点数为 244，$121<244<364$，因此 T 的高度至少为 6。

二、综合应用题

01.【解答】

在非空的二叉树中，由度为 0 和度为 2 的结点之间的关系 $n_0=n_2+1$，可知 $n_2=n_0-1$。因此总结点数 $n=n_0+n_1+n_2=2n_0+n_1-1$。

①当 $n_1=1$ 时，$n=2n_0$，$h=\lceil\log_2(n+1)\rceil=\lceil\log_2(2n_0+1)\rceil$。

②当 $n_1=0$ 时，$n=2n_0-1$，$h=\lceil\log_2(n+1)\rceil=\lceil\log_2(2n_0)\rceil=\lceil\log_2(n_0)\rceil+1$。

02.【解答】

满二叉树中 $n_1=0$，由二叉树的性质 1 可知 $n_0=n_2+1$，即 $n_2=n_0-1$，$n=n_0+n_1+n_2=2n_0-1$，则 $n_0=(n+1)/2$。分支结点个数 $n_2=n-(n+1)/2=(n-1)/2$。高度为 h 的满二叉树的结点数 $n=1+2^1+2^2+\cdots+2^{h-1}=2^h-1$，即高度 $h=\log_2(n+1)$。

03.【解答】

在完全二叉树中，若第 9 层是满的，则结点数 $=2^{9-1}=256$，而现在第 9 层只有 240 个结点，说明第 9 层未满，是最后一层。1～8 层是满的，所以总结点数 $=2^8-1+240=495$。

因为第 9 层是最后一层，所以第 9 层的结点都是叶结点。且第 9 层的 240 个结点的双亲在第 8 层中，其双亲个数为 120，即第 8 层有 120 个分支结点，其余为叶结点，所以第 8 层的叶结点个数为 $2^{8-1}-120=8$。因此，总的叶结点个数 $=8+240=248$。

【另解】总结点数 $n=n_0+n_1+n_2$，$n_2=n_0-1$，$n=n_0+n_1+n_2=2n_0+n_1-1$。若 $n_1=1$，则 $2n_0+n_1-1=2n_0=495$，不符合；若 $n_1=0$，则 $2n_0+n_1-1=2n_0-1=495$，则 $n_0=248$。

注　意

对于本题，应理解完全二叉树中只有最底层的结点是不满的，其他各层的结点都是满的。

04.【解答】

1）第 1 层有 $m^0=1$ 个结点，第 2 层有 m^1 个结点，第 3 层有 m^2 个结点……一般地，第 i 层有 m^{i-1} 个结点（$1\leqslant i\leqslant h$）。

2）在 m 叉树的情形下，结点 i 的第 1 个子女编号为 $j=(i-1)m+2$，反过来，结点 i 的双亲的编号是 $\lfloor(i-2)/m\rfloor+1$，根结点没有双亲，所以要求 $i>1$。

3）因为结点 i 的第 1 个子女编号为 $(i-1)m+2$，若设该结点子女的序号为 $k=1,2,\cdots,m$，则第 k 个子女结点的编号为 $(i-1)m+k+1$（$1\leqslant k\leqslant m$）。

4）结点 i 不是其双亲的第 m 个子女时才有右兄弟。设其双亲编号为 j，可得 $j=\lfloor(i+m-2)/m\rfloor$，结点 j 的第 m 个子女的编号为 $(j-1)m+m+1=jm+1=\lfloor(i+m-2)/m\rfloor m+1$，所以当结点的编号 $i\leqslant\lfloor(i+m-2)/m\rfloor m$ 时才有右兄弟，右兄弟的编号为 $i+1$。或者，对于任意一个双亲结点 j，其第 m 个子女结点的编号是 $jm+1$，故若不为第 m 个子女结点，则 $(i-1)\%m!=0$。

05.【解答】

首先，必须明确二叉树中任意两个结点必然存在最近的公共祖先结点，最坏的情况下是根结点（两个结点分别在根结点的左右分支中），而且从最近的公共祖先结点到根结点的全部祖先结点都是公共的。由二叉树顺序存储的性质可知，任意一个结点 i 的双亲结点的编号为 $i/2$。求解 i 和 j 最近公共祖先结点的算法步骤如下（设从数组下标 1 开始存储）：

1）若 $i>j$，则结点 i 所在层次大于或等于结点 j 所在层次。结点 i 的双亲结点为结点 $i/2$，若 $i/2=j$，则结点 $i/2$ 是原结点 i 和结点 j 的最近公共祖先结点，若 $i/2\neq j$，则令 $i=i/2$，即以该结点 i 的双亲结点为起点，采用递归的方法继续查找。

2）若 $j>i$，则结点 j 所在层次大于或等于结点 i 所在层次。结点 j 的双亲结点为结点 $j/2$，若 $j/2=i$，则结点 $j/2$ 是原结点 i 和结点 j 的最近公共祖先结点，若 $j/2\neq i$，则令 $j=j/2$。重复上述过程，直到找到它们最近的公共祖先结点为止。

本题代码如下：

```
ElemType Comm_Ancestor(SqTree T,int i,int j){
//本算法在二叉树中查找结点 i 和结点 j 的最近公共祖先结点
    if(T[i]!='#'&&T[j]!='#'){          //结点存在
        while(i!=j){                   //两个编号不同时循环
            if(i>j)
                i=i/2;                 //向上找 i 的祖先
            else
                j=j/2;                 //向上找 j 的祖先
        }
        return  T[i];
    }
}
```

由解题中算法的步骤描述可知，本题也很容易地联想到采用递归的方法求解。

06.【解答】

1）正则 k 叉树中仅含有两类结点：叶结点（个数记为 n_0）和度为 k 的分支结点（个数记为 n_k）。树 T 中的结点总数 $n=n_0+n_k=n_0+m$。树中所含的边数 $e=n-1$，这些边均是从 m 个度为 k 的结点发出的，即 $e=mk$。整理得 $n_0+m=mk+1$，所以 $n_0=(k-1)m+1$。

2）高度为 h 的正则 k 叉树 T 中，含最多结点的树形为：除第 h 层外，第 1 到第 $h-1$ 层的结点都是度为 k 的分支结点；而第 h 层均为叶结点，即树是“满”树。此时第 j（$1\leqslant j\leqslant h$）层的结点数为 k^{j-1}，结点总数 M_1 为

$$M_1=\sum_{j=1}^{h}k^{j-1}=\frac{k^h-1}{k-1}$$

含最少结点的正则 k 叉树的树形为：第 1 层只有根结点，第 2 到第 $h-1$ 层仅含 1 个分支结点和 $k-1$ 个叶结点，第 h 层有 k 个叶结点。也就是说，除根外，第 2 到第 h 层中每层的结点数均为 k，所以 T 中所含结点总数 M_2 为

$$M_2=1+(h-1)k$$

5.3 二叉树的遍历和线索二叉树

5.3.1 二叉树的遍历

二叉树的遍历是指按某条搜索路径访问树中每个结点，使得每个结点均被访问一次，而且仅被访问一次。二叉树是一种非线性结构，每个结点都可能有两棵子树，因此需要寻找一种规律，以便使二叉树上的结点能排列在一个线性队列上，进而便于遍历。

命题追踪 ▶ 二叉树遍历方式的分析（2009、2011、2012）

命题追踪 ▶ （算法题）二叉树遍历的相关应用（2014、2017、2022）

由二叉树的递归定义可知，遍历一棵二叉树便要决定对根结点 N、左子树 L 和右子树 R 的访问顺序。按照先遍历左子树再遍历右子树的原则，常见的遍历次序有先序（NLR）、中序（LNR）和后序（LRN）三种遍历算法，其中序指的是根结点在何时被访问。

1．先序遍历（PreOrder）

若二叉树为空，则什么也不做；否则，公众号：小兔网盘 免费分享无水印PDF

1）访问根结点；

2）先序遍历左子树；

3）先序遍历右子树。

图 5.7 中的虚线表示对该二叉树进行先序遍历的路径，得到先序遍历序列为 124635。

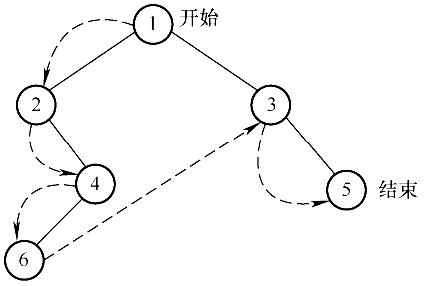


图 5.7　二叉树的先序遍历

对应的递归算法如下：

```
void PreOrder(BiTree T){
    if(T!=NULL){
        visit(T);                    //访问根结点
        PreOrder(T->lchild);         //递归遍历左子树
        PreOrder(T->rchild);         //递归遍历右子树
    }
}
```

2．中序遍历（InOrder）

若二叉树为空，则什么也不做；否则，

1）中序遍历左子树；

2）访问根结点；

3）中序遍历右子树。

命题追踪 ▶ 中序序列中结点关系的分析（2017、2024）

图 5.8 中的虚线表示对该二叉树进行中序遍历的路径，得到中序遍历序列为 264135。

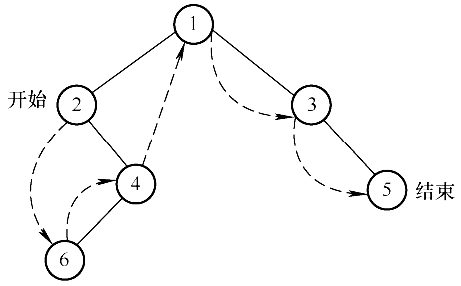


图 5.8　二叉树的中序遍历

对应的递归算法如下：

```
void InOrder(BiTree T){
    if(T!=NULL){
        InOrder(T->lchild);          //递归遍历左子树
```

```
        visit(T);                    //访问根结点
        InOrder(T->rchild);          //递归遍历右子树
    }
}
```

3．后序遍历（PostOrder）

若二叉树为空，则什么也不做；否则，

1）后序遍历左子树；

2）后序遍历右子树；

3）访问根结点。

图 5.9 中的虚线表示对该二叉树进行后序遍历的路径，得到后序遍历序列为 642531。

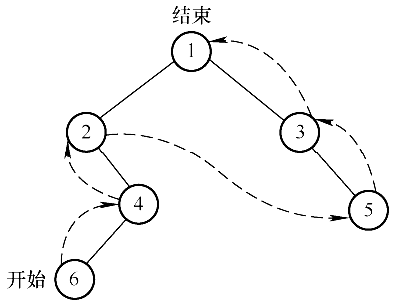


图 5.9　二叉树的后序遍历

对应的递归算法如下：

```
void PostOrder(BiTree T){
    if(T!=NULL){
        PostOrder(T->lchild);        //递归遍历左子树
        PostOrder(T->rchild);        //递归遍历右子树
        visit(T);                    //访问根结点
    }
}
```

上述三种遍历算法中，递归遍历左、右子树的顺序都是固定的，只是访问根结点的顺序不同。不管采用哪种遍历算法，每个结点都访问一次且仅访问一次，所以时间复杂度都是 $O(n)$。在递归遍历中，递归工作栈的栈深恰好为树的深度，所以在最坏情况下，二叉树是有 n 个结点且深度为 n 的单支树，遍历算法的空间复杂度为 $O(n)$。

4．层次遍历

图 5.10 所示为二叉树的层次遍历，即按照箭头所指方向，按照 1, 2, 3, 4 的层次顺序，自上而下、从左至右对二叉树中的各个结点进行逐层访问。

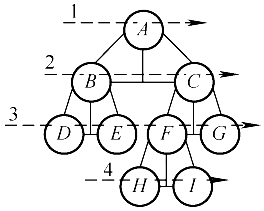


图 5.10　二叉树的层次遍历

进行层次遍历时，需要借助一个队列。层次遍历的思想如下：①首先将根结点入队。②若队

列非空，则队头结点出队，访问该结点，若它有左孩子，则将其左孩子入队；若它有右孩子，则将其右孩子入队。③重复步骤②，直至队列为空。

二叉树的层次遍历算法如下：

```
void LevelOrder(BiTree T){
    InitQueue(Q);                       //初始化辅助队列
    BiTree p;
    EnQueue(Q,T);                       //将根结点入队
    while(!IsEmpty(Q)){                 //队列不空则循环
        DeQueue(Q, p);                  //队头结点出队
        visit(p);                       //访问出队结点
        if(p->lchild!=NULL)
            EnQueue(Q,p->lchild);       //若左孩子不空，则左孩子入队
        if(p->rchild!=NULL)
            EnQueue(Q,p->rchild);       //若右孩子不空，则右孩子入队
    }
}
```

在复习过程中，读者应将上述二叉树层次遍历的算法作为一个模板，熟练掌握其执行过程，并达到熟练手写的程度。

注　意

遍历是二叉树各种操作的基础，例如对于一棵给定二叉树求结点的双亲、求结点的孩子、求二叉树的深度、求叶结点个数、判断两棵二叉树是否相同等。所有这些操作都是在遍历的过程中进行的，因此必须掌握二叉树的各种遍历过程，并能灵活运用以解决各种问题。

5．由遍历序列构造二叉树

命题追踪 ▶ 先序序列对应的不同二叉树的分析（2015）

对于一棵给定的二叉树，其先序序列、中序序列、后序序列和层序序列都是确定的。然而，只给出四种遍历序列中的任意一种，却不能唯一地确定一棵二叉树。若已知中序序列，再给出其他三种遍历序列中的任意一种，就可以唯一地确定一棵二叉树。

（1）由先序序列和中序序列构造二叉树

命题追踪 ▶ 先序序列和中序序列相同时确定的二叉树（2017）

命题追踪 ▶ 由先序序列和中序序列构造一棵二叉树（2020、2021）

在先序序列中，第一个结点一定是二叉树的根结点；而在中序遍历中，根结点必然将中序序列分割成两个子序列，前一个子序列是根的左子树的中序序列，后一个子序列是根的右子树的中序序列。左子树的中序序列和先序序列的长度是相等的，右子树的中序序列和先序序列的长度是相等的。根据这两个子序列，可以在先序序列中找到左子树的先序序列和右子树的先序序列，如图 5.11 所示。如此递归地分解下去，便能唯一地确定这棵二叉树。

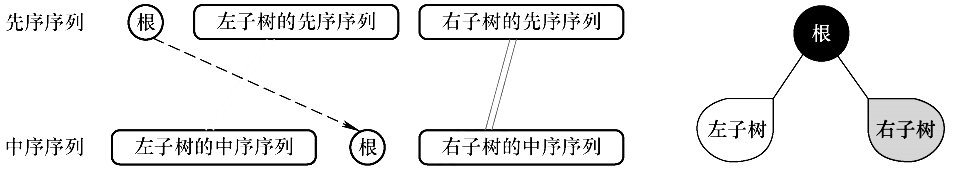


图 5.11　由先序序列和中序序列构造二叉树

例如，求先序序列（*ABCDEFGHI*）和中序序列（*BCAEDGHFI*）所确定的二叉树。首先，由先序序列可知 *A* 为二叉树的根结点。中序序列中 *A* 之前的 *BC* 为左子树的中序序列，*EDGHFI* 为右子树的中序序列。然后，由先序序列可知 *B* 是左子树的根结点，*D* 是右子树的根结点。以此类推，就能将剩下的结点继续分解下去，最后得到的二叉树如图 5.12(c)所示。

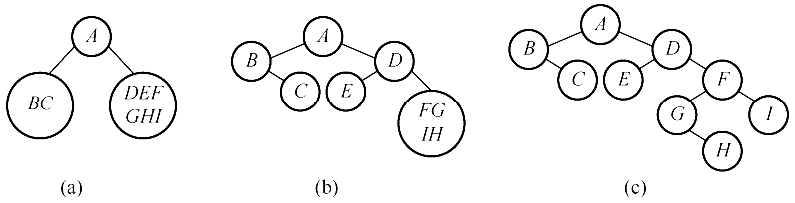


图 5.12　一棵二叉树的构造过程

（2）由后序序列和中序序列构造二叉树

命题追踪 ▶ 由后序序列和树形构造一棵二叉树（2017、2023）

同理，由二叉树的后序序列和中序序列也可以唯一地确定一棵二叉树。因为后序序列的最后一个结点就如同先序序列的第一个结点，可以将中序序列分割成两个子序列，如图 5.13 所示，然后采用类似的方法递归地进行分解，进而唯一地确定这棵二叉树。

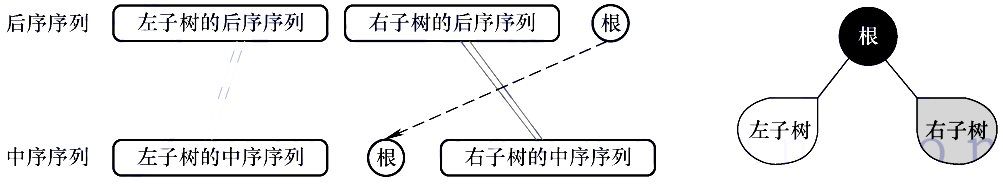


图 5.13　由后序序列和中序序列构造二叉树

请读者分析后序序列（*CBEHGIFDA*）和中序序列（*BCAEDGHFI*）所确定的二叉树。

（3）由层序序列和中序序列构造二叉树

在层序遍历中，第一个结点一定是二叉树的根结点，这样就将中序序列分割成了左子树的中序序列和右子树的中序序列。若存在左子树，则层序序列的第二个结点一定是左子树的根，可进一步划分左子树；若存在右子树，则层序序列中紧接着的下一个结点一定是右子树的根，可进一步划分右子树，如图 5.14 所示。采用这种方法继续分解，就能唯一确定这棵二叉树。

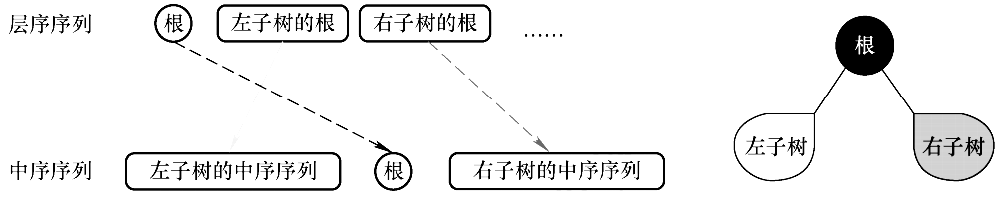


图 5.14　由层序序列和中序序列构造二叉树

请读者分析层序序列（*ABDCEFGIH*）和中序序列（*BCAEDGHFI*）所确定的二叉树。

注意，先序序列、后序序列和层序序列的两两组合，无法唯一确定一棵二叉树。例如，图 5.15 所示的两棵二叉树的先序序列都为 *AB*，后序序列都为 *BA*，层序序列都为 *AB*。

图 5.15　两棵不同的二叉树

5.3.2　线索二叉树

1．线索二叉树的基本概念

遍历二叉树是以一定的规则将二叉树中的结点排列成一个线性序列，从而得到几种遍历序列，使得该序列中的每个结点（第一个和最后一个除外）都有一个直接前驱和直接后继。

命题追踪 ▶ 线索二叉树的定义（2010）

传统的二叉链表存储仅能体现一种父子关系，不能直接得到结点在遍历中的前驱或后继。前面提到，在含 n 个结点的二叉树中，有 $n+1$ 个空指针。这是因为每个叶结点都有 2 个空指针，每个度为 1 的结点都有 1 个空指针，空指针总数为 $2n_0+n_1$，又 $n_0=n_2+1$，所以空指针总数为 $n_0+n_1+n_2+1=n+1$。由此设想能否利用这些空指针来存放指向其前驱或后继的指针？这样就可以像遍历单链表那样方便地遍历二叉树。引入线索二叉树正是为了加快查找结点前驱和后继的速度。

规定：若无左子树，令 lchild 指向其前驱结点；若无右子树，令 rchild 指向其后继结点。如图 5.16 所示，还需增加两个标志域，以标识指针域指向左（右）孩子或前驱（后继）。

lchild	ltag	data	rtag	rchild

图 5.16　线索二叉树的结点结构

其中，标志域的含义如下：

$$\texttt{ltag}=\begin{cases}0, & \texttt{lchild域指示结点的左孩子}\\ 1, & \texttt{lchild域指示结点的前驱}\end{cases}$$

$$\texttt{rtag}=\begin{cases}0, & \texttt{rchild域指示结点的右孩子}\\ 1, & \texttt{rchild域指示结点的后继}\end{cases}$$

线索二叉树的存储结构描述如下：

```
typedef struct ThreadNode{
    ElemType data;                              //数据元素
    struct ThreadNode *lchild,*rchild;          //左、右孩子指针
    int ltag,rtag;                              //左、右线索标志
}ThreadNode,*ThreadTree;
```

以这种结点结构构成的二叉链表作为二叉树的存储结构，称为*线索链表*，其中指向结点前驱和后继的指针称为*线索*。加上线索的二叉树称为*线索二叉树*。

2．中序线索二叉树的构造

二叉树的线索化是将二叉链表中的空指针改为指向前驱或后继的线索。而前驱或后继的信息只有在遍历时才能得到，因此线索化的实质就是遍历一次二叉树。

命题追踪 ▶ 中序线索二叉树中线索的指向（2014）

以中序线索二叉树的建立为例。附设指针 pre 指向刚刚访问过的结点，指针 p 指向正在访问的结点，即 pre 指向 p 的前驱。在中序遍历的过程中，检查 p 的左指针是否为空，若为空就将它指向 pre；检查 pre 的右指针是否为空，若为空就将它指向 p，如图 5.17 所示。

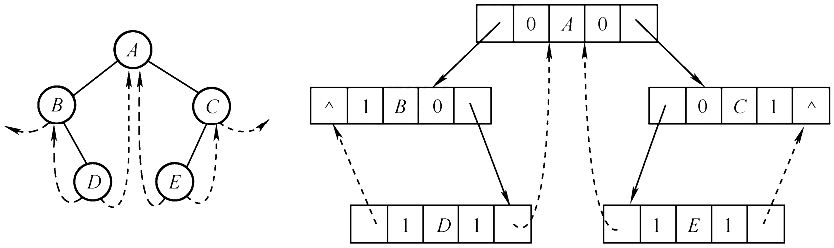


图 5.17　中序线索二叉树及其二叉链表示

通过中序遍历对二叉树线索化的递归算法如下：

```
void InThread(ThreadTree &p,ThreadTree &pre){
    if(p!=NULL){
        InThread(p->lchild,pre);        //递归，线索化左子树
        if(p->lchild==NULL){            //当前结点的左子树为空
            p->lchild=pre;              //建立当前结点的前驱线索
            p->ltag=1;
        }
        if(pre!=NULL&&pre->rchild==NULL){//前驱结点非空且其右子树为空
            pre->rchild=p;              //建立前驱结点的后继线索
            pre->rtag=1;
        }
        pre=p;                          //标记当前结点成为刚刚访问过的结点
        InThread(p->rchild,pre);        //递归，线索化右子树
    }
}
```

通过中序遍历建立中序线索二叉树的主过程算法如下：

```
void CreateInThread(ThreadTree T){
    ThreadTree pre=NULL;
    if(T!=NULL){                        //非空二叉树，线索化
        InThread(T,pre);                //线索化二叉树
        pre->rchild=NULL;               //处理遍历的最后一个结点
        pre->rtag=1;
    }
}
```

为方便起见，可在二叉树的线索链表上也添加一个头结点，令其 lchild 域的指针指向二叉树的根结点，其 rchild 域的指针指向中序遍历时访问的最后一个结点；令二叉树中序序列中的第一个结点的 lchild 域指针和最后一个结点的 rchild 域指针均指向头结点。这好比为二叉树建立了一个双向线索链表，方便从前往后或从后往前对线索二叉树进行遍历，如图 5.18 所示。

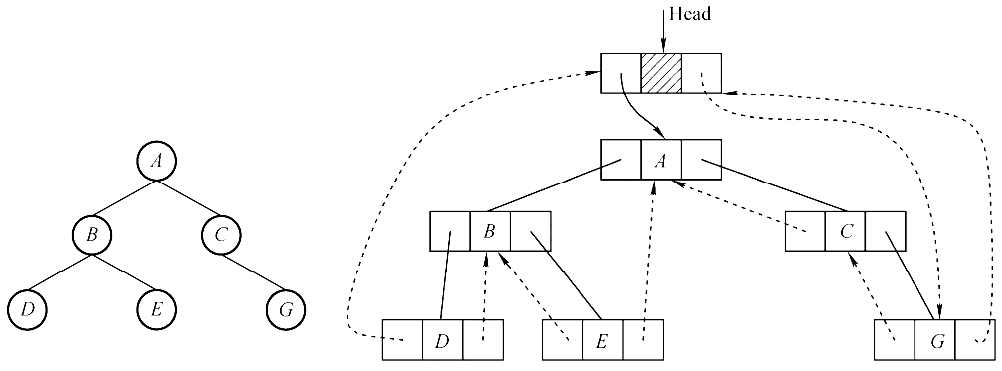


图 5.18　带头结点的中序线索二叉树

3．中序线索二叉树的遍历

中序线索二叉树的结点中隐含了线索二叉树的前驱和后继信息。在对其进行遍历时，只要先找到序列中的第一个结点，然后依次找结点的后继，直至其后继为空。在中序线索二叉树中找结点后继的规律是：若其右标志为“1”，则右链为线索，指示其后继，否则遍历右子树中第一个访问的结点（右子树中最左下的结点）为其后继。不含头结点的线索二叉树的遍历算法如下。

1）求中序线索二叉树的中序序列下的第一个结点：

```
ThreadNode *Firstnode(ThreadNode *p){
    while(p->ltag==0) p=p->lchild;  //最左下结点（不一定是叶结点）
    return p;
}
```

2）求中序线索二叉树中结点 p 在中序序列下的后继：

```
ThreadNode *Nextnode(ThreadNode *p){
    if(p->rtag==0) return Firstnode(p->rchild); //右子树中最左下结点
    else return p->rchild;          //若 rtag==1 则直接返回后继线索
}
```

请读者自行分析并完成求中序线索二叉树的最后一个结点和结点 p 前驱的运算[①]。

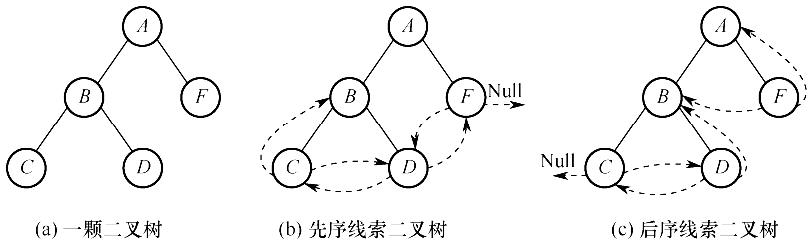
3）利用上面两个算法，可写出不含头结点的中序线索二叉树的中序遍历的算法：

```
void Inorder(ThreadNode *T){
    for(ThreadNode *p=Firstnode(T);p!=NULL; p=Nextnode(p))
        visit(p);
}
```

4．先序线索二叉树和后序线索二叉树

上面给出了建立中序线索二叉树的代码，建立先序线索二叉树和建立后序线索二叉树的代码类似，只需变动线索化改造的代码段与调用线索化左右子树递归函数的位置。

以图 5.19(a)的二叉树为例给出手工求先序线索二叉树的过程：先序序列为 *ABCDF*，然后依次判断每个结点的左右链域，若为空，则将其改造为线索。结点 *A*, *B* 均有左右孩子；结点 *C* 无左孩子，将左链域指向前驱 *B*，无右孩子，将右链域指向后继 *D*；结点 *D* 无左孩子，将左链域指向前驱 *C*，无右孩子，将右链域指向后继 *F*；结点 *F* 无左孩子，将左链域指向前驱 *D*，无右孩子，也无后继，所以置空，得到的先序线索二叉树如图 5.19(b)所示。求后序线索二叉树的过程：后序序列为 *CDBFA*，结点 *C* 无左孩子，也无前驱，所以置空，无右孩子，将右链域指向后继 *D*；结点 *D* 无左孩子，将左链域指向前驱 *C*，无右孩子，将右链域指向后继 *B*；结点 *F* 无左孩子，将左链域指向前驱 *B*，无右孩子，将右链域指向后继 *A*，得到的后序线索二叉树如图 5.19(c)所示。



(a) 一颗二叉树　(b) 先序线索二叉树　(c) 后序线索二叉树

图 5.19　先序线索二叉树和后序线索二叉树

如何在先序线索二叉树中找结点的后继？若有左孩子，则左孩子就是其后继；若无左孩子但有右孩子，则右孩子就是其后继；若为叶结点，则右链域直接指示了结点的后继。

① 将函数 1 中的 ltag 和 lchild 换成 rtag 和 rchild，即求中序线索二叉树的最后一个结点。将函数 2 中的 rtag 和 rchild 换成 ltag 和 lchild，此外调用函数改为求左子树的中序线索二叉树的最后一个结点，即求中序线索二叉树中结点 p 的前驱。

命题追踪 ▶ 后序线索二叉树中线索的指向（2013）

在后序线索二叉树中找结点的后继较为复杂，可分三种情况：①若结点 x 是二叉树的根，则其后继为空；②若结点 x 是其双亲的右孩子，或是其双亲的左孩子且其双亲没有右子树，则其后继即双亲；③若结点 x 是其双亲的左孩子，且其双亲有右子树，则其后继为双亲的右子树上按后序遍历列出的第一个结点。图 5.19(c)中找结点 B 的后继无法通过链域找到，可见在后序线索二叉树上找后继时需知道结点双亲，即需采用带标志域的三叉链表作为存储结构。

5.3.3 本节试题精选

一、单项选择题

01. 在下列关于二叉树遍历的说法中，正确的是（ ）。

A. 若有一个结点是二叉树中某个子树的中序遍历结果序列的最后一个结点，则它一定是该子树的前序遍历结果序列的最后一个结点

B. 若有一个结点是二叉树中某个子树的前序遍历结果序列的最后一个结点，则它一定是该子树的中序遍历结果序列的最后一个结点

C. 若有一个叶结点是二叉树中某个子树的中序遍历结果序列的最后一个结点，则它一定是该子树的前序遍历结果序列的最后一个结点

D. 若有一个叶结点是二叉树中某个子树的前序遍历结果序列的最后一个结点，则它一定是该子树的中序遍历结果序列的最后一个结点

02. 在任何一棵二叉树中，若结点 a 有左孩子 b、右孩子 c，则在结点的先序序列、中序序列、后序序列中，（ ）。

A. 结点 b 一定在结点 a 的前面　　B. 结点 a 一定在结点 c 的前面

C. 结点 b 一定在结点 c 的前面　　D. 结点 a 一定在结点 b 的前面

03. 设 n, m 为一棵二叉树上的两个结点，在中序遍历时，n 在 m 前的条件是（ ）。

A. n 在 m 右方　　B. n 是 m 祖先　　C. n 在 m 左方　　D. n 是 m 子孙

04. 设 n, m 为一棵二叉树上的两个结点，在后序遍历时，n 在 m 前的充分条件是（ ）。

A. n 在 m 右方　　B. n 是 m 祖先　　C. n 在 m 左方　　D. n 是 m 子孙

05. 某非空二叉树采用顺序存储结构，树中的结点信息按完全二叉树的层次序列依次存放在如下所示的一维数组中，则该二叉树的后序遍历序列为（ ）。

0	1	2	3	4	5	6	7	8	9	10	11	12
a	*b*	*c*		*d*	*e*	*f*			*g*			*h*

A. *ghbefhca*　　B. *gbdehcfa*　　C. *gdbhefca*　　D. *bgdehcfa*

06. 在二叉树的前序序列、中序序列和后序序列中，所有叶结点的先后顺序（ ）。

A. 都不相同　　B. 完全相同

C. 前序和中序相同，而与后序不同　　D. 中序和后序相同，而与前序不同

07. 对二叉树的结点从 1 开始进行连续编号，要求每个结点的编号大于其左、右孩子的编号，同一结点的左、右孩子中，其左孩子的编号小于其右孩子的编号，可采用（ ）次序的遍历实现编号。

A. 先序遍历　　B. 中序遍历　　C. 后序遍历　　D. 层次遍历

08. 按某种顺序对二叉树的结点进行编号，编号为 $1, 2, \cdots, n$，规定：树中任一结点 v，其编号等于 v 的左子树上的最小编号减 1，而 v 的右子树中的最小编号等于 v 的左子树上的

最大编号加 1，则说明该二叉树是按（　）次序编号的。

A. 中序遍历　　B. 先序遍历　　C. 后序遍历　　D. 层次遍历

09. 前序序列为 A, B, C，后序序列为 C, B, A 的二叉树共有（　）。

A. 1 棵　　B. 2 棵　　C. 3 棵　　D. 4 棵

10. 一棵完全二叉树的后序遍历序列为 *CDBFGEA*，则其先序遍历序列是（　）。

A. *CBDAFEG*　　B. *ABECDFG*　　C. *ABCDEFG*　　D. 无法确定

11. 设结点 X 和 Y 是二叉树中任意的两个结点。在该二叉树的先序遍历序列中 X 在 Y 之前，而在其后序遍历序列中 X 在 Y 之后，则 X 和 Y 的关系是（　）。

A. X 是 Y 的左兄弟　　B. X 是 Y 的右兄弟

C. X 是 Y 的祖先　　D. X 是 Y 的后裔

12. 若二叉树中结点的先序序列是 $\cdots a \cdots b \cdots$，中序序列是 $\cdots b \cdots a \cdots$，则（　）。

A. 结点 a 和结点 b 分别在某结点的左子树和右子树中

B. 结点 b 在结点 a 的右子树中

C. 结点 b 在结点 a 的左子树中

D. 结点 a 和结点 b 分别在某结点的两棵非空子树中

13. 一棵二叉树的先序遍历序列为 1234567，它的中序遍历序列可能是（　）。

A. 3124567　　B. 1234567　　C. 4135627　　D. 1463572

14. 下列序列中，不能唯一地确定一棵二叉树的是（　）。

A. 层次序列和中序序列　　B. 先序序列和中序序列

C. 后序序列和中序序列　　D. 先序序列和后序序列

15. 若一棵二叉树的中序序列和后序序列相同，则（　）。

A. 二叉树为空树或二叉树任一结点没有左子树

B. 二叉树为空树或二叉树任一结点没有右子树

C. 二叉树为空树或二叉树中每个结点的度为 1

D. 二叉树为空树或二叉树为满二叉树

16. 已知一棵二叉树的后序序列为 *DABEC*，中序序列为 *DEBAC*，则先序序列为（　）。

A. *ACBED*　　B. *DECAB*　　C. *DEABC*　　D. *CEDBA*

17. 已知一棵二叉树的先序遍历结果为 *ABCDEF*，中序遍历结果为 *CBAEDF*，则后序遍历的结果为（　）。

A. *CBEFDA*　　B. *FEDCBA*　　C. *CBEDFA*　　D. 不确定

18. 已知一棵二叉树的层次序列为 *ABCDEF*，中序序列为 *BADCFE*，则先序序列为（　）。

A. *ACBEDF*　　B. *ABCDEF*　　C. *BDFECA*　　D. *FCEDBA*

19. 某二叉树中结点 x 在先序、中序、后序遍历序列中的编号分别为 $\mathrm{pre}(x)$、$\mathrm{in}(x)$、$\mathrm{post}(x)$（假设都从 1 开始依次顺序编号），a 和 b 是该二叉树中的两个结点，其中 a 是 b 的祖先，则下列选项中不可能出现的是（　）。

A. $\mathrm{pre}(a) < \mathrm{pre}(b)$　　B. $\mathrm{post}(a) < \mathrm{post}(b)$　　C. $\mathrm{in}(a) < \mathrm{in}(b)$　　D. $\mathrm{in}(a) > \mathrm{in}(b)$

20. 某二叉树采用二叉链表存储结构，若要删除该二叉链表中的所有结点，并释放它们占用的存储空间，则采用（　）遍历方法最合适。

A. 中序　　B. 层次　　C. 后序　　D. 先序

21. 某二叉树 T 采用二叉链表存储结构，T 的中序遍历序列为一个升序序列，要求采用某种方法对 T 进行某种操作之后得到一棵新的二叉树 T'，要求 T' 的中序遍历序列为一个降序

序列，则下列关于该算法的叙述中，正确的是（ ）。

A. 采用中序遍历的方法最合适　　B. 采用后序遍历的方法最合适

C. T'中的根结点还是原 T 中的根结点　　D. T'中的叶结点不一定是原 T 中的叶结点

22. 引入线索二叉树的目的是（ ）。

A. 加快查找结点的前驱或后继的速度　　B. 为了能在二叉树中方便插入和删除

C. 为了能方便找到双亲　　D. 使二叉树的遍历结果唯一

23. 线索二叉树是一种（ ）结构。

A. 逻辑　　B. 逻辑和存储　　C. 物理　　D. 线性

24. n 个结点的线索二叉树上含有的线索数为（ ）。

A. $2n$　　B. $n-1$　　C. $n+1$　　D. n

25. 判断线索二叉树中*p结点有右孩子结点的条件是（ ）。

A. `p!=NULL`　　B. `p->rchild!=NULL`

C. `p->rtag==0`　　D. `p->rtag==1`

26. 一棵左子树为空的二叉树在先序线索化后，其中空的链域的个数是（ ）。

A. 不确定　　B. 0个　　C. 1个　　D. 2个

27. 在线索二叉树中，下列说法不正确的是（ ）。

A. 在中序线索树中，若某结点有右孩子，则其后继结点是它的右子树的最左下结点

B. 在中序线索树中，若某结点有左孩子，则其前驱结点是它的左子树的最右下结点

C. 线索二叉树是利用二叉树的 $n+1$ 个空指针来存放结点的前驱和后继信息的

D. 每个结点通过线索都可以直接找到它的前驱和后继

28. 二叉树在线索化后，仍不能有效求解的问题是（ ）。

A. 先序线索二叉树中求先序后继　　B. 中序线索二叉树中求中序后继

C. 中序线索二叉树中求中序前驱　　D. 后序线索二叉树中求后序后继

29. 若 X 是二叉中序线索树中一个有左孩子的结点，且 X 不为根，则 X 的前驱为（ ）。

A. X 的双亲　　B. X 的右子树中最左的结点

C. X 的左子树中最右的结点　　D. X 的左子树中最右的叶结点

30. 若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟 Y，则 X 的右线索指向的是（ ）。

A. X 的双亲　　B. 以 Y 为根的子树的最左下结点

C. X 的左兄弟 Y　　D. 以 Y 为根的子树的最右下结点

31. （ ）的遍历仍需要栈的支持。

A. 前序线索树　　B. 中序线索树　　C. 后序线索树　　D. 所有线索树

32. 某二叉树的先序序列和后序序列正好相反，则该二叉树一定是（ ）。

A. 空或只有一个结点　　B. 高度等于其结点数

C. 任意一个结点无左孩子　　D. 任意一个结点无右孩子

33. 某非空二叉树的先序序列和中序序列正好相反，则下列叙述中正确的是（ ）。

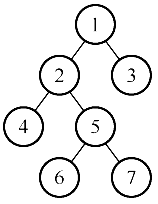
A. 该二叉树一定只有一个结点

B. 只有一个叶结点的二叉树一定满足条件

C. 任意一个结点无左孩子的二叉树一定满足条件

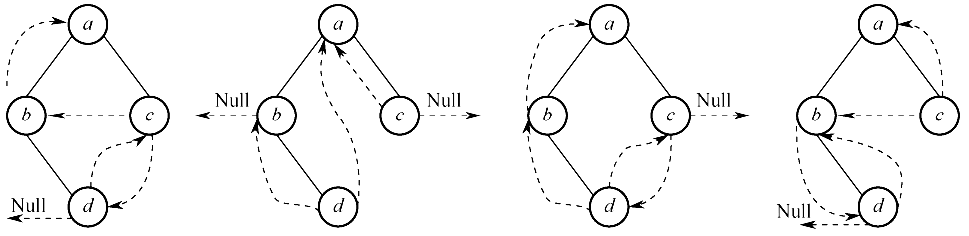
D. 任意一个结点无右孩子的二叉树一定满足条件

34. 【2009统考真题】给定二叉树如下图所示。设N代表二叉树的根，L代表根结点的左子树，R代表根结点的右子树。若遍历后的结点序列是3175624，则其遍历方式是（ ）。



A. LRN　　B. NRL　　C. RLN　　D. RNL

35. 【2010 统考真题】下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是（　）。



A.　　B.　　C.　　D.

36. 【2011 统考真题】一棵二叉树的前序遍历序列和后序遍历序列分别为 1, 2, 3, 4 和 4, 3, 2, 1，该二叉树的中序遍历序列不会是（　）。

A. 1, 2, 3, 4　　B. 2, 3, 4, 1　　C. 3, 2, 4, 1　　D. 4, 3, 2, 1

37. 【2012 统考真题】若一棵二叉树的前序遍历序列为 a, e, b, d, c，后序遍历序列为 b, c, d, e, a，则根结点的孩子结点（　）。

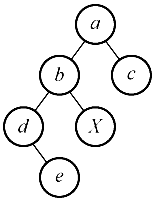
A. 只有 e　　B. 有 e、b　　C. 有 e、c　　D. 无法确定

38. 【2013 统考真题】若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟结点 Y，则 X 的右线索指向的是（　）。

A. X 的父结点　　B. 以 Y 为根的子树的最左下结点

C. X 的左兄弟结点 Y　　D. 以 Y 为根的子树的最右下结点

39. 【2014 统考真题】若对下图所示的二叉树进行中序线索化，则结点 X 的左、右线索指向的结点分别是（　）。

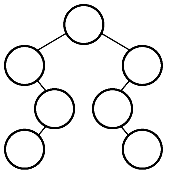


A. e, c　　B. e, a　　C. d, c　　D. b, a

40. 【2015 统考真题】先序序列为 a, b, c, d 的不同二叉树的个数是（　）。

A. 13　　B. 14　　C. 15　　D. 16

41. 【2017 统考真题】某二叉树的树形如下图所示，其后序序列为 e, a, c, b, d, g, f，树中与结点 a 同层的结点是（　）。



A. c　　B. d　　C. f　　D. g

42. 【2017 统考真题】要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须

满足的条件是（ ）。

A. 只有左子树　　B. 只有右子树

C. 结点的度均为1　　D. 结点的度均为2

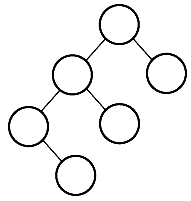
43. 【2022统考真题】若结点 p 与 q 在二叉树 T 的中序遍历序列中相邻，且 p 在 q 之前，则下列 p 与 q 的关系中，不可能的是（ ）。

I. q 是 p 的双亲　　II. q 是 p 的右孩子

III. q 是 p 的右兄弟　　IV. q 是 p 的双亲的双亲

A. 仅I　　B. 仅III　　C. 仅II、III　　D. 仅II、IV

44. 【2023统考真题】已知一棵二叉树的树形如下图所示，若其后序遍历序列为 *fdbeca*，则其先（前）序遍历序列是（ ）。



A. *aedfbc*　　B. *acebdf*　　C. *cabefd*　　D. *dfebac*

45. 【2024统考真题】若 p、q 和 v 均为二叉树 T 中的结点，v 有两个孩子结点，T 的中序遍历序列形如“…, p, v, q, …”，则在下列叙述中，正确的是（ ）。

A. p 没有右孩子，q 没有左孩子　　B. p 没有右孩子，q 有左孩子

C. p 有右孩子，q 没有左孩子　　D. p 有右孩子，q 有左孩子

二、综合应用题

01. 若某非空二叉树的先序序列和后序序列正好相反，则该二叉树的形态是什么？

02. 若某非空二叉树的先序序列和后序序列正好相同，则该二叉树的形态是什么？

03. 假设二叉树采用二叉链表存储结构，设计一个非递归算法求二叉树的高度。

04. 二叉树按二叉链表形式存储，试编写一个判别给定二叉树是否是完全二叉树的算法。

05. 假设二叉树采用二叉链表存储结构存储，试设计一个算法，计算一棵给定二叉树的所有双分支结点个数。

06. 设树 B 是一棵采用链式结构存储的二叉树，编写一个把树 B 中所有结点的左、右子树进行交换的函数。

07. 假设二叉树采用二叉链存储结构存储，设计一个算法，求先序遍历序列中第 k（$1 \leqslant k \leqslant$ 二叉树中结点个数）个结点的值。

08. 已知二叉树以二叉链表存储，编写算法完成：对于树中每个元素值为 x 的结点，删除以它为根的子树，并释放相应的空间。

09. 在二叉树中查找值为 x 的结点，试编写算法（用C语言）打印值为 x 的结点的所有祖先，假设值为 x 的结点不多于一个。

10. 设一棵二叉树的结点结构为(LLINK,INFO,RLINK)，ROOT为指向该二叉树根结点的指针，p和q分别为指向该二叉树中任意两个结点的指针，试编写算法ANCESTOR(ROOT,p,q,r)，找到p和q的最近公共祖先结点r。

11. 假设二叉树采用二叉链表存储结构，设计一个算法，求非空二叉树 b 的宽度（具有结点数最多的那一层的结点个数）。

12. 设有一棵满二叉树（所有结点值均不同），已知其先序序列为 *pre*，设计一个算法求其后序序列 *post*。

13. 设计一个算法将二叉树的叶结点按从左到右的顺序连成一个单链表，表头指针为 head。二叉树按二叉链表方式存储，链接时用叶结点的右指针域来存放单链表指针。

14. 试设计判断两棵二叉树是否相似的算法。所谓二叉树 T_1 和 T_2 相似，指的是 T_1 和 T_2 都是空的二叉树或都只有一个根结点；或者 T_1 的左子树和 T_2 的左子树是相似的，且 T_1 的右子树和 T_2 的右子树是相似的。

15. 【2014 统考真题】二叉树的带权路径长度（WPL）是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T，采用二叉链表存储，结点结构为

left	weight	right

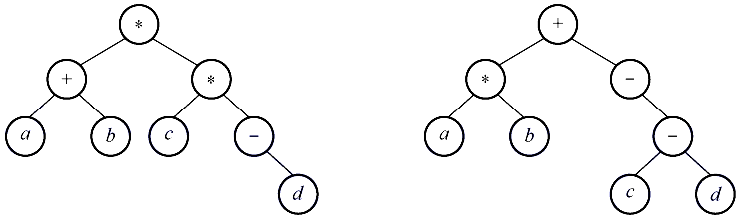
其中叶结点的 weight 域保存该结点的非负权值。设 root 为指向 T 的根结点的指针，请设计求 T 的 WPL 的算法，要求：

1）给出算法的基本设计思想。

2）使用 C 或 C++语言，给出二叉树结点的数据类型定义。

3）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

16. 【2017 统考真题】请设计一个算法，将给定的表达式树（二叉树）转换为等价的中缀表达式（通过括号反映操作符的计算次序）并输出。例如，当下列两棵表达式树作为算法的输入时：



输出的等价中缀表达式分别为 (a+b)*(c*(-d)) 和 (a*b)+(-(c-d))。

二叉树结点定义如下：

```
typedef struct node{
    char data[10];                  //存储操作数或操作符
    struct node *left, *right;
}BTree;
```

要求：

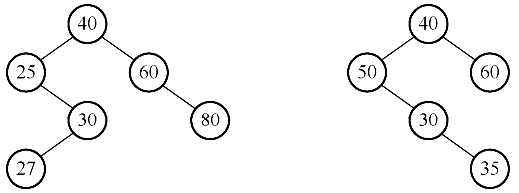
1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

17. 【2022 统考真题】已知非空二叉树 T 的结点值均为正整数，采用顺序存储方式保存，数据结构定义如下：

```
typedef struct {                    //MAX_SIZE 为已定义常量
      int SqBiTNode[MAX_SIZE];      //保存二叉树结点值的数组
      int ElemNum;                  //实际占用的数组元素个数
} SqBiTree;
```

T 中不存在的结点在数组 SqBiTNode 中用 -1 表示。例如，对于下图所示的两棵非空二叉树 T_1 和 T_2，



二叉树 T_1　　二叉树 T_2

T_1的存储结果如下：

T1.SqBiTNode	40	25	60	-1	30	-1	80	-1	-1	27		

T1.ElemNum=10

T_2的存储结果如下：

T2.SqBiTNode	40	50	60	-1	30	-1	-1	-1	-1	-1	35	

T2.ElemNum=11

请设计一个尽可能高效的算法，判定一棵采用这种方式存储的二叉树是否为二叉搜索树，若是，则返回true，否则，返回false。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用C或C++语言描述算法，关键之处给出注释。

5.3.4 答案与解析

一、单项选择题

01. C

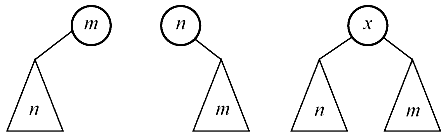
二叉树中序遍历的最后一个结点一定是从根开始沿右子女指针链走到底的结点，设用p指示。若结点p不是叶结点（其左子树非空），则前序遍历的最后一个结点在它的左子树中，选项A、B错误；若结点p是叶结点，则前序与中序遍历的最后一个结点就是它，选项C正确。若中序遍历的最后一个结点p不是叶结点，它还有一个左子女q，结点q是叶结点，那么结点q是前序遍历的最后一个结点，但不是中序遍历的最后一个结点，选项D错误。

02. C

三种遍历方式中，都先遍历左子树，再遍历右子树，因此b一定在c的前面访问。

03. C

中序遍历时，先访问左子树，再访问根结点，后访问右子树。n在m前的3种可能性如下图所示，从中看出n总是在m的左方。



【另解】设n和m的最近公共祖先p，则有以下可能：

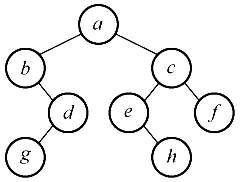
情形1，m和n分别在p的左、右（右、左）分支上；情形2，m或n为p结点，另一结点在p的分支上。只有n和m分别处于p的左、右分支上，m为祖先结点且n位于m的左分支，n为祖先结点且m位于n的右分支，符合题意。

04. D

后序遍历的顺序是LRN，若n在N的左子树，m在N的右子树，则在后序遍历的过程中n在m之前访问；若n是m的子孙，设m在N的位置，则n无论是在m的左子树还是在右子树，在后序遍历的过程中n都在m之前访问。其他都不可以。选项C要成立，就需加上两个结点位于同一层这个条件。

05. C

在二叉树的数组存储结构中，下标为i的结点的左、右孩子的下标分别为$2i+1$和$2i+2$（若存在），画出二叉树的形态如下图所示，则后序遍历序列为$gdbhefca$。



06．B

三种遍历方式中，访问左、右子树的先后顺序是不变的，只是访问根结点的顺序不同，因此叶结点的先后顺序完全相同。此外，读者可以采用特殊值法，画一个结点数为 3 的满二叉树，采用三种遍历方式来验证答案的正确性。

07．C

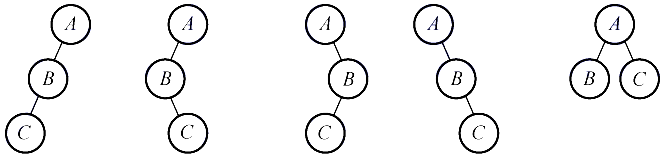
对每个顶点从 1 开始按序编号，要求结点编号大于其左、右孩子编号，并且左孩子编号小于右孩子编号。编号越大说明遍历顺序越靠后，因此，三者遍历顺序为先左子树、再右子树、后根结点。4 个选项中仅后序遍历满足要求。

08．B

结点 v 的编号比其左子树上的最小编号还小，而 v 的右子树中的最小编号大于 v 的左子树中的最大编号，因此 v 的编号比其左、右子树上的所有编号都小，显然是按先序遍历次序。

09．D

前序为 A、B、C 的不同二叉树共有 5 种，其中后序为 C、B、A 的有 4 种（前 4 种），都是单支树，如下图所示。



10．C

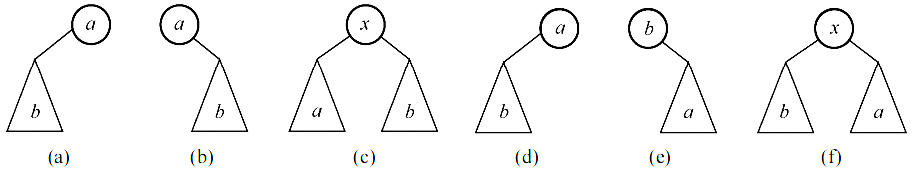
7 个结点的完全二叉树是一棵 3 层的满二叉树，画出相应二叉树的树形，根据后序遍历序列填入相应的结点，得到相应的完全二叉树，求得其先序遍历序列为 $ABCDEFG$。

11．C

二叉树的前序遍历为 NLR，后序遍历为 LRN。根据题意，在前序序列中 X 在 Y 之前，在后序序列中 X 在 Y 之后，若设 X 在根的位置，Y 在其左子树或右子树中，即满足要求。

12．C

先序序列是…a…b…，因此 a 和 b 结点的 3 种情况如下图(a)～(c)所示。中序序列是…b…a…，因此 a 和 b 结点的 3 种情况如下图(d)～(f)所示，相同部分是 b 在 a 的左子树中。



13．B

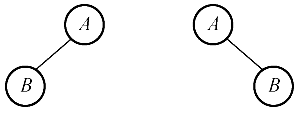
解法 1：由题可知，1 为根结点，2 为 1 的孩子。对于选项 A，3 应为 1 的左孩子，前序序列应为 13…，不符题意。类似地，选项 C 也是错误的。对于选项 B，2 为 1 的右孩子，3 为 2 的右孩子……满足题意。对于选项 D，463572 应为 1 的右子树，2 为 1 的右孩子，46357 为 2 的左子树，3 为 2 的左孩子，46 为 3 的左子树，57 为 3 的右子树，前序序列 4、6 应相连，5、7 应相连，不符题意。

解法 2：前序遍历时需要借助栈。二叉树的前序序列和中序序列的关系相当于以前序序列作为入栈次序，以中序序列作为出栈次序。题中以 1234567 入栈；对于选项 A，第一个出栈的是 3，所以 1 不可能在 2 之前出栈，错误。对于选项 C，1 不可能在 3 之前出栈，错误。对于选项 D，6 第三个出栈，此时栈顶元素是 5，不是 3，错误。选项 B 正确。

解法 3：因前序序列和中序序列可以确定一棵二叉树，所以可试着用题目中的序列构造出相应的二叉树，即可得知，只有选项 B 的序列可以构造出二叉树。

14．D

先序序列为 NLR，后序序列为 LRN，虽然可以唯一确定树的根结点，但无法划分左、右子树。例如，先序序列为 *AB*，后序序列为 *BA*，则其对应的二叉树如下图所示。

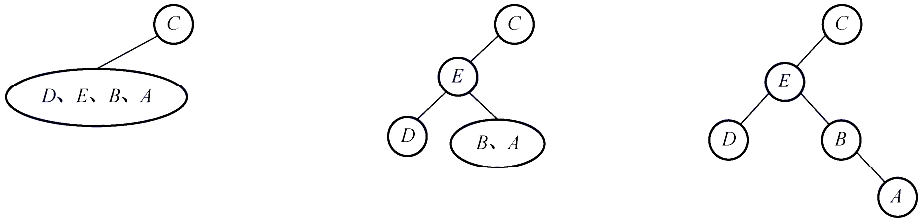


15．B

中序遍历是“左根右”，后序遍历是“左右根”，当任一结点没有右子树时，两种遍历都是“左根”。显然，当二叉树为空树或只有根结点时，其中序序列和后序序列也相同。

16．D

根据后序序列与中序序列可构造出二叉树，如下图所示。由图可知先序序列为 *CEDBA*。

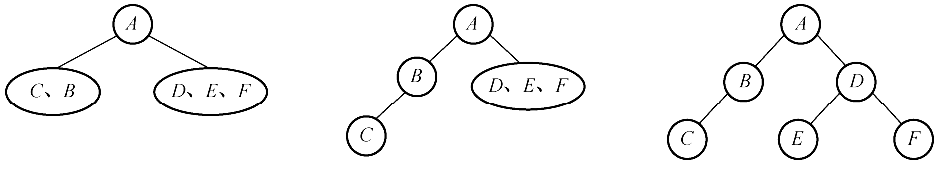


(a) 确定根结点　(b) 确定左子树根结点　(c) 确定剩下的子树

17．A

对于这种遍历序列问题，先根据遍历的性质排除若干项，若还无法确定答案，则再根据遍历结果得到二叉树，找到对应遍历序列。例如，在本题中，已知先序和中序遍历结果，可知本树的根结点为 *A*，左子树有 *C* 和 *B*，其余为右子树，则后序遍历结果中，*A* 一定在最后，并且 *C* 和 *B* 一定在前面，排除选项 B 和 D。又因先序中有 *DEF*，中序中有 *EDF*，则 *D* 为这个子树的根，所以 *D* 在后序中排在 *EF* 之后。

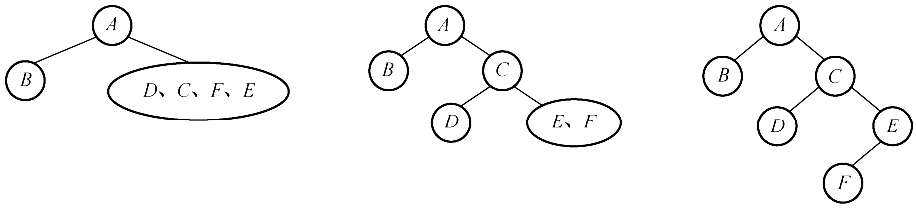
根据二叉树的递归定义，要确定二叉树，就要分别找到根结点和左、右子树。因此，根据遍历结果，必定要确定根结点位置和如何划分左、右子树，才可以确定最终的二叉树。因此，仅有先序和后序遍历不能唯一确定一棵二叉树，而二者之一加上中序遍历都可以唯一确定一棵二叉树。如在本题中，根据先序和中序遍历的结果确定二叉树的过程如下图所示。



(a) 确定根结点　(b) 确定左子树　(c) 确定右子树

18．B

可构造出二叉树如下图所示。因此，先序序列为 *ABCDEF*。



(a) 确定根结点　　(b) 确定右子树的根结点　　(c) 确定剩下的子树

19．B

先序遍历是根左右，祖先 a 先于子孙 b 访问，即 pre(a) < pre(b)，因此选项 A 一定成立。后序遍历是左右根，子孙 b 先于祖先 a 访问，即 post(b) < post(a)，因此选项 B 一定不成立。中序遍历是左根右，左子树中的子孙先于祖先访问，右子树中的子孙后于祖先访问，因此子孙的编号可能小于祖先的编号，也可能大于祖先的编号，选项 C 和 D 都有可能。

20．C

删除一个结点时，需要先递归地删除它的左、右孩子，并释放它们所占的存储空间，然后删除该结点，并删除它所占的存储空间，这正好和后序遍历的访问顺序相吻合。

21．B

只要交换 T 中所有分支结点的左右子树，就能得到一棵中序遍历序列为降序序列的树，而这并不会改变根结点，叶结点也仅仅交换位置，仍是原 T 中的叶结点，选项 C、D 错误。交换 T 中所有分支结点的左右子树，要么先处理根结点，然后递归地处理左右子树，即先序遍历；要么先处理左右子树，然后处理根结点，即后序遍历；中序遍历是不适合的。选项 A 错误，选项 B 正确。

22．A

线索是前驱结点和后继结点的指针，引入线索的目的是加快对二叉树的遍历。

23．C

二叉树是一种逻辑结构，但线索二叉树是加上线索后的链表结构，即它是二叉树在计算机内部的一种存储结构，所以是一种物理结构。

24．C

n 个结点共有链域指针 $2n$ 个，其中，除根结点外，每个结点都被一个指针指向。剩余的链域建立线索，共 $2n-(n-1)=n+1$ 个线索。

25．C

线索二叉树中用 `ltag/rtag` 标识结点的左/右指针域是否为线索，其值为 1 时，对应指针域为线索，其值为 0 时，对应指针域为左/右孩子。

26．D

对左子树为空的二叉树进行先序线索化，根结点的左子树为空并且也没有前驱结点（先遍历根结点），先序遍历的最后一个元素为叶结点，左、右子树均为空且有前驱无后继结点，所以线索化后，树中空链域有 2 个。

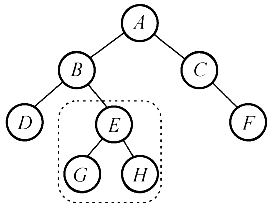
27．D

不是每个结点通过线索都可以直接找到它的前驱和后继。在先序线索二叉树中查找一个结点的先序后继很简单，而查找先序前驱必须知道该结点的双亲结点。同样，在后序线索二叉树中查找一个结点的后序前驱也很简单，而查找后序后继也必须知道该结点的双亲结点，二叉链表中没有存放双亲的指针。

28．D

后序线索二叉树不能有效解决求后序后继的问题。如下图所示，结点 E 的右指针指向右孩子，

而在后序序列中 E 的后继结点为 B，在查找 E 的后继时仍然只能按常规方法来查找。



29．C

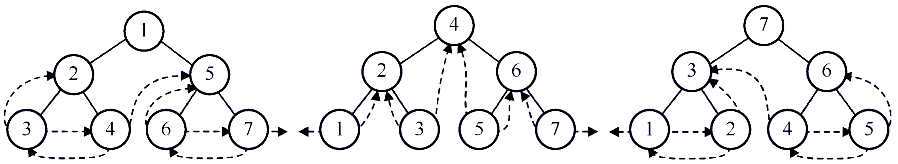
在二叉中序线索树中，某结点若有左孩子，则按照中序“左根右”的顺序，该结点的前驱结点为左子树中最右的一个结点（注意，并不一定是最右叶结点）。

30．A

在二叉树的后序遍历中，叶结点 X 的后继是其双亲，因此 X 的右线索应指向该结点。

31．C

后序线索树遍历时，最后访问根结点，若从右孩子 x 返回访问父结点，则结点 x 的右孩子不一定为空（右指针无法指向其后继），因此通过指针可能无法遍历整棵树。如下图所示，结点中的数字表示遍历的顺序，图(c)中结点 6 的右指针指向其右孩子 5，而不指向其后序后继结点 7，因此后序遍历还需要栈的支持，而图(a)和图(b)均可遍历。



32．B

非空二叉树的先序序列和后序序列相反，即“根左右”与“左右根”顺序相反，因此树只有根结点，或根结点只有左子树或右子树，其子树也有同样的性质，任意结点只有一个孩子，才能满足先序序列和后序序列正好相反。此时树形应为一个长链，树中仅有一个叶结点。

33．D

非空二叉树的先序序列和中序序列相反，即“根左右”与“左根右”顺序相反，因此树只有根结点，或任意一个结点只有左孩子，此时树形应该是一棵向左倾斜的单支树，这棵单支树只有一个叶结点。但是，只有一个叶结点的二叉树不能保证任意一个结点无右孩子。

34．D

分析遍历后的结点序列，可以看出根结点是在中间被访问的，而且右子树结点在左子树之前，则遍历的方法是 RNL。本题考查的遍历方法并不是二叉树遍历的 3 种基本遍历方法，对于考生而言，重要的是掌握遍历的思想。

35．D

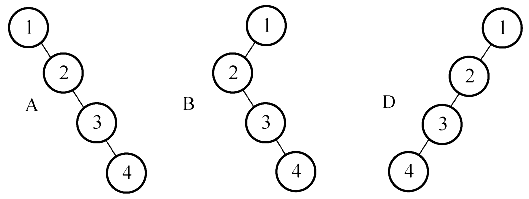
题中所给二叉树的后序序列为 $dbca$。结点 d 无前驱和左子树，左链域空，无右子树，右链域指向其后继结点 b；结点 b 无左子树，左链域指向其前驱结点 d；结点 c 无左子树，左链域指向其前驱结点 b，无右子树，右链域指向其后继结点 a。

36．C

前序序列为 NLR，后序序列为 LRN，因为前序序列和后序序列刚好相反，所以不可能存在一个结点同时有左右孩子，即二叉树的高度为 4。1 为根结点，根结点只能有左孩子（或右孩子），

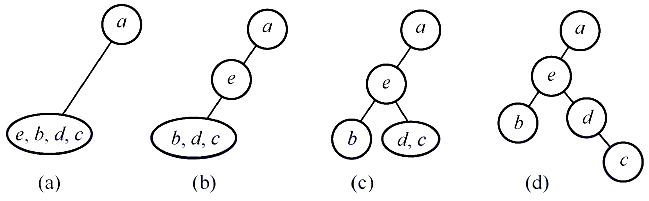
因此在中序序列中，1 或在序列首或在序列尾，选项 A、B、C、D 皆满足要求。仅考虑以 1 的孩子结点 2 为根结点的子树，它也只能有左孩子（或右孩子），因此在中序序列中，2 或在序列首或在序列尾，选项 A、B、D 皆满足要求。

【另解】画出各选项与题干信息所对应的二叉树如下，所以选项 A、B、D 均满足。



37．A

前序序列和后序序列不能唯一确定一棵二叉树，但可以确定二叉树中结点的祖先关系：当两个结点的前序序列为 *XY* 与后序序列为 *YX* 时，则 *X* 为 *Y* 的祖先。考虑前序序列 *a*, *e*, *b*, *d*, *c*、后序序列 *b*, *c*, *d*, *e*, *a*，可知 *a* 为根结点，*e* 为 *a* 的孩子结点；此外，由 *a* 的孩子结点的前序序列 *e*, *b*, *d*, *c*、后序序列 *b*, *c*, *d*, *e*，可知 *e* 是 *bcd* 的祖先，所以根结点的孩子结点只有 *e*。



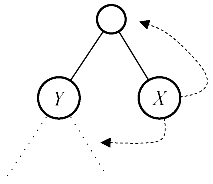
排除法：显然 *a* 为根结点，且确定 *e* 为 *a* 的孩子结点，排除选项 D。各种遍历算法中左右子树的遍历次序是固定的，若 *b* 也为 *a* 的孩子结点，则在前序序列和后序序列中 *e*、*b* 的相对次序应是不变的，所以排除选项 B，同理排除选项 C。

特殊法：前序序列和后序序列对应多棵不同的二叉树树形，我们只需画出满足该条件的任意一棵二叉树即可，任意一棵二叉树必定满足正确选项的要求。

显然选择选项 A，最终得到的二叉树满足题设中前序序列和后序序列的要求。

38．A。

根据后序线索二叉树的定义，*X* 结点为叶结点且有左兄弟，因此这个结点为右孩子结点，利用后序遍历的方式可知 *X* 结点的后序后继是其父结点，即其右线索指向的是父结点。为了更加形象，在解题的过程中可以画出如下所示的草图。



39．D

线索二叉树的线索实际上指向的是相应遍历序列特定结点的前驱结点和后继结点，所以先写出二叉树的中序遍历序列 *debxac*，中序遍历中在 *x* 左边和右边的字符，就是它在中序线索化的左、右线索，即 *b*, *a*。

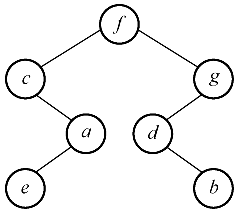
40．B

根据二叉树前序遍历和中序遍历的递归算法中递归工作栈的状态变化得出：前序序列和中序

序列的关系相当于以前序序列为入栈次序，以中序序列为出栈次序。因为前序序列和中序序列可以唯一地确定一棵二叉树，所以题意相当于“以序列 a, b, c, d 为入栈次序，则出栈序列的个数为？”，对于 n 个不同元素入栈，出栈序列的个数为 $\frac{1}{n+1}C_{2n}^{n}=14$。

41. B

后序序列先访问左子树，接着访问右子树，最后访问父结点，递归进行。根结点左子树的叶结点首先被访问，它是 e。接下来是它的父结点 a，然后是 a 的父结点 c。接着访问根结点的右子树。它的叶结点 b 首先被访问，然后是 b 的父结点 d，再后是 d 的父结点 g，最后是根结点 f，如下图所示。因此 d 与 a 同层，选项 B 正确。 公众号：小兔网盘 免费分享无水印PDF



42. B

先序序列先访问父结点，接着访问左子树，然后访问右子树。中序序列先访问左子树，接着访问父结点，然后访问右子树，递归进行。若所有非叶结点只有右子树，则先序序列和中序序列都先访问父结点，后访问右子树，递归进行。

43. B

对于此类题，每种情况只需举出一个反例即可。如图 1 所示，q 是 p 的双亲，中序遍历序列为{p, q}，选项 I 可能。如图 2 所示，q 是 p 的右孩子，中序遍历序列为{p, q}，选项 II 可能。如图 4 所示，q 是 p 的双亲的双亲，中序遍历序列为{x, p, q}，选项 IV 可能。如图 3 所示，q 是 p 的右兄弟，F 是 q 和 p 的父结点，中序遍历要求先遍历左子树，再访问根结点，最后遍历右子树，因此一定先访问 p，再访问 F，最后访问 q，p 和 q 不可能相邻出现，选项 III 不可能。

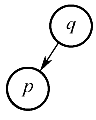


图 1

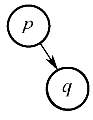


图 2

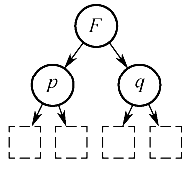


图 3

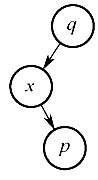
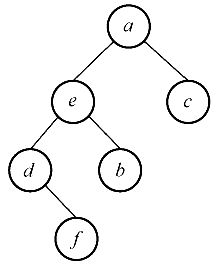


图 4

44. A

根据二叉树的树形和后序遍历序列，可以轻松地将各字母填入结点中，如下图所示。



然后对该二叉树进行先序遍历，得到序列 *aedfbc*。

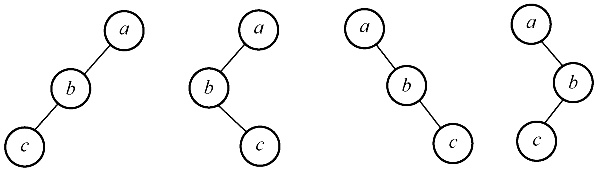
45. A

根据中序遍历的特点，v 有左右子树，以 v 为根的子树的中序序列为：v 的左子树的中序序列, v, v 的右子树的中序序列。又因为 T 的中序序列为“$\cdots, p, v, q, \cdots$”，可知 p 属于 v 的左子树，q 属于 v 的右子树。在 v 的左子树的中序序列中：假设 p 有右孩子，则 p 的右孩子在中序序列中应在 p 之后，与 p 是最后一个遍历结点矛盾，因此 p 不存在右孩子。在 v 的右子树的中序序列中：假设 q 存在左孩子，则 q 的左孩子在中序序列中应在 q 之前，与 q 是第一个遍历结点矛盾，因此 q 不存在左孩子。

二、综合应用题

01.【解答】

二叉树的先序序列是 NLR，后序序列是 LRN。要使 NLR = NRL（后序序列反序）成立，L 或 R 应为空，这样的二叉树每层只有一个结点，即二叉树的形态是其高度等于结点个数。以 3 个结点 a, b, c 为例，其形态如下图所示。



02.【解答】

二叉树的先序序列是 NLR，后序序列是 LRN。要使 NLR = LRN 成立，L 和 R 应均为空，所以满足条件的二叉树只有一个根结点。

03.【解答】

采用层次遍历的算法，设置变量 `level` 记录当前结点所在的层数，设置变量 `last` 指向当前层的最右结点，每次层次遍历出队时与 `last` 指针比较，若两者相等，则层数加 1，并让 `last` 指向下一层的最右结点，直到遍历完成。`level` 的值即二叉树的高度。

算法实现如下：

```
int Btdepth(BiTree T){
//采用层次遍历的非递归方法求解二叉树的高度
    if(!T)
        return  0;                  //树空，高度为 0
    int front=-1,rear=-1;
    int last=0,level=0;             //last 指向当前层的最右结点
    BiTree Q[MaxSize];              //设置队列 Q，元素是二叉树结点指针且容量足够
    Q[++rear]=T;                    //将根结点入队
    BiTree p;
    while(front<rear){              //队不空，则循环
            p=Q[++front];           //队列元素出队，即正在访问的结点
        if(p->lchild)
            Q[++rear]=p->lchild;//左孩子入队
        if(p->rchild)
            Q[++rear]=p->rchild;//右孩子入队
        if(front==last){            //处理该层的最右结点
                level++;            //层数增 1
            last=rear;              //last 指向下层
        }
    }
    return  level;
}
```

求某层的结点个数、每层的结点个数、树的最大宽度等，都可采用与此题类似的思想。当然，

此题可编写为递归算法，其实现如下：

```
int Btdepth2(BiTree T){
    if(T==NULL)
        return  0;                    //空树，高度为0
    ldep=Btdepth2(T->lchild);         //左子树高度
    rdep=Btdepth2(T->rchild);         //右子树高度
    if(ldep>rdep)
         return ldep+1;               //树的高度为子树最大高度加根结点
    else
         return rdep+1;
}
```

04.【解答】

根据完全二叉树的定义，具有 n 个结点的完全二叉树与满二叉树中编号从 1～n 的结点一一对应。算法思想：采用层次遍历算法，将所有结点加入队列（包括空结点）。遇到空结点时，查看其后是否有非空结点。若有，则二叉树不是完全二叉树。

算法实现如下：

```
bool IsComplete(BiTree T){
//本算法判断给定二叉树是否为完全二叉树
    InitQueue(Q);
    if(!T)
        return true;                  //空树为满二叉树
    EnQueue(Q,T);
    while(!IsEmpty(Q)){
        DeQueue(Q,p);
        if(p){                        //结点非空，将其左、右子树入队列
            EnQueue(Q,p->lchild);
            EnQueue(Q,p->rchild);
        }
        else                          //结点为空，检查其后是否有非空结点
            while(!IsEmpty(Q)){
                DeQueue(Q,p);
                if(p)                 //结点非空，则二叉树为非完全二叉树
                    return false;
            }
    }
    return true;
}
```

05.【解答】

计算一棵二叉树 b 中所有双分支结点个数的递归模型 f(b) 如下：

```
f(b)=0                                若 b=NULL
f(b)=f(b->lchild)+f(b->rchild)+1      若*b 为双分支结点
f(b)=f(b->lchild)+f(b->rchild)        其他情况（*b 为单分支结点或叶结点）
```

具体算法实现如下：

```
int DsonNodes(BiTree b){
    if(b==NULL)
        return 0;
    else if(b->lchild!=NULL&&b->rchild!=NULL)  //双分支结点
        return  DSonNodes(b->lchild)+DsonNodes(b->rchild)+1;
    else
        return  DSonNodes(b->lchild)+DsonNodes(b->rchild);
}
```

当然，本题也可以设置一个全局变量 Num，每遍历到一个结点时，判断每个结点是否为分支

结点（左、右结点都不为空，注意是双分支），若是，则 Num++。

06.【解答】

采用递归算法实现交换二叉树的左、右子树，首先交换 b 结点的左孩子的左、右子树，然后交换 b 结点的右孩子的左、右子树，最后交换 b 结点的左、右孩子，当结点为空时递归结束（后序遍历的思想）。算法实现如下：

```
void swap(BiTree b){
//本算法递归地交换二叉树的左、右子树
    if(b){
        swap(b->lchild);          //递归地交换左子树
        swap(b->rchild);          //递归地交换右子树
        temp=b->lchild;           //交换左、右孩子结点
        b->lchild=b->rchild;
        b->rchild=temp;
    }
}
```

07.【解答】

设置一个全局变量 i（初值为 1）来表示进行先序遍历时，当前访问的是第几个结点。然后可以借用先序遍历的代码模型，先序遍历二叉树。当二叉树 b 为空时，返回特殊字符'#'；当 k==i 时，该结点即要找的结点，返回 b->data；当 k≠i 时，递归地在左子树中查找，若找到则返回该值，否则继续递归地在右子树中查找，并返回其结果。对应的递归模型如下：

```
f(b,k)='#'                                              当 b=NULL 时
f(b,k)=b->data                                          当 i=k 时
f(b,k)=((ch=f(b->lchild,k))=='#'?f(b->rchild,k):ch)     其他情况
```

算法的实现如下：

```
int i=1;                                  //遍历序号的全局变量
ElemType PreNode(BiTree b,int k){
//本算法查找二叉树先序遍历序列中第 k 个结点的值
    if(b==NULL)                           //空结点，则返回特殊字符
        return '#';
    if(i==k)                              //相等，则当前结点即第 k 个结点
        return  b->data;
    i++;                                  //下一个结点
    ch=PreNode(b->lchild,k);              //左子树中递归寻找
    if(ch!='#')                           //在左子树中，则返回该值
        return ch;
    ch=PreNode(b->rchild,k);              //在右子树中递归寻找
        return ch;
}
```

本题实质上就是一个遍历算法的实现，只不过用一个全局变量来记录访问的序号，求其他遍历序列的第 k 个结点也采用相似的方法。二叉树的遍历算法可以引申出大量的算法题，因此考生务必要熟练掌握二叉树的遍历算法。

08.【解答】

删除以元素值 x 为根的子树，只要能删除其左、右子树，就可以释放值为 x 的根结点，因此宜采用后序遍历。算法思想：删除值为 x 的结点，意味着应将其父结点的左（右）子女指针置空，用层次遍历易于找到某结点的父结点。本题要求删除树中每个元素值为 x 的结点的子树，因此要遍历完整棵二叉树。算法实现如下：

```
void DeleteXTree(BiTree &bt){         //删除以 bt 为根的子树
    if(bt){
```